

exploracao-beforeIT-pt2

September 7, 2026

1 Introdução

Este exercício tem dois propósitos: - Conhecer o funcionamento da simulação do Before.IT a partir das condições iniciais da parte 1. - Entender como as modificações que foram introduzidas para transferir dinheiro da população mais pobre para a mais rica afetam a simulação.

Para isso, vamos começar importando e iniciando o modelo.

```
[35]: import BeforeIT as Bit # importando o modelo
      using DataFrames
      using Plots
      using StatsBase
      using Random

      Random.seed!(123)

      model = Bit.Model(
          Bit.AUSTRIA2010Q1.parameters, # parametros das regras necessárias para
          ↪rodar o modelo
          Bit.AUSTRIA2010Q1.initial_conditions, # condições iniciais analisadas abaixo
      );
```

2 Step de simulação e suas funções

Na implementação do Before.IT, função `Bit.step!` é responsável por rodar a simulação por um passo temporal, saindo do tempo `t` para o tempo `t+1`, sendo a unidade temporal um trimestre.

A função `Bit.step!` é composta por outras 48 funções que são chamadas sequencialmente. Iremos explorar estas funções uma por uma, explicando como elas alteram o estado do modelo.

2.1 Índice

Cada link leva à seção didática da função correspondente, na ordem de execução de `src/one_step.jl`.

Abertura do período e ambiente macroeconômico

- `Bit.finance_insolvent_firms!(model)`
- `Bit.set_growth_inflation_expectations!(model)`

- Bit.set_epsilon!(model)
- Bit.set_growth_inflation_EA!(model)
- Bit.set_central_bank_rate!(model)
- shock!(model)
- Bit.set_bank_rate!(model)

Planejamento empresarial, mercados de fatores e produção

- Bit.set_firms_expectations_and_decisions!(model)
- Bit.search_and_matching_credit!(model)
- Bit.search_and_matching_labour!(model)
- Bit.set_firms_wages!(model)
- Bit.set_firms_production!(model)
- Bit.update_workers_wages!(model)

Rendas esperadas e orçamentos das famílias

- Bit.set_gov_social_benefits!(model)
- Bit.set_bank_expected_profits!(model)
- Bit.set_households_budget_act!(model)
- Bit.set_households_budget_inact!(model)
- Bit.set_households_budget_firms!(model)
- Bit.set_households_budget_bank!(model)

Demanda pública, setor externo e mercado de bens

- Bit.set_gov_expenditure!(model)
- Bit.set_rotw_import_export!(model)
- Bit.search_and_matching!(model)

Preços realizados e estoques

- Bit.set_inflation_priceindex!(model)
- Bit.set_sector_specific_priceindex!(model)
- Bit.set_capital_formation_priceindex!(model)
- Bit.set_households_priceindex!(model)
- Bit.set_firms_stocks!(model)

Lucros, rendas e transferências realizadas

- Bit.set_firms_profits!(model)
- Bit.set_bank_profits!(model)
- Bit.set_bank_equity!(model)
- Bit.set_households_income_act!(model)
- Bit.set_households_income_inact!(model)
- Bit.set_households_income_firms!(model)
- Bit.set_households_income_bank!(model)
- Bit.set_gambling_transfers!(model)

Fechamento contábil e atualização dos balanços

- Bit.set_households_deposits_act!(model)
- Bit.set_households_deposits_inact!(model)
- Bit.set_households_deposits_firms!(model)
- Bit.set_households_deposits_bank!(model)
- Bit.set_central_bank_equity!(model)
- Bit.set_gov_revenues!(model)
- Bit.set_gov_loans!(model)
- Bit.set_firms_deposits!(model)
- Bit.set_firms_loans!(model)
- Bit.set_firms_equity!(model)
- Bit.set_rotw_deposits!(model)
- Bit.set_bank_deposits!(model)

Agregação e encerramento do período

- Bit.set_gross_domestic_product!(model)
- Bit.set_time!(model)

2.2 Refinanciamento das empresas insolventes

Objetivo econômico A função `Bit.finance_insolvent_firms!(model)` identifica empresas que possuem simultaneamente depósitos (D_i) e patrimônio líquido negativos (E_i). Para cada empresa insolvente, o banco absorve a perda, recapitaliza a empresa, zera seus depósitos e ajusta sua dívida ao valor do capital usado como garantia.

Essa operação representa a resolução da insolvência antes do início das demais atividades do trimestre. Nas condições iniciais do modelo, nenhuma empresa está insolvente e, portanto, a função não provoca alterações.

Equações Para cada empresa com $D_i < 0$ e $E_i < 0$, a recapitalização registrada é:

$$R_i = L_i - D_i - \zeta_b \bar{P}_{CF} K_i$$

e a implementação atualiza:

$$E'_k = E_k - R_i, \quad E'_i = E_i + R_i, \quad L'_i = \zeta_b \bar{P}_{CF} K_i, \quad D'_i = 0.$$

Definição das variáveis

Símbolo	Código	Significado
i	índice	Empresa analisada
D_i	<code>model.firms.D_i[i]</code>	Depósito ou posição financeira da empresa
E_i	<code>model.firms.E_i[i]</code>	Patrimônio líquido da empresa
L_i	<code>model.firms.L_i[i]</code>	Empréstimos da empresa
E_k	<code>model.bank.E_k</code>	Patrimônio líquido do banco

Símbolo	Código	Significado
K_i	<code>model.firms.K_i[i]</code>	Estoque físico de capital
$\bar{P}_{CF}$	<code>model.agg.P_bar_CF</code>	Índice de preços da formação de capital
ζ_b	<code>model.prop.zeta_b</code>	Parcela do capital preservada como garantia
R_i	expressão intermediária	Valor absorvido pelo banco na resolução

Valores desejados, esperados e realizados A função não calcula valores desejados ou esperados. Ela corrige posições patrimoniais já realizadas das empresas insolventes antes das decisões do novo período.

Campos atualizados e usos posteriores Para cada empresa insolvente, são atualizados `model.firms.D_i`, `model.firms.E_i` e `model.firms.L_i`; `model.bank.E_k` absorve a contrapartida. Esses saldos serão usados nas decisões empresariais e no mercado de crédito.

```
[2]: firms = model.firms

ids_before = findall((firms.D_i .< 0) .& (firms.E_i .< 0))
bank_equity_before = model.bank.E_k

Bit.finance_insolvent_firms!(model)

ids_after = findall((firms.D_i .< 0) .& (firms.E_i .< 0))

DataFrame(
    moment = ["before", "after"],
    insolvent_firms = [length(ids_before), length(ids_after)],
    bank_equity = [bank_equity_before, model.bank.E_k],
)
```

```
[2]:
```

	moment	insolvent_firms	bank_equity
	String	Int64	Float64
1	before	0	89460.0
2	after	0	89460.0

2.3 Formação das expectativas agregadas de crescimento e inflação

Objetivo econômico A função `Bit.set_growth_inflation_expectations!(model)` estima o PIB, o crescimento econômico e a inflação esperados para o próximo trimestre. Essa expectativa é compartilhada entre todos os agentes e será utilizada nos próximos passos.

Equações Primeiro, define-se o logaritmo do PIB:

$$y_t = \log(Y_t)$$

O valor esperado é estimado por um processo autorregressivo AR(1):

$$y_t^e = \alpha_Y y_{t-1} + \beta_Y + \varepsilon_{Y,t}$$

Convertendo novamente para o nível do PIB:

$$Y_t^e = \exp(y_t^e)$$

A taxa de crescimento esperada é:

$$\gamma_t^e = \frac{Y_t^e}{Y_{t-1}} - 1$$

Para a inflação, o modelo também estima um processo AR(1):

$$q_t^e = \alpha_\pi q_{t-1} + \beta_\pi + \varepsilon_{\pi,t}$$

$$\pi_t^e = \exp(q_t^e) - 1$$

Definição das variáveis

Símbolo	Código	Significado
Y_{t-1}	<code>model.agg.Y[model.prop.T_prime - 1]</code>	PIB realizado mais recente
y_t^e	<code>lY_e</code>	Logaritmo do PIB esperado
Y_t^e	<code>model.agg.Y_e</code>	PIB esperado
γ_t^e	<code>model.agg.gamma_e</code>	Crescimento esperado
π_t^e	<code>model.agg.pi_e</code>	Inflação esperada
α_Y, β_Y	retornos de <code>Bit.estimate</code>	Parâmetros estimados para o PIB
α_π, β_π	retornos de <code>Bit.estimate</code>	Parâmetros estimados para a inflação
$\varepsilon_{Y,t}, \varepsilon_{\pi,t}$	retornos de <code>Bit.estimate</code>	Inovações aleatórias estimadas

Valores desejados, esperados e realizados `Y_e`, `gamma_e` e `pi_e` são expectativas agregadas. A função não define valores desejados pelos agentes nem altera o PIB e a inflação realizados, armazenados nos históricos `Y` e `pi_`.

Campos atualizados e usos posteriores A função atualiza `model.agg.Y_e`, `model.agg.gamma_e` e `model.agg.pi_e`. Ela altera somente as expectativas; o PIB e a inflação efetivamente realizados ainda não são modificados.

Aleatoriedade Os parâmetros α e β são estimados a partir dos dados históricos. `Bit.estimate` sorteia separadamente os termos ε do PIB e da inflação; por isso, reexecutar a célula pode alterar as expectativas.

```
[3]: expectations_before = (
    expected_gdp = model.agg.Y_e,
    expected_growth = model.agg.gamma_e,
    expected_inflation = model.agg.pi_e,
)

Bit.set_growth_inflation_expectations!(model)

DataFrame(
    moment = ["before", "after"],
    expected_gdp = [
        expectations_before.expected_gdp,
        model.agg.Y_e,
    ],
    expected_growth = [
        expectations_before.expected_growth,
        model.agg.gamma_e,
    ],
    expected_inflation = [
        expectations_before.expected_inflation,
        model.agg.pi_e,
    ],
)
```

```
[3]:
```

	moment	expected_gdp	expected_growth	expected_inflation
	String	Float64	Float64	Float64
1	before	0.0	0.0	0.0
2	after	1.33838e5	-0.00592608	0.00223145

2.4 Como α e β são calculados?

Os parâmetros não são fixos. Eles são reestimados em cada trimestre usando todo o histórico disponível até aquele momento.

Para uma série $z_1, \dots, z_n$, o modelo estima a regressão autorregressiva:

$$z_t = \alpha z_{t-1} + \beta + u_t$$

A estimação usa os pares:

$$(z_1, z_2), (z_2, z_3), \dots, (z_{n-1}, z_n)$$

que podem ser escritos como:

$$\mathbf{y} = \begin{bmatrix} z_2 \\ z_3 \\ \vdots \\ z_n \end{bmatrix}, \quad \mathbf{X} = \begin{bmatrix} z_1 & 1 \\ z_2 & 1 \\ \vdots & \vdots \\ z_{n-1} & 1 \end{bmatrix}$$

Os valores de α e β são escolhidos para minimizar a soma dos erros quadráticos:

$$(\hat{\alpha}, \hat{\beta}) = \underset{\alpha, \beta}{\operatorname{argmin}} \sum_{t=2}^n (z_t - \alpha z_{t-1} - \beta)^2$$

Em forma matricial:

$$\begin{bmatrix} \hat{\alpha} \\ \hat{\beta} \end{bmatrix} = \mathbf{X}^+ \mathbf{y}$$

onde $\mathbf{X}^+$ é a pseudoinversa de $\mathbf{X}$, calculada no modelo por decomposição em valores singulares, ou SVD.

A interpretação dos parâmetros é:

- $\hat{\alpha}$ mede a persistência da série;
- $\hat{\beta}$ é o intercepto ou deslocamento médio;
- u_t representa a parte que a regressão não conseguiu explicar.

Após a estimação, calcula-se a variância dos resíduos:

$$\hat{\sigma}_u^2 = \operatorname{Var}(\hat{u}_t)$$

e sorteia-se um novo choque:

$$\varepsilon_t \sim \mathcal{N}(0, \hat{\sigma}_u^2)$$

A previsão seguinte é então:

$$z_{n+1}^e = \hat{\alpha} z_n + \hat{\beta} + \varepsilon_t$$

Para o PIB, o modelo aplica esse procedimento sobre $z_t = \log(Y_t)$. O mesmo processo é aplicado à série histórica de inflação.

2.5 Sorteio dos choques externos

Objetivo econômico A próxima função, `Bit.set_epsilon!(model)`, sorteia três choques aleatórios:

- $\varepsilon_{Y,EA}$: choque sobre o PIB da área do euro;
- ε_E : choque sobre a demanda por exportações;
- ε_I : choque sobre a oferta de importações.

Equações Durante a calibração, o modelo estima uma matriz de covariância $\mathbf{C}$ a partir dos resíduos históricos dessas três variáveis:

$$\mathbf{C} = \text{Cov}(\varepsilon_{Y,EA}, \varepsilon_E, \varepsilon_I)$$

Essa matriz preserva tanto a volatilidade individual de cada choque quanto suas correlações. Por exemplo, uma alteração no PIB da área do euro pode estar associada a uma alteração na demanda por exportações.

O modelo aplica a decomposição de Cholesky:

$$\mathbf{C} = \mathbf{L}\mathbf{L}^\top$$

e sorteia três números independentes de uma distribuição normal padrão:

$$\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Os choques correlacionados são obtidos por:

$$\varepsilon = \mathbf{L}\mathbf{z}$$

onde:

$$\varepsilon = \begin{bmatrix} \varepsilon_{Y,EA} \\ \varepsilon_E \\ \varepsilon_I \end{bmatrix} \sim \mathcal{N}(\mathbf{0}, \mathbf{C})$$

Definição das variáveis

Símbolo	Código	Significado
$\mathbf{C}$	<code>model.prop.C</code>	Matriz de covariância dos choques externos
$\mathbf{L}$	<code>retorno de cholesky(C)</code>	Fator triangular da decomposição de Cholesky
$\mathbf{z}$	<code>randn(3, 1)</code>	Vetor de três sorteios normais independentes
$\varepsilon_{Y,EA}$	<code>model.agg.epsilon_Y_EA</code>	Choque do PIB da área do euro
ε_E	<code>model.agg.epsilon_E</code>	Choque da demanda por exportações
ε_I	<code>model.agg.epsilon_I</code>	Choque da oferta de importações

Valores desejados, esperados e realizados Os três resultados são inovações estocásticas, não valores desejados nem resultados econômicos realizados. Eles entram como perturbações nas equações executadas posteriormente.

Campos atualizados e usos posteriores A função apenas sorteia e armazena os choques. Seus efeitos serão aplicados posteriormente ao PIB da área do euro, às exportações e às importações.

Aleatoriedade Quando `model.prop.C` é aproximadamente nula, os três choques são exatamente zero. Caso contrário, eles dependem de `randn`; reexecutar a célula pode produzir valores diferentes.

```
[4]: shocks_before = (
    gdp_euro_area = model.agg.epsilon_Y_EA,
    exports = model.agg.epsilon_E,
    imports = model.agg.epsilon_I,
)

Bit.set_epsilon!(model)

DataFrame(
    shock = [
        "PIB da área do euro",
        "Exportações",
        "Importações",
    ],
    before = [
        shocks_before.gdp_euro_area,
        shocks_before.exports,
        shocks_before.imports,
    ],
    after = [
        model.agg.epsilon_Y_EA,
        model.agg.epsilon_E,
        model.agg.epsilon_I,
    ],
)
```

```
[4]:
```

	shock	before	after
	String	Float64	Float64
1	PIB da área do euro	0.0	-0.000343733
2	Exportações	0.0	0.0127531
3	Importações	0.0	0.00277058

2.6 Atualização da economia da área do euro

Objetivo econômico A função `Bit.set_growth_inflation_EA!(model)` atualiza o PIB, o crescimento e a inflação da área do euro.

Equações O novo PIB é calculado por um processo AR(1):

$$Y_{EA,t} = \exp(\alpha_{Y,EA} \log(Y_{EA,t-1}) + \beta_{Y,EA} + \varepsilon_{Y,EA,t})$$

O crescimento realizado da área do euro é:

$$\gamma_{EA,t} = \frac{Y_{EA,t}}{Y_{EA,t-1}} - 1$$

Para a inflação, a função sorteia um choque independente:

$$\varepsilon_{\pi,EA,t} = \sigma_{\pi,EA} z_t, \quad z_t \sim \mathcal{N}(0, 1)$$

A nova inflação é calculada por:

$$\pi_{EA,t} = \exp(\alpha_{\pi,EA} \log(1 + \pi_{EA,t-1}) + \beta_{\pi,EA} + \varepsilon_{\pi,EA,t}) - 1$$

Os parâmetros α e β são estimados durante a calibração usando dados históricos trimestrais da área do euro. Diferentemente das expectativas domésticas, eles não são reestimados a cada período.

Definição das variáveis

Símbolo	Código	Significado
$Y_{EA,t-1}$	<code>model.rotw.Y_EA</code> antes da chamada	PIB anterior da área do euro
$Y_{EA,t}$	<code>model.rotw.Y_EA</code> após a chamada	Novo PIB da área do euro
$\gamma_{EA,t}$	<code>model.rotw.gamma_EA</code>	Crescimento realizado da área do euro
$\pi_{EA,t}$	<code>model.rotw.pi_EA</code>	Inflação realizada da área do euro
$\varepsilon_{Y,EA,t}$	<code>model.agg.epsilon_Y_EA</code>	Choque do PIB sorteado anteriormente
$\varepsilon_{\pi,EA,t}$	<code>randn() * model.rotw.sigma_pi_EA</code>	Choque da inflação sorteado nesta função
$\alpha_{Y,EA}, \beta_{Y,EA}$	<code>model.rotw.alpha_Y_EA, model.rotw.beta_Y_EA</code>	Parâmetros do PIB externo
$\alpha_{\pi,EA}, \beta_{\pi,EA}$	<code>model.rotw.alpha_pi_EA, model.rotw.beta_pi_EA</code>	Parâmetros da inflação externa

Valores desejados, esperados e realizados A função não calcula planos desejados. `Y_EA`, `gamma_EA` e `pi_EA` tornam-se os valores externos realizados no período; os choques representam inovações, não expectativas armazenadas.

Campos atualizados e usos posteriores São atualizados `model.rotw.Y_EA`, `model.rotw.gamma_EA` e `model.rotw.pi_EA`. Crescimento e inflação externos alimentam a regra de Taylor, enquanto o nível de atividade externa compõe a dinâmica do setor externo.

Aleatoriedade O choque $\varepsilon_{Y,EA,t}$ foi sorteado pela função anterior, enquanto o choque da inflação é sorteado nesta função.

Como a inflação externa usa um novo sorteio de `randn`, reexecutar a célula pode alterar `pi_EA` e, por consequência, as etapas seguintes.

```
[5]: euro_area_before = (
    gdp = model.rotw.Y_EA,
    growth = model.rotw.gamma_EA,
    inflation = model.rotw.pi_EA,
)

Bit.set_growth_inflation_EA!(model)

DataFrame(
    variable = ["GDP", "Growth", "Inflation"],
    before = [
        euro_area_before.gdp,
        euro_area_before.growth,
        euro_area_before.inflation,
    ],
    after = [
        model.rotw.Y_EA,
        model.rotw.gamma_EA,
        model.rotw.pi_EA,
    ],
)
```

```
[5]:
```

	variable	before	after
	String	Float64	Float64
1	GDP	2.35485e6	2.35787e6
2	Growth	0.0	0.00128357
3	Inflation	0.00193832	-0.000740652

2.7 Atualização da taxa básica de juros

Objetivo econômico A função `Bit.set_central_bank_rate!(model)` determina a taxa básica de juros do novo trimestre por meio de uma regra de Taylor.

Equações A taxa antes da restrição é:

$$\tilde{r}_t = \rho r_{t-1} + (1 - \rho) [r^* + \pi^* + \xi_\pi (\pi_{EA,t} - \pi^*) + \xi_\gamma \gamma_{EA,t}]$$

A taxa utilizada pelo modelo é limitada a valores não negativos:

$$r_t = \max(0, \tilde{r}_t)$$

Definição das variáveis

Símbolo	Código	Significado
r_{t-1}	<code>model.cb.r_bar</code> antes da chamada	Taxa básica anterior
r_t	<code>model.cb.r_bar</code> após a chamada	Taxa básica definida para o período
ρ	<code>model.cb.rho</code>	Persistência da taxa de juros
r^*	<code>model.cb.r_star</code>	Taxa real de equilíbrio
π^*	<code>model.cb.pi_star</code>	Meta trimestral de inflação
$\pi_{EA,t}$	<code>model.rotw.pi_EA</code>	Inflação realizada da área do euro
$\gamma_{EA,t}$	<code>model.rotw.gamma_EA</code>	Crescimento realizado da área do euro
ξ_π	<code>model.cb.xi_pi</code>	Reação à inflação
ξ_γ	<code>model.cb.xi_gamma</code>	Reação ao crescimento

Quanto maior ρ , mais lentamente o banco central modifica a taxa. Quando a inflação fica acima da meta, o componente associado a ξ_π tende a elevar a taxa.

Durante a calibração, estima-se inicialmente:

$$r_t = \rho r_{t-1} + \gamma_1 \pi_{EA,t} + \gamma_2 \gamma_{EA,t} + u_t$$

Os pesos da regra de Taylor são obtidos por:

$$\xi_\pi = \frac{\gamma_1}{1-\rho} \quad \text{e} \quad \xi_\gamma = \frac{\gamma_2}{1-\rho}$$

A meta trimestral corresponde a uma inflação anual de 2%:

$$\pi^* = (1 + 0.02)^{1/4} - 1$$

Valores desejados, esperados e realizados A regra produz a taxa de política efetivamente adotada no período. Ela reage a crescimento e inflação externos já realizados; não cria uma taxa desejada separada nem uma expectativa de juros.

Campos atualizados e usos posteriores A função atualiza apenas a taxa básica `model.cb.r_bar`. A taxa cobrada pelo banco comercial será atualizada posteriormente.

```
[6]: policy_rate_before = model.cb.r_bar

Bit.set_central_bank_rate!(model)

DataFrame(
  metric = [
```

```

    "Previous policy rate",
    "New policy rate",
    "Policy rate change",
],
value = [
    policy_rate_before,
    model.cb.r_bar,
    model.cb.r_bar - policy_rate_before,
],
)

```

[6]:

	metric	value
	String	Float64
1	Previous policy rate	0.00164593
2	New policy rate	0.00163333
3	Policy rate change	-1.25985e-5

2.8 Aplicação de um choque

Objetivo econômico Neste ponto, o modelo pode receber uma intervenção externa.

Equações Representando o estado atual do modelo por $\mathcal{M}_t$ e o choque por S_t , temos:

$$\mathcal{M}_t^+ = S_t(\mathcal{M}_t)$$

Na simulação padrão, utiliza-se `NoShock`, definido por:

$$S_t(\mathcal{M}_t) = \mathcal{M}_t$$

Definição das variáveis

Símbolo	Código	Significado
$\mathcal{M}_t$	<code>model</code>	Estado do modelo antes do choque
S_t	<code>shock!</code>	Função de choque escolhida para a simulação
$\mathcal{M}_t^+$	<code>model após shock!(model)</code>	Estado após a intervenção
<code>NoShock</code>	<code>Bit.NoShock()</code>	Choque padrão que não modifica o modelo

Portanto, nenhuma variável é modificada.

Esse ponto também permite aplicar cenários alternativos, como uma taxa de juros fixa, uma mudança de produtividade ou uma alteração temporária da propensão ao consumo.

O choque é aplicado depois da regra de Taylor e antes da atualização da taxa do banco comercial. Assim, um choque de juros pode substituir a taxa definida anteriormente pelo Banco Central.

Valores desejados, esperados e realizados `shock!` não cria obrigatoriamente uma categoria desejada ou esperada: ele altera diretamente os campos definidos pelo cenário. Com `NoShock`, nenhum valor é alterado.

Campos atualizados e usos posteriores Os campos atualizados dependem do tipo de choque. `InterestRateShock` pode alterar `model.cb.r_bar`, `ProductivityShock` altera `model.firms.alpha_bar_i` e `ConsumptionShock` altera `model.prop.psi`; `NoShock` não atualiza campos.

```
[7]: shock! = Bit.NoShock()

shock_result = shock!(model)

@assert shock_result === nothing
```

2.9 Atualização da taxa de juros do banco comercial

Objetivo econômico A função `Bit.set_bank_rate!(model)` calcula a taxa cobrada pelo banco comercial sobre empréstimos e posições financeiras negativas.

Equações A taxa é definida por:

$$r_t = \bar{r}_t + \mu$$

Definição das variáveis

Símbolo	Código	Significado
$\bar{r}_t$	<code>model.cb.r_bar</code>	Taxa básica definida pelo Banco Central
μ	<code>model.prop.mu</code>	Prêmio de risco do banco comercial
r_t	<code>model.bank.r</code>	Taxa cobrada pelo banco

O prêmio de risco é calculado durante a calibração como:

$$\mu = \frac{\text{juros pagos pelas empresas}}{\text{dívida total das empresas}} - \bar{r}$$

Assim, qualquer alteração da taxa básica é transmitida diretamente à taxa do banco, mantendo constante o prêmio de risco μ .

Valores desejados, esperados e realizados `r_t` é a taxa bancária vigente no período, calculada a partir da taxa básica já definida. A função não armazena separadamente uma taxa desejada ou esperada.

Campos atualizados e usos posteriores A função atualiza `model.bank.r`.

```
[8]: bank_rate_before = model.bank.r
policy_rate = model.cb.r_bar
risk_premium = model.prop.mu
expected_bank_rate = policy_rate + risk_premium

Bit.set_bank_rate!(model)

@assert isapprox(model.bank.r, expected_bank_rate)

DataFrame(
  metric = [
    "Previous bank rate",
    "Central bank policy rate",
    "Risk premium",
    "New bank rate",
  ],
  value = [
    bank_rate_before,
    policy_rate,
    risk_premium,
    model.bank.r,
  ],
)
```

```
[8]:
```

	metric	value
	String	Float64
1	Previous bank rate	0.0283599
2	Central bank policy rate	0.00163333
3	Risk premium	0.026714
4	New bank rate	0.0283473

2.10 Expectativas e decisões das empresas

Objetivo econômico A função `Bit.set_firms_expectations_and_decisions!(model)` transforma as expectativas macroeconômicas em planos individuais para cada empresa.

Ela determina:

- a quantidade que a empresa pretende vender;
- o novo preço;
- o investimento desejado;
- a necessidade de insumos;
- o emprego desejado;
- o lucro esperado;
- o valor esperado do capital e da dívida;
- o novo crédito desejado.

Esses valores são planos. O crédito, as contratações e a produção efetiva acontecem nas etapas seguintes.

Equações

Quantidade planejada A empresa parte da demanda observada anteriormente e aplica o crescimento esperado:

$$Q_i^s = Q_i^d(1 + \gamma^e)$$

Inflação de custos O índice de custo dos insumos da empresa é:

$$P_i^M = \sum_s a_{s,G_i} \bar{P}_s$$

A pressão inflacionária causada pelos custos é:

$$\pi_i^c = (1 + \tau_{SIF}) \frac{\bar{w}_i}{\bar{\alpha}_i} \left(\frac{\bar{P}_{HH}}{P_i} - 1 \right) + \frac{1}{\beta_i} \left(\frac{P_i^M}{P_i} - 1 \right) + \frac{\delta_i}{\kappa_i} \left(\frac{\bar{P}_{CF}}{P_i} - 1 \right)$$

Os três componentes representam custos de trabalho, insumos intermediários e capital.

O novo preço combina a inflação de custos da empresa com a inflação agregada esperada:

$$P_i' = P_i(1 + \pi_i^c)(1 + \pi^e)$$

Necessidade de capital, insumos e trabalhadores A produção planejada é limitada pela capacidade do capital existente:

$$\tilde{Q}_i = \min(Q_i^s, K_i \kappa_i)$$

O investimento desejado é:

$$I_i^d = \frac{\delta_i}{\kappa_i} \tilde{Q}_i$$

A quantidade desejada de insumos intermediários é:

$$DM_i^d = \frac{\tilde{Q}_i}{\beta_i}$$

O emprego desejado é:

$$N_i^d = \max \left[1, \text{round} \left(\frac{\tilde{Q}_i}{\bar{\alpha}_i} \right) \right]$$

O limite inferior igual a um significa que cada empresa deseja manter pelo menos um trabalhador.

Lucro esperado O lucro anterior é ajustado pela inflação e pelo crescimento esperados:

$$\Pi_i^e = \Pi_i(1 + \pi^e)(1 + \gamma^e)$$

Varição esperada dos depósitos A variação esperada dos depósitos considera lucro, amortização da dívida, impostos e dividendos:

$$DD_i^e = \Pi_i^e - \theta L_i - \tau_{FIRM} \max(0, \Pi_i^e) - \theta_{DIV}(1 - \tau_{FIRM}) \max(0, \Pi_i^e)$$

Impostos e dividendos são pagos somente quando o lucro esperado é positivo.

Capital e dívida esperados O valor monetário esperado do capital é:

$$K_i^e = \bar{P}_{CF}(1 + \pi^e)K_i$$

A dívida esperada depois da amortização é:

$$L_i^e = (1 - \theta)L_i$$

Crédito desejado A empresa solicita crédito quando seus depósitos atuais, somados à variação esperada, não são suficientes:

$$DL_i^d = \max(0, -DD_i^e - D_i)$$

Portanto, se:

$$D_i + DD_i^e \geq 0,$$

a empresa não solicita um novo empréstimo.

Definição das variáveis

Símbolo	Código	Significado
i	índice	Empresa analisada
s	índice	Produto ou setor fornecedor
G_i	G_i[i]	Setor econômico da empresa
Q_i^d	Q_d_i[i]	Demanda anteriormente direcionada à empresa
Q_i^s	Q_s_i[i]	Quantidade planejada para venda
γ^e	model.agg.gamma_e	Crescimento agregado esperado
π^e	model.agg.pi_e	Inflação agregada esperada

Símbolo	Código	Significado
π_i^c	pi_c_i[i]	Inflação provocada pelos custos da empresa
P_i	P_i[i]	Preço anterior da empresa
P'_i	new_P_i[i]	Novo preço da empresa
$\bar{P}_{HH}$	P_bar_HH	Índice de preços do consumo das famílias
$\bar{P}_{CF}$	P_bar_CF	Índice de preços da formação de capital
$\bar{P}_s$	P_bar_g[s]	Índice de preços do produto do setor s
a_{s,G_i}	a_sg[s, G_i]	Participação do produto s nos insumos do setor da empresa
$\bar{w}_i$	w_bar_i[i]	Salário médio de referência
$\bar{\alpha}_i$	alpha_bar_i[i]	Produtividade normal do trabalho
β_i	beta_i[i]	Produtividade dos insumos intermediários
κ_i	kappa_i[i]	Produtividade do capital
δ_i	delta_i[i]	Taxa de depreciação do capital
K_i	K_i[i]	Estoque físico de capital
I_i^d	I_d_i[i]	Investimento desejado
DM_i^d	DM_d_i[i]	Insumos intermediários desejados
N_i^d	N_d_i[i]	Número desejado de trabalhadores
Π_i	Pi_i[i]	Lucro realizado anteriormente
Π_i^e	Pi_e_i[i]	Lucro esperado
D_i	D_i[i]	Depósitos ou liquidez da empresa
DD_i^e	variável intermediária	Variação esperada dos depósitos
L_i	L_i[i]	Dívida atual
L_i^e	L_e_i[i]	Dívida esperada depois da amortização
K_i^e	K_e_i[i]	Valor esperado do capital usado como garantia
DL_i^d	DL_d_i[i]	Novo empréstimo desejado
τ_{SIF}	tau_SIF	Contribuição social paga pelo empregador
τ_{FIRM}	tau_FIRM	Imposto sobre o lucro empresarial
θ	theta	Parcela trimestral de amortização da dívida
θ_{DIV}	theta_DIV	Parcela do lucro distribuída como dividendos

Valores desejados, esperados e realizados Q_{s_i} , I_{d_i} , DM_{d_i} , N_{d_i} e DL_{d_i} são valores desejados; Pi_{e_i} , K_{e_i} e L_{e_i} são esperados. O preço P_i é redefinido nesta etapa, mas crédito, emprego, compras e produção realizados ainda não são determinados.

Campos atualizados e usos posteriores São atualizados `model.firms.Q_s_i`, `I_d_i`, `DM_d_i`, `N_d_i`, `Pi_e_i`, `DL_d_i`, `K_e_i`, `L_e_i` e `P_i`. Esses campos alimentam os mercados de crédito, trabalho e bens, além da definição de salários e da produção.

```
[9]: firms = model.firms

price_before = copy(firms.P_i)

Bit.set_firms_expectations_and_decisions!(model)

@assert all(firms.N_d_i .>= 1)
@assert all(firms.DL_d_i .>= 0)

firm_decisions = DataFrame(
    firm_id = firms.ID,
    sector = firms.G_i,
    target_quantity = firms.Q_s_i,
    price_before = price_before,
    new_price = firms.P_i,
    desired_investment = firms.I_d_i,
    desired_materials = firms.DM_d_i,
    desired_employment = firms.N_d_i,
    expected_profit = firms.Pi_e_i,
    desired_new_loans = firms.DL_d_i,
    expected_capital = firms.K_e_i,
    expected_loans = firms.L_e_i,
)

first(firm_decisions, 10)
```

```
[9]:
```

	firm_id	sector	target_quantity	price_before	new_price	desired_investment	desired_materials
	Int64	Int64	Float64	Float64	Float64	Float64	Float64
1	1	1	32.2737	1.0	1.00223	8.37302	19.4049
2	2	1	10.7579	1.0	1.00223	2.79101	6.46829
3	3	1	10.7579	1.0	1.00223	2.79101	6.46829
4	4	1	10.7579	1.0	1.00223	2.79101	6.46829
5	5	1	10.7579	1.0	1.00223	2.79101	6.46829
6	6	1	10.7579	1.0	1.00223	2.79101	6.46829
7	7	1	10.7579	1.0	1.00223	2.79101	6.46829
8	8	1	21.5158	1.0	1.00223	5.58201	12.9366
9	9	1	10.7579	1.0	1.00223	2.79101	6.46829
10	10	1	10.7579	1.0	1.00223	2.79101	6.46829

2.11 Mercado de crédito

Objetivo econômico A função `Bit.search_and_matching_credit!(model)` determina quanto crédito cada empresa efetivamente recebe.

Equações Primeiro, são selecionadas as empresas que desejam novos empréstimos:

$$\mathcal{J}_{FG} = \{i : \Delta L_i^d > 0\}$$

A ordem dessas empresas é embaralhada aleatoriamente. Em seguida, o banco avalia cada empresa de maneira sequencial.

O crédito concedido à empresa i é:

$$\Delta L_i = \max \left[0, \min \left(\Delta L_i^d, \zeta_{LTV} K_i^e - L_i^e, \frac{E_k}{\zeta} - \sum_j L_j^e - S_{\Delta L} \right) \right]$$

Essa expressão aplica três limites ao empréstimo:

1. Demanda da empresa

$$\Delta L_i \leq \Delta L_i^d$$

A empresa não recebe mais crédito do que solicitou.

2. Garantia disponível

$$L_i^e + \Delta L_i \leq \zeta_{LTV} K_i^e$$

A dívida total não pode ultrapassar uma fração do valor esperado do capital da empresa.

3. Capacidade do banco

$$\sum_j L_j^e + \sum_j \Delta L_j \leq \frac{E_k}{\zeta}$$

O total de empréstimos não pode ultrapassar a capacidade permitida pelo patrimônio do banco.

Definição das variáveis

Símbolo	Código	Significado
i	índice da empresa	Empresa avaliada
$\mathcal{J}_{FG}$	I_FG	Conjunto de empresas que solicitaram crédito
ΔL_i^d	DL_d_i[i]	Novo empréstimo desejado pela empresa

Símbolo	Código	Significado
ΔL_i	DL_i[i]	Novo empréstimo efetivamente concedido
K_i^e	K_e_i[i]	Valor esperado do capital usado como garantia
L_i^e	L_e_i[i]	Dívida esperada após a amortização programada
ζ_{LTV}	zeta_LTV	Limite da dívida em relação ao valor da garantia
E_k	E_k	Patrimônio líquido do banco
ζ	zeta	Exigência de capital próprio do banco
E_k/ζ	—	Volume máximo de crédito sustentado pelo banco
$\sum_j L_j^e$	s_L_e	Total das dívidas esperadas antes dos novos empréstimos
$S_{\Delta L}$	s_DL	Crédito novo já concedido às empresas anteriores

Valores desejados, esperados e realizados DL_d_i é o crédito desejado, enquanto K_e_i e L_e_i são valores esperados usados nos limites. DL_i é o crédito efetivamente concedido; a dívida total realizada será atualizada posteriormente.

Campos atualizados e usos posteriores A função armazena o crédito concedido em `model.firms.DL_i`. A dívida total das empresas ainda não é atualizada nesta etapa.

Aleatoriedade Como `fshuffle!` embaralha a ordem de atendimento, pode ocorrer raciocínio de crédito. Quando a capacidade do banco é insuficiente, as primeiras empresas avaliadas podem receber crédito enquanto as últimas recebem apenas uma parte ou nada; reexecutar a célula pode alterar essa distribuição.

```
[10]: firms = model.firms

desired_loans = copy(firms.DL_d_i)
collateral_headroom = max.(
    0.0,
    model.prop.zeta_LTV .* firms.K_e_i .- firms.L_e_i,
)

bank_credit_capacity = model.bank.E_k / model.prop.zeta
existing_expected_loans = sum(firms.L_e_i)
bank_credit_headroom = max(
    0.0,
    bank_credit_capacity - existing_expected_loans,
)
```

```

Bit.search_and_matching_credit!(model)

credit_allocation = DataFrame(
    firm_id = firms.ID,
    desired_loan = desired_loans,
    collateral_headroom = collateral_headroom,
    actual_loan = firms.DL_i,
)

DataFrame(
    metric = [
        "Bank credit capacity",
        "Existing expected loans",
        "Available credit before allocation",
        "Total desired loans",
        "Total granted loans",
    ],
    value = [
        bank_credit_capacity,
        existing_expected_loans,
        bank_credit_headroom,
        sum(desired_loans),
        sum(firms.DL_i),
    ],
)

```

[10]:

	metric	value
	String	Float64
1	Bank credit capacity	2.982e6
2	Existing expected loans	225073.0
3	Available credit before allocation	2.75693e6
4	Total desired loans	4104.33
5	Total granted loans	4104.33

2.12 Mercado de trabalho

Objetivo econômico A função `Bit.search_and_matching_labour!(model)` ajusta o número de trabalhadores de cada empresa por meio de demissões e contratações.

Equações Para cada empresa, calcula-se a diferença entre o emprego desejado e o emprego atual:

$$V_i = N_i^d - N_i$$

Definição das variáveis

Símbolo	Código	Significado
i	índice da empresa	Empresa analisada

Símbolo	Código	Significado
h	índice do trabalhador	Trabalhador analisado
N_i^d	$N_d_i[i]$	Número de trabalhadores desejado pela empresa
N_i	$N_i[i]$	Número atual de trabalhadores da empresa
V_i	$V_i[i]$	Número de vagas, quando positivo, ou excesso de trabalhadores, quando negativo
O_h	$O_h[h]$	Situação ocupacional do trabalhador
$\mathcal{H}_E$	H_E	Conjunto de trabalhadores empregados
$\mathcal{H}_U$	H_U	Conjunto de trabalhadores desempregados
$\mathcal{I}_V$	I_V	Conjunto de empresas com vagas abertas

A variável de ocupação é interpretada como:

$$O_h = \begin{cases} 0, & \text{se o trabalhador está desempregado,} \\ i, & \text{se o trabalhador está empregado na empresa } i. \end{cases}$$

Demissões Quando:

$$V_i < 0,$$

a empresa possui mais trabalhadores do que deseja. Os trabalhadores empregados são visitados em ordem aleatória e demitidos até que o excesso seja eliminado.

Para cada demissão:

$$O_h \leftarrow 0$$

$$N_i \leftarrow N_i - 1$$

$$V_i \leftarrow V_i + 1$$

Contratações Depois das demissões, o modelo identifica os trabalhadores desempregados:

$$\mathcal{H}_U = \{h : O_h = 0\}$$

e as empresas que ainda possuem vagas:

$$\mathcal{J}_V = \{i : V_i > 0\}$$

Para cada contratação:

$$O_h \leftarrow i$$

$$N_i \leftarrow N_i + 1$$

$$V_i \leftarrow V_i - 1$$

O processo termina quando não existem mais trabalhadores desempregados ou empresas com vagas:

$$\mathcal{H}_U = \emptyset \quad \text{ou} \quad \mathcal{J}_V = \emptyset$$

Valores desejados, esperados e realizados `N_d_i` é o emprego desejado. Depois de demissões e contratações, `N_i` e `O_h` registram o emprego e a ocupação efetivamente realizados; não há variável de emprego esperado nesta função.

Campos atualizados e usos posteriores São atualizados `model.firms.N_i` e `model.w_act.O_h`. O emprego realizado limita a produção, e a ocupação determina salários e rendas nas etapas posteriores.

O vetor V_i é temporário e não é armazenado no estado do modelo.

Aleatoriedade O pareamento é aleatório e não considera qualificação, salário, experiência ou localização. Reexecutar a célula pode mudar quais trabalhadores são desligados ou contratados.

```
[11]: firms = model.firms

employment_before = copy(firms.N_i)
occupation_before = copy(model.w_act.O_h)
desired_employment = copy(firms.N_d_i)

Bit.search_and_matching_labour!(model)

@assert sum(firms.N_i) == count(>(0), model.w_act.O_h)

firm_employment = DataFrame(
    firm_id = firms.ID,
    desired_employment = desired_employment,
    employment_before = employment_before,
    employment_after = firms.N_i,
    remaining_gap = desired_employment .- firms.N_i,
```



```
)

DataFrame(
  metric = [
    "Total employment before",
    "Total employment after",
    "Unemployed workers before",
    "Unemployed workers after",
    "Remaining vacancies",
  ],
  value = [
    sum(employment_before),
    sum(firms.N_i),
    count(==(0), occupation_before),
    count(==(0), model.w_act.0_h),
    sum(max.(0, desired_employment .- firms.N_i)),
  ],
)
```

[11]:

	metric	value
	String	Int64
1	Total employment before	3866
2	Total employment after	3864
3	Unemployed workers before	252
4	Unemployed workers after	254
5	Remaining vacancies	0

2.13 Definição dos salários pelas empresas

Objetivo econômico A função `Bit.set_firms_wages!(model)` determina o salário oferecido por cada empresa depois das contratações e demissões.

Equações Primeiro, calcula-se a produção que a empresa deseja realizar e consegue sustentar com seu capital e seus insumos:

$$\tilde{Y}_i = \min(Q_i^s, K_i \kappa_i, M_i \beta_i)$$

Em seguida, compara-se essa produção com a capacidade normal dos trabalhadores empregados:

$$m_i = \min\left(1.5, \frac{\tilde{Y}_i}{N_i \bar{\alpha}_i}\right)$$

O novo salário da empresa é:

$$w_i = \bar{w}_i m_i$$

Definição das variáveis

Símbolo	Código	Significado
i	índice da empresa	Empresa analisada
w_i	<code>w_i[i]</code>	Novo salário definido pela empresa
$\bar{w}_i$	<code>w_bar_i[i]</code>	Salário médio de referência da empresa
m_i	<code>wage_multiplier[i]</code>	Fator de ajuste aplicado ao salário de referência
$\tilde{Y}_i$	<code>feasible_output[i]</code>	Produção desejada que capital e insumos conseguem sustentar
Q_i^s	<code>Q_s_i[i]</code>	Quantidade que a empresa planeja produzir ou vender
K_i	<code>K_i[i]</code>	Estoque de capital da empresa
κ_i	<code>kappa_i[i]</code>	Produtividade do capital
$K_i \kappa_i$	<code>K_i[i] * kappa_i[i]</code>	Capacidade produtiva proporcionada pelo capital
M_i	<code>M_i[i]</code>	Estoque de insumos intermediários
β_i	<code>beta_i[i]</code>	Produtividade dos insumos intermediários
$M_i \beta_i$	<code>M_i[i] * beta_i[i]</code>	Capacidade produtiva proporcionada pelos insumos
N_i	<code>N_i[i]</code>	Número atual de trabalhadores da empresa
$\bar{\alpha}_i$	<code>alpha_bar_i[i]</code>	Produtividade normal de cada trabalhador
$N_i \bar{\alpha}_i$	—	Capacidade produtiva normal do trabalho

Interpretação do fator de ajuste Quando:

$$m_i < 1,$$

o salário fica abaixo do valor de referência porque a empresa possui capacidade de trabalho superior à produção planejada.

Quando:

$$m_i = 1,$$

o salário é igual ao valor de referência.

Quando:

$$1 < m_i \leq 1.5,$$

o salário fica acima do valor de referência. O limite de 1.5 impede que ele ultrapasse 150% do salário médio da empresa.

Valores desejados, esperados e realizados Q_{s_i} representa a produção planejada usada no cálculo. w_i é a oferta salarial definida para o período; a função não cria uma expectativa salarial separada nem altera ainda o salário individual dos trabalhadores.

Campos atualizados e usos posteriores Essa função atualiza `model.firms.w_i`. Os salários individuais dos trabalhadores, `model.w_act.w_h`, serão atualizados em uma etapa posterior.

```
[12]: firms = model.firms

wage_before = copy(firms.w_i)

feasible_output = min.(
    firms.Q_s_i,
    firms.K_i .* firms.kappa_i,
    firms.M_i .* firms.beta_i,
)

labour_capacity = firms.N_i .* firms.alpha_bar_i

wage_multiplier = min.(
    1.5,
    feasible_output ./ labour_capacity,
)

expected_wage = firms.w_bar_i .* wage_multiplier

Bit.set_firms_wages!(model)

@assert all(isapprox.(firms.w_i, expected_wage))

firm_wages = DataFrame(
    firm_id = firms.ID,
    reference_wage = firms.w_bar_i,
    wage_before = wage_before,
    feasible_output = feasible_output,
    labour_capacity = labour_capacity,
    wage_multiplier = wage_multiplier,
    new_wage = firms.w_i,
)

first(firm_wages, 10)
```

[12]:

	firm_id	reference_wage	wage_before	feasible_output	labour_capacity	wage_multiplier	
	Int64	Float64	Float64	Float64	Float64	Float64	
1	1	0.26971	0.0	32.2737	32.4661	0.994074	...
2	2	0.26971	0.0	10.7579	10.822	0.994074	...
3	3	0.26971	0.0	10.7579	10.822	0.994074	...
4	4	0.26971	0.0	10.7579	10.822	0.994074	...
5	5	0.26971	0.0	10.7579	10.822	0.994074	...
6	6	0.26971	0.0	10.7579	10.822	0.994074	...
7	7	0.26971	0.0	10.7579	10.822	0.994074	...
8	8	0.26971	0.0	21.5158	21.6441	0.994074	...
9	9	0.26971	0.0	10.7579	10.822	0.994074	...
10	10	0.26971	0.0	10.7579	10.822	0.994074	...

2.14 Produção das empresas

Objetivo econômico A função `Bit.set_firms_production!(model)` calcula quanto cada empresa efetivamente produz no trimestre.

Equações

Produtividade efetiva do trabalho Primeiro, calcula-se a produção desejada que pode ser sustentada pelo capital e pelos insumos:

$$\tilde{Y}_i = \min(Q_i^s, K_i \kappa_i, M_i \beta_i)$$

A produtividade efetiva do trabalho é:

$$\alpha_i = \bar{\alpha}_i \min\left(1.5, \frac{\tilde{Y}_i}{N_i \bar{\alpha}_i}\right)$$

O limite de 1.5 permite que a produtividade efetiva alcance no máximo 150% da produtividade normal.

Função de produção de Leontief A produção realizada é determinada pelo recurso mais restritivo:

$$Y_i = \min(Q_i^s, N_i \alpha_i, K_i \kappa_i, M_i \beta_i)$$

Essa é uma função de produção de Leontief. Trabalho, capital e insumos são complementares: uma quantidade maior de apenas um recurso não aumenta a produção quando outro recurso constitui o gargalo.

Definição das variáveis

Símbolo	Código	Significado
i	índice da empresa	Empresa analisada
Y_i	<code>Y_i[i]</code>	Produção efetivamente realizada
Q_i^s	<code>Q_s_i[i]</code>	Quantidade que a empresa planeja produzir ou vender
$\widetilde{Y}_i$	<code>feasible_output[i]</code>	Produção desejada limitada por capital e insumos
N_i	<code>N_i[i]</code>	Número de trabalhadores empregados
$\bar{\alpha}_i$	<code>alpha_bar_i[i]</code>	Produtividade normal de cada trabalhador
α_i	<code>effective_productivity[i]</code>	Produtividade efetiva de cada trabalhador
$N_i\alpha_i$	<code>labour_output[i]</code>	Capacidade produtiva do trabalho
K_i	<code>K_i[i]</code>	Estoque de capital
κ_i	<code>kappa_i[i]</code>	Produtividade do capital
$K_i\kappa_i$	<code>capital_output[i]</code>	Capacidade produtiva do capital
M_i	<code>M_i[i]</code>	Estoque de insumos intermediários
β_i	<code>beta_i[i]</code>	Produtividade dos insumos
$M_i\beta_i$	<code>material_output[i]</code>	Capacidade produtiva dos insumos

Valores desejados, esperados e realizados `Q_s_i` é a quantidade planejada, enquanto `Y_i` é a produção realizada diante das restrições de trabalho, capital e insumos. As vendas realizadas ainda serão determinadas no mercado de bens.

Campos atualizados e usos posteriores A função atualiza `model.firms.Y_i`. Essa produção ainda não corresponde às vendas: a quantidade vendida será determinada posteriormente no mercado de bens.

```
[13]: firms = model.firms

@assert all(firms.N_i .> 0)

production_before = copy(firms.Y_i)

capital_output = firms.K_i .* firms.kappa_i
material_output = firms.M_i .* firms.beta_i

feasible_output = min.(
    firms.Q_s_i,
    capital_output,
```

```

    material_output,
)

productivity_multiplier = min.(
    1.5,
    feasible_output ./ (firms.N_i .* firms.alpha_bar_i),
)

effective_productivity =
    firms.alpha_bar_i .* productivity_multiplier

labour_output = firms.N_i .* effective_productivity

expected_production = min.(
    firms.Q_s_i,
    labour_output,
    capital_output,
    material_output,
)

Bit.set_firms_production!(model)

@assert all(isapprox.(firms.Y_i, expected_production))

firm_production = DataFrame(
    firm_id = firms.ID,
    target_quantity = firms.Q_s_i,
    production_before = production_before,
    labour_output = labour_output,
    capital_output = capital_output,
    material_output = material_output,
    actual_production = firms.Y_i,
)

first(firm_production, 10)

```

[13]:

	firm_id	target_quantity	production_before	labour_output	capital_output	material_output	
	Int64	Float64	Float64	Float64	Float64	Float64	
1	1	32.2737	32.4661	32.2737	38.1954	38.1954	...
2	2	10.7579	10.822	10.7579	12.7318	12.7318	...
3	3	10.7579	10.822	10.7579	12.7318	12.7318	...
4	4	10.7579	10.822	10.7579	12.7318	12.7318	...
5	5	10.7579	10.822	10.7579	12.7318	12.7318	...
6	6	10.7579	10.822	10.7579	12.7318	12.7318	...
7	7	10.7579	10.822	10.7579	12.7318	12.7318	...
8	8	21.5158	21.6441	21.5158	25.4636	25.4636	...
9	9	10.7579	10.822	10.7579	12.7318	12.7318	...
10	10	10.7579	10.822	10.7579	12.7318	12.7318	...

2.15 Atualização dos salários dos trabalhadores

Objetivo econômico A função `Bit.update_workers_wages!(model)` transfere o salário definido pelas empresas para os trabalhadores empregados.

Equações Cada trabalhador possui uma variável de ocupação, O_h , que identifica a empresa onde ele trabalha:

$$O_h = \begin{cases} 0, & \text{se o trabalhador está desempregado,} \\ i, & \text{se o trabalhador está empregado na empresa } i. \end{cases}$$

Para cada trabalhador empregado, o salário individual passa a ser igual ao salário oferecido por sua empresa:

$$w_h^{\text{nov}} = w_{O_h}, \quad O_h \neq 0$$

Para os trabalhadores desempregados, a função não modifica o salário armazenado:

$$w_h^{\text{nov}} = w_h^{\text{anterior}}, \quad O_h = 0$$

Definição das variáveis

Símbolo	Código	Significado
h	índice do trabalhador	Trabalhador analisado
i	índice da empresa	Empresa analisada
O_h	<code>O_h[h]</code>	Empresa que emprega o trabalhador; zero indica desemprego
w_i	<code>model.firms.w_i[i]</code>	Salário oferecido pela empresa i
w_h	<code>model.w_act.w_h[h]</code>	Salário individual armazenado para o trabalhador h
w_{O_h}	<code>model.firms.w_i[O_h[h]]</code>	Salário oferecido pela empresa onde o trabalhador está empregado

Todos os trabalhadores de uma mesma empresa recebem o mesmo salário, pois o modelo não considera diferenças individuais de qualificação, experiência ou produtividade.

Valores desejados, esperados e realizados `model.firms.w_i` já contém o salário oferecido pelas empresas. A função registra esse salário como valor corrente dos trabalhadores empregados; não calcula valores desejados ou esperados.

Campos atualizados e usos posteriores A função atualiza `model.w_act.w_h`. Ela não altera o salário definido pelas empresas, `model.firms.w_i`, nem a ocupação dos trabalhadores, `model.w_act.0_h`.

```
[14]: previous_wages = copy(model.w_act.w_h)
      employers = copy(model.w_act.0_h)

      Bit.update_workers_wages!(model)

      employed = employers .!= 0
      unemployed = .!employed

      @assert all(
          model.w_act.w_h[employed] .==
          model.firms.w_i[employers[employed]]
      )

      @assert all(
          model.w_act.w_h[unemployed] .==
          previous_wages[unemployed]
      )
```

2.16 Atualização dos benefícios sociais

Objetivo econômico A função `Bit.set_gov_social_benefits!(model)` atualiza os benefícios sociais pagos pelo governo de acordo com a taxa esperada de crescimento do PIB.

Equações O benefício social geral é atualizado por:

$$sb_t^{\text{other}} = sb_{t-1}^{\text{other}} (1 + \gamma_t^e)$$

O benefício adicional destinado às pessoas inativas é atualizado por:

$$sb_t^{\text{inact}} = sb_{t-1}^{\text{inact}} (1 + \gamma_t^e)$$

Definição das variáveis

Símbolo	Código	Significado
t	<code>model.agg.t</code>	Período atual da simulação
γ_t^e	<code>model.agg.gamma_e</code>	Taxa esperada de crescimento do PIB
sb_t^{other}	<code>model.gov.sb_other</code>	Benefício social geral atualizado
sb_t^{inact}	<code>model.gov.sb_inact</code>	Benefício adicional destinado às pessoas inativas

Símbolo	Código	Significado
sb_{t-1}^{other}	<code>other_benefit_before</code>	Benefício social geral antes da atualização
sb_{t-1}^{inact}	<code>inactive_benefit_before</code>	Benefício das pessoas inativas antes da atualização

Quando $\gamma_t^e > 0$, os benefícios aumentam. Quando $\gamma_t^e < 0$, eles diminuem.

Valores desejados, esperados e realizados O crescimento `gamma_e` é esperado, mas `sb_other` e `sb_inact` tornam-se os valores correntes usados no período. A função não registra pagamentos realizados aos beneficiários.

Campos atualizados e usos posteriores As pessoas inativas recebem posteriormente tanto o benefício geral quanto o benefício específico para inativos. Essa função apenas atualiza os valores armazenados no governo; os pagamentos serão contabilizados nas etapas seguintes.

```
[15]: government = model.gov
expected_growth = model.agg.gamma_e

other_benefit_before = government.sb_other
inactive_benefit_before = government.sb_inact

expected_other_benefit =
    other_benefit_before * (1 + expected_growth)

expected_inactive_benefit =
    inactive_benefit_before * (1 + expected_growth)

Bit.set_gov_social_benefits!(model)

@assert isapprox(
    government.sb_other,
    expected_other_benefit,
)

@assert isapprox(
    government.sb_inact,
    expected_inactive_benefit,
)

DataFrame(
    benefit_type = [
        "General social benefit",
        "Inactive-person benefit",
    ],
    previous_value = [
```

```

        other_benefit_before,
        inactive_benefit_before,
    ],
    updated_value = [
        government.sb_other,
        government.sb_inact,
    ],
)

```

[15]:

	benefit_type	previous_value	updated_value
	String	Float64	Float64
1	General social benefit	0.590286	0.586788
2	Inactive-person benefit	2.23847	2.2252

2.17 Lucro esperado do banco

Objetivo econômico A função `Bit.set_bank_expected_profits!(model)` calcula o lucro que o banco espera obter no período atual.

Equações O cálculo parte do lucro realizado no período anterior e aplica dois fatores: a inflação esperada e o crescimento econômico esperado.

$$\Pi_k^e = \Pi_k (1 + \pi^e) (1 + \gamma^e)$$

Definição das variáveis

Símbolo	Código	Significado
Π_k^e	<code>model.bank.Pi_e_k</code>	Lucro esperado do banco
Π_k	<code>model.bank.Pi_k</code>	Lucro realizado pelo banco no período anterior
π^e	<code>model.agg.pi_e</code>	Taxa de inflação esperada
γ^e	<code>model.agg.gamma_e</code>	Taxa esperada de crescimento do PIB
$1 + \pi^e$	<code>1 + expected_inflation</code>	Fator de atualização dos preços
$1 + \gamma^e$	<code>1 + expected_growth</code>	Fator de atualização da atividade econômica

O termo combinado:

$$(1 + \pi^e) (1 + \gamma^e)$$

representa o crescimento nominal esperado da economia. Assim, o modelo pressupõe que o lucro do banco acompanha proporcionalmente a variação esperada dos preços e da atividade econômica.

Valores desejados, esperados e realizados Pi_e_k é o lucro esperado; Pi_k continua sendo o lucro realizado do período anterior. Nenhum lucro novo é realizado nesta etapa.

Campos atualizados e usos posteriores A função atualiza `model.bank.Pi_e_k`. Esse valor será utilizado posteriormente no cálculo da renda e do orçamento do proprietário do banco.

```
[16]: bank = model.bank

previous_profit = bank.Pi_k
expected_inflation = model.agg.pi_e
expected_growth = model.agg.gamma_e

nominal_adjustment =
    (1 + expected_inflation) * (1 + expected_growth)

expected_bank_profit =
    previous_profit * nominal_adjustment

Bit.set_bank_expected_profits!(model)

@assert isapprox(
    bank.Pi_e_k,
    expected_bank_profit,
)

DataFrame(
    previous_profit = [previous_profit],
    expected_inflation = [expected_inflation],
    expected_growth = [expected_growth],
    nominal_adjustment = [nominal_adjustment],
    expected_profit = [bank.Pi_e_k],
)
```

```
[16]:
```

	previous_profit	expected_inflation	expected_growth	nominal_adjustment	expected_profit
	Float64	Float64	Float64	Float64	Float64
1	6476.29	0.00223145	-0.00592608	0.996292	6452.28

2.18 Orçamento dos trabalhadores ativos

Objetivo econômico A função `Bit.set_households_budget_act!(model)` calcula os orçamentos desejados de consumo e investimento habitacional dos trabalhadores ativos.

O grupo de trabalhadores ativos inclui tanto os empregados quanto os desempregados.

Equações

Renda esperada dos trabalhadores empregados Para um trabalhador empregado, $O_h \neq 0$, a renda esperada é:

$$Y_h^e = [w_h (1 - \tau_{\text{SIW}} - \tau_{\text{INC}} (1 - \tau_{\text{SIW}})) + sb^{\text{other}}] \bar{P}_{HH} (1 + \pi^e)$$

Renda esperada dos trabalhadores desempregados Para um trabalhador desempregado, $O_h = 0$, a renda esperada é:

$$Y_h^e = (\theta_{\text{UB}} w_h + sb^{\text{other}}) \bar{P}_{HH} (1 + \pi^e)$$

Desconto das apostas Se o trabalhador participa do mercado de apostas, uma parcela de sua renda esperada é reservada como aposta:

$$B_h = g \max(0, Y_h^e)$$

Caso não participe:

$$B_h = 0$$

A renda disponível para a definição dos orçamentos é:

$$\tilde{Y}_h^e = Y_h^e - B_h$$

Orçamentos desejados O orçamento desejado de consumo é:

$$C_h^d = \frac{\psi \tilde{Y}_h^e}{1 + \tau_{\text{VAT}}}$$

O orçamento desejado de investimento habitacional é:

$$I_h^d = \frac{\psi_H \tilde{Y}_h^e}{1 + \tau_{\text{CF}}}$$

Definição das variáveis

Símbolo	Código	Significado
h	índice do trabalhador	Trabalhador analisado
O_h	<code>0_h[h]</code>	Empresa do trabalhador; zero indica desemprego
Y_h^e	<code>expected_income[h]</code>	Renda esperada do trabalhador
$\tilde{Y}_h^e$	<code>budget_income[h]</code>	Renda esperada depois da aposta
w_h	<code>model.w_act.w_h[h]</code>	Salário do trabalhador
τ_{SIW}	<code>model.prop.tau_SIW</code>	Alíquota de contribuição social do trabalhador

Símbolo	Código	Significado
τ_{INC}	<code>model.prop.tau_INC</code>	Alíquota do imposto de renda
θ_{UB}	<code>model.prop.theta_UB</code>	Taxa de reposição do seguro-desemprego
sb^{other}	<code>model.gov.sb_other</code>	Benefício social geral
$\bar{P}_{HH}$	<code>model.agg.P_bar_HH</code>	Índice de preços das famílias
π^e	<code>model.agg.pi_e</code>	Inflação esperada
g	<code>model.prop.gambling_income_share</code>	Parcela da renda apostada pelos participantes
B_h	<code>gambling_stakes[h]</code>	Valor reservado para apostas
ψ	<code>model.prop.psi</code>	Parcela da renda destinada ao consumo
ψ_H	<code>model.prop.psi_H</code>	Parcela da renda destinada ao investimento habitacional
τ_{VAT}	<code>model.prop.tau_VAT</code>	Alíquota do imposto sobre o consumo
τ_{CF}	<code>model.prop.tau_CF</code>	Alíquota do imposto sobre formação de capital
C_h^d	<code>model.w_act.C_d_h[h]</code>	Orçamento desejado de consumo
I_h^d	<code>model.w_act.I_d_h[h]</code>	Orçamento desejado de investimento habitacional

Valores desejados, esperados e realizados A renda usada no cálculo é esperada e líquida da aposta prevista. C_d_h e I_d_h são orçamentos desejados; consumo e investimento realizados ainda permanecem inalterados.

Campos atualizados e usos posteriores A função atualiza `model.w_act.C_d_h` e `model.w_act.I_d_h`. Ela apenas define os orçamentos desejados; as compras efetivas serão determinadas posteriormente pelo mercado de bens.

```
[17]: active_workers = model.w_act

expected_income =
    Bit.households_income_act(model; expected = true)

gambling_stakes, _, _ =
    Bit.gambling_transfers(model; income_act = expected_income)

budget_income = expected_income - gambling_stakes

expected_consumption_budget =
    model.prop.psi .* budget_income ./ (1 + model.prop.tau_VAT)

expected_investment_budget =
    model.prop.psi_H .* budget_income ./ (1 + model.prop.tau_CF)
```

```

Bit.set_households_budget_act!(model)

@assert all(isapprox.(
    active_workers.C_d_h,
    expected_consumption_budget,
))

@assert all(isapprox.(
    active_workers.I_d_h,
    expected_investment_budget,
))

active_workers_budget = DataFrame(
    worker_id = active_workers.ID,
    employer_id = active_workers.O_h,
    expected_income = expected_income,
    gambling_stake = gambling_stakes,
    budget_income = budget_income,
    consumption_budget = active_workers.C_d_h,
    investment_budget = active_workers.I_d_h,
)

first(active_workers_budget, 10)

```

[17]:

	worker_id	employer_id	expected_income	gambling_stake	budget_income	consumption_budget
	Int64	Int64	Float64	Float64	Float64	Float64
1	1	1	0.763288	0.0	0.763288	0.60227
2	2	1	0.763288	0.0	0.763288	0.60227
3	3	1	0.763288	0.0	0.763288	0.60227
4	4	2	0.763288	0.0	0.763288	0.60227
5	5	3	0.763288	0.0	0.763288	0.60227
6	6	4	0.763288	0.0	0.763288	0.60227
7	7	5	0.763288	0.0	0.763288	0.60227
8	8	6	0.763288	0.0	0.763288	0.60227
9	9	7	0.763288	0.0	0.763288	0.60227
10	10	8	0.763288	0.0	0.763288	0.60227

2.19 Orçamento das pessoas inativas

Objetivo econômico A função `Bit.set_households_budget_inact!(model)` calcula os orçamentos desejados de consumo e investimento habitacional das pessoas economicamente inativas.

Equações A renda esperada de cada pessoa inativa é formada pelo benefício específico para inativos e pelo benefício social geral:

$$Y_h^e = (sb^{\text{inact}} + sb^{\text{other}}) \bar{P}_{HH} (1 + \pi^e)$$

Desconto das apostas Se a pessoa inativa participa do mercado de apostas, uma parcela de sua renda esperada é reservada como aposta:

$$B_h = g \max(0, Y_h^e)$$

Caso não participe:

$$B_h = 0$$

A renda disponível para a definição dos orçamentos é:

$$\widetilde{Y}_h^e = Y_h^e - B_h$$

Orçamentos desejados O orçamento desejado de consumo é:

$$C_h^d = \frac{\psi \widetilde{Y}_h^e}{1 + \tau_{\text{VAT}}}$$

O orçamento desejado de investimento habitacional é:

$$I_h^d = \frac{\psi_H \widetilde{Y}_h^e}{1 + \tau_{\text{CF}}}$$

Definição das variáveis

Símbolo	Código	Significado
h	índice da pessoa inativa	Pessoa analisada
Y_h^e	<code>expected_income[h]</code>	Renda esperada
$\widetilde{Y}_h^e$	<code>budget_income[h]</code>	Renda esperada depois da aposta
sb^{inact}	<code>model.gov.sb_inact</code>	Benefício específico para pessoas inativas
sb^{other}	<code>model.gov.sb_other</code>	Benefício social geral
$\bar{P}_{HH}$	<code>model.agg.P_bar_HH</code>	Índice de preços das famílias
π^e	<code>model.agg.pi_e</code>	Taxa de inflação esperada
g	<code>model.prop.gambling_income_share</code>	Parcela da renda apostada pelos participantes
B_h	<code>gambling_stakes[h]</code>	Valor reservado para apostas
ψ	<code>model.prop.psi</code>	Parcela da renda destinada ao consumo
ψ_H	<code>model.prop.psi_H</code>	Parcela da renda destinada ao investimento habitacional
τ_{VAT}	<code>model.prop.tau_VAT</code>	Alíquota do imposto sobre consumo

Símbolo	Código	Significado
τ_{CF}	<code>model.prop.tau_CF</code>	Alíquota do imposto sobre formação de capital
C_h^d	<code>model.w_inact.C_d_h[h]</code>	Orçamento desejado de consumo
I_h^d	<code>model.w_inact.I_d_h[h]</code>	Orçamento desejado de investimento habitacional

Antes das apostas, todas as pessoas inativas recebem a mesma renda esperada no modelo. Diferenças no orçamento podem surgir quando apenas algumas delas participam do mercado de apostas.

Valores desejados, esperados e realizados A renda usada no cálculo é esperada e líquida da aposta prevista. `C_d_h` e `I_d_h` são orçamentos desejados; as compras realizadas serão definidas posteriormente.

Campos atualizados e usos posteriores A função atualiza `model.w_inact.C_d_h` e `model.w_inact.I_d_h`. As compras efetivas serão determinadas posteriormente pelo mercado de bens.

```
[18]: inactive_workers = model.w_inact

expected_income =
    Bit.households_income_inact(model; expected = true)

_, gambling_stakes, _ =
    Bit.gambling_transfers(model; income_inact = expected_income)

budget_income = expected_income - gambling_stakes

expected_consumption_budget =
    model.prop.psi .* budget_income ./ (1 + model.prop.tau_VAT)

expected_investment_budget =
    model.prop.psi_H .* budget_income ./ (1 + model.prop.tau_CF)

Bit.set_households_budget_inact!(model)

@assert all(isapprox.(
    inactive_workers.C_d_h,
    expected_consumption_budget,
))

@assert all(isapprox.(
    inactive_workers.I_d_h,
    expected_investment_budget,
))
```



```

inactive_workers_budget = DataFrame(
    worker_id = inactive_workers.ID,
    expected_income = expected_income,
    gambling_stake = gambling_stakes,
    budget_income = budget_income,
    consumption_budget = inactive_workers.C_d_h,
    investment_budget = inactive_workers.I_d_h,
)

first(inactive_workers_budget, 10)

```

[18]:

	worker_id	expected_income	gambling_stake	budget_income	consumption_budget	investment_b
	Int64	Float64	Float64	Float64	Float64	Float64
1	1	2.81827	0.0	2.81827	2.22375	0.184628
2	2	2.81827	0.0	2.81827	2.22375	0.184628
3	3	2.81827	0.0	2.81827	2.22375	0.184628
4	4	2.81827	0.0	2.81827	2.22375	0.184628
5	5	2.81827	0.0	2.81827	2.22375	0.184628
6	6	2.81827	0.0	2.81827	2.22375	0.184628
7	7	2.81827	0.0	2.81827	2.22375	0.184628
8	8	2.81827	0.0	2.81827	2.22375	0.184628
9	9	2.81827	0.0	2.81827	2.22375	0.184628
10	10	2.81827	0.0	2.81827	2.22375	0.184628

2.20 Orçamento dos proprietários das empresas

Objetivo econômico A função `Bit.set_households_budget_firms!(model)` calcula os orçamentos desejados de consumo e investimento habitacional dos proprietários das empresas.

Equações Cada empresa representa também uma família proprietária. Sua renda esperada é composta pelos dividendos esperados e pelo benefício social geral:

$$Y_i^e = \theta_{\text{DIV}} (1 - \tau_{\text{INC}}) (1 - \tau_{\text{FIRM}}) \max(0, \Pi_i^e) + sb^{\text{other}} \bar{P}_{HH} (1 + \pi^e)$$

O operador $\max(0, \Pi_i^e)$ impede a distribuição de dividendos quando a empresa espera prejuízo.

Receitas provenientes das apostas As apostas realizadas pelos trabalhadores ativos e pelas pessoas inativas são somadas:

$$B = \sum_h B_h^{\text{act}} + \sum_h B_h^{\text{inact}}$$

Quando a empresa pertence ao conjunto de operadoras do mercado de apostas, ela recebe uma parcela igual desse total:

$$R_i = \begin{cases} \frac{B}{N_G}, & \text{se a empresa } i \text{ é uma operadora,} \\ 0, & \text{caso contrário.} \end{cases}$$

A renda disponível para definir os orçamentos é:

$$\widetilde{Y}_i^e = Y_i^e + R_i$$

Orçamentos desejados O orçamento desejado de consumo é:

$$C_i^d = \frac{\psi \widetilde{Y}_i^e}{1 + \tau_{\text{VAT}}}$$

O orçamento desejado de investimento habitacional é:

$$I_i^d = \frac{\psi_H \widetilde{Y}_i^e}{1 + \tau_{\text{CF}}}$$

Definição das variáveis

Símbolo	Código	Significado
i	índice da empresa	Empresa e família proprietária analisada
Y_i^e	<code>expected_income[i]</code>	Renda esperada da família proprietária
$\widetilde{Y}_i^e$	<code>budget_income[i]</code>	Renda esperada incluindo receitas das apostas
Π_i^e	<code>model.firms.Pi_e_i[i]</code>	Lucro esperado da empresa
θ_{DIV}	<code>model.prop.theta_DIV</code>	Parcela do lucro distribuída como dividendos
τ_{FIRM}	<code>model.prop.tau_FIRM</code>	Alíquota do imposto sobre o lucro da empresa
τ_{INC}	<code>model.prop.tau_INC</code>	Alíquota do imposto de renda
sb^{other}	<code>model.gov.sb_other</code>	Benefício social geral
$\bar{P}_{HH}$	<code>model.agg.P_bar_HH</code>	Índice de preços das famílias
π^e	<code>model.agg.pi_e</code>	Taxa de inflação esperada
B	<code>sum(gambling_receipts)</code>	Volume total esperado de apostas
N_G	quantidade de operadoras	Número de empresas que recebem as apostas
R_i	<code>gambling_receipts[i]</code>	Receita da empresa proveniente das apostas
ψ	<code>model.prop.psi</code>	Parcela da renda destinada ao consumo

Símbolo	Código	Significado
ψ_H	<code>model.prop.psi_H</code>	Parcela da renda destinada ao investimento habitacional
τ_{VAT}	<code>model.prop.tau_VAT</code>	Alíquota do imposto sobre consumo
τ_{CF}	<code>model.prop.tau_CF</code>	Alíquota do imposto sobre formação de capital
C_i^d	<code>model.firms.C_d_h[i]</code>	Orçamento desejado de consumo do proprietário
I_i^d	<code>model.firms.I_d_h[i]</code>	Orçamento desejado de investimento habitacional

Valores desejados, esperados e realizados A renda e as receitas de apostas usadas aqui são esperadas. `C_d_h` e `I_d_h` são orçamentos desejados; renda, transferências e compras realizadas não são atualizadas nesta etapa.

Campos atualizados e usos posteriores A função atualiza `model.firms.C_d_h` e `model.firms.I_d_h`. Ela não altera ainda a renda efetiva dos proprietários nem realiza as transferências das apostas.

```
[19]: firms = model.firms

active_income =
    Bit.households_income_act(model; expected = true)

inactive_income =
    Bit.households_income_inact(model; expected = true)

_, _, gambling_receipts = Bit.gambling_transfers(
    model;
    income_act = active_income,
    income_inact = inactive_income,
)

expected_income =
    Bit.households_income_firms(model; expected = true)

budget_income = expected_income + gambling_receipts

expected_consumption_budget =
    model.prop.psi .* budget_income ./ (1 + model.prop.tau_VAT)

expected_investment_budget =
    model.prop.psi_H .* budget_income ./ (1 + model.prop.tau_CF)

Bit.set_households_budget_firms!(model)
```

```

@assert all(isapprox.(
    firms.C_d_h,
    expected_consumption_budget,
))

@assert all(isapprox.(
    firms.I_d_h,
    expected_investment_budget,
))

firm_owners_budget = DataFrame(
    firm_id = firms.ID,
    expected_profit = firms.Pi_e_i,
    expected_income = expected_income,
    gambling_receipt = gambling_receipts,
    budget_income = budget_income,
    consumption_budget = firms.C_d_h,
    investment_budget = firms.I_d_h,
)

first(firm_owners_budget, 10)

```

[19]:

	firm_id	expected_profit	expected_income	gambling_receipt	budget_income	consumption_budg
	Int64	Float64	Float64	Float64	Float64	Float64
1	1	3.75058	2.72783	0.0	2.72783	2.15239
2	2	1.25019	1.30134	0.0	1.30134	1.02682
3	3	1.25019	1.30134	0.0	1.30134	1.02682
4	4	1.25019	1.30134	0.0	1.30134	1.02682
5	5	1.25019	1.30134	0.0	1.30134	1.02682
6	6	1.25019	1.30134	0.0	1.30134	1.02682
7	7	1.25019	1.30134	0.0	1.30134	1.02682
8	8	2.50039	2.01459	0.0	2.01459	1.58961
9	9	1.25019	1.30134	0.0	1.30134	1.02682
10	10	1.25019	1.30134	0.0	1.30134	1.02682

2.21 Orçamento do proprietário do banco

Objetivo econômico A função `Bit.set_households_budget_bank!(model)` calcula os orçamentos desejados de consumo e investimento habitacional do proprietário do banco.

Equações Sua renda esperada é formada pelos dividendos esperados do banco e pelo benefício social geral:

$$Y_k^e = \theta_{\text{DIV}} (1 - \tau_{\text{INC}}) (1 - \tau_{\text{FIRM}}) \max(0, \Pi_k^e) + sb^{\text{other}} \bar{P}_{HH} (1 + \pi^e)$$

O operador:

$$\max(0, \Pi_k^e)$$

impede a distribuição de dividendos quando o banco espera prejuízo.

Orçamentos desejados O orçamento desejado de consumo é:

$$C_k^d = \frac{\psi Y_k^e}{1 + \tau_{\text{VAT}}}$$

O orçamento desejado de investimento habitacional é:

$$I_k^d = \frac{\psi_H Y_k^e}{1 + \tau_{\text{CF}}}$$

Definição das variáveis

Símbolo	Código	Significado
Y_k^e	<code>expected_income</code>	Renda esperada do proprietário do banco
Π_k^e	<code>model.bank.Pi_e_k</code>	Lucro esperado do banco
θ_{DIV}	<code>model.prop.theta_DIV</code>	Parcela do lucro distribuída como dividendos
τ_{FIRM}	<code>model.prop.tau_FIRM</code>	Alíquota do imposto sobre o lucro
τ_{INC}	<code>model.prop.tau_INC</code>	Alíquota do imposto de renda
sb^{other}	<code>model.gov.sb_other</code>	Benefício social geral
$\bar{P}_{HH}$	<code>model.agg.P_bar_HH</code>	Índice de preços das famílias
π^e	<code>model.agg.pi_e</code>	Taxa de inflação esperada
ψ	<code>model.prop.psi</code>	Parcela da renda destinada ao consumo
ψ_H	<code>model.prop.psi_H</code>	Parcela da renda destinada ao investimento habitacional
τ_{VAT}	<code>model.prop.tau_VAT</code>	Alíquota do imposto sobre consumo
τ_{CF}	<code>model.prop.tau_CF</code>	Alíquota do imposto sobre formação de capital
C_k^d	<code>model.bank.C_d_h</code>	Orçamento desejado de consumo
I_k^d	<code>model.bank.I_d_h</code>	Orçamento desejado de investimento habitacional

Diferentemente dos proprietários das empresas, o proprietário do banco não recebe as receitas do mercado de apostas.

Valores desejados, esperados e realizados Pi_e_k compõe a renda esperada do proprietário. C_d_h e I_d_h são orçamentos desejados; consumo e investimento realizados serão definidos no mercado de bens.

Campos atualizados e usos posteriores A função atualiza `model.bank.C_d_h` e `model.bank.I_d_h`. As compras efetivas serão determinadas posteriormente pelo mercado de bens.

```
[20]: bank = model.bank

expected_income =
    Bit.households_income_bank(model; expected = true)

expected_consumption_budget =
    model.prop.psi * expected_income / (1 + model.prop.tau_VAT)

expected_investment_budget =
    model.prop.psi_H * expected_income / (1 + model.prop.tau_CF)

Bit.set_households_budget_bank!(model)

@assert isapprox(
    bank.C_d_h,
    expected_consumption_budget,
)

@assert isapprox(
    bank.I_d_h,
    expected_investment_budget,
)

DataFrame(
    expected_bank_profit = [bank.Pi_e_k],
    expected_owner_income = [expected_income],
    consumption_budget = [bank.C_d_h],
    investment_budget = [bank.I_d_h],
)
```

```
[20]:
```

	expected_bank_profit	expected_owner_income	consumption_budget	investment_budget
	Float64	Float64	Float64	Float64
1	6452.28	3681.67	2905.01	241.191

2.22 Orçamento de consumo do governo

Objetivo econômico A função `Bit.set_gov_expenditure!(model)` calcula o consumo público agregado e distribui seu orçamento igualmente entre os governos locais.

Equações

Consumo público agregado O consumo do governo segue um processo autorregressivo em logaritmos:

$$\log C_{G,t} = \alpha_G \log C_{G,t-1} + \beta_G + \varepsilon_{G,t}$$

Consequentemente:

$$C_{G,t} = \exp(\alpha_G \log C_{G,t-1} + \beta_G + \varepsilon_{G,t})$$

O choque do governo é sorteado de uma distribuição normal:

$$\varepsilon_{G,t} \sim \mathcal{N}(0, \sigma_G^2)$$

Distribuição entre os governos locais O orçamento de cada governo local é:

$$C_j^d = \frac{C_{G,t}}{J} \left(\sum_g c_{G,g} \bar{P}_g \right) (1 + \pi^e)$$

Como o orçamento é dividido igualmente, todos os governos locais recebem o mesmo valor.

Definição das variáveis

Símbolo	Código	Significado
$C_{G,t}$	<code>model.gov.C_G</code>	Consumo agregado do governo no período atual
$C_{G,t-1}$	<code>government_consumption_before</code>	Consumo agregado do governo no período anterior
α_G	<code>model.gov.alpha_G</code>	Coefficiente autorregressivo do consumo público
β_G	<code>model.gov.beta_G</code>	Constante do processo de consumo público
$\varepsilon_{G,t}$	<code>implied_government_shock</code>	Choque aleatório do consumo público
σ_G	<code>model.gov.sigma_G</code>	Escala do choque aleatório
J	<code>local_government_count</code>	Número de governos locais
C_j^d	<code>model.gov.C_d_j[j]</code>	Orçamento de consumo do governo local j
g	índice do produto	Produto analisado
$c_{G,g}$	<code>model.prop.c_G_g[g]</code>	Coefficiente de consumo público do produto g
$\bar{P}_g$	<code>model.agg.P_bar_g[g]</code>	Índice de preços do produto g
π^e	<code>model.agg.pi_e</code>	Taxa de inflação esperada

Valores desejados, esperados e realizados C_G é o nível agregado gerado para o período e $C_{d,j}$ é o orçamento desejado de cada governo local. O consumo público efetivamente realizado ainda será determinado no mercado de bens.

Campos atualizados e usos posteriores A função atualiza `model.gov.C_G` e `model.gov.C_d_j`. O consumo efetivamente realizado será determinado posteriormente pelo mercado de bens.

Aleatoriedade Como a função sorteia um novo choque $\varepsilon_{G,t}$, reexecutar a célula pode produzir um novo orçamento público.

```
[21]: government = model.gov

government_consumption_before = government.C_G
local_government_count = length(government.C_d_j)

Bit.set_gov_expenditure!(model)

government_consumption_after = government.C_G

implied_government_shock =
    log(government_consumption_after) -
    government.alpha_G * log(government_consumption_before) -
    government.beta_G

government_price_factor =
    sum(model.prop.c_G_g .* model.agg.P_bar_g)

expected_local_budget =
    government_consumption_after /
    local_government_count *
    government_price_factor *
    (1 + model.agg.pi_e)

@assert all(isapprox.(
    government.C_d_j,
    expected_local_budget,
))

local_government_budget = DataFrame(
    local_government_id = 1:local_government_count,
    aggregate_consumption = fill(
        government_consumption_after,
        local_government_count,
    ),
    implied_shock = fill(
        implied_government_shock,
```



```

        local_government_count,
    ),
    consumption_budget = government.C_d_j,
)

local_government_budget

```

[21]:

	local_government_id	aggregate_consumption	implied_shock	consumption_budget
	Int64	Float64	Float64	Float64
1	1	14928.2	0.00975426	95.9068
2	2	14928.2	0.00975426	95.9068
3	3	14928.2	0.00975426	95.9068
4	4	14928.2	0.00975426	95.9068
5	5	14928.2	0.00975426	95.9068
6	6	14928.2	0.00975426	95.9068
7	7	14928.2	0.00975426	95.9068
8	8	14928.2	0.00975426	95.9068
9	9	14928.2	0.00975426	95.9068
10	10	14928.2	0.00975426	95.9068
11	11	14928.2	0.00975426	95.9068
12	12	14928.2	0.00975426	95.9068
13	13	14928.2	0.00975426	95.9068
14	14	14928.2	0.00975426	95.9068
15	15	14928.2	0.00975426	95.9068
16	16	14928.2	0.00975426	95.9068
17	17	14928.2	0.00975426	95.9068
18	18	14928.2	0.00975426	95.9068
19	19	14928.2	0.00975426	95.9068
20	20	14928.2	0.00975426	95.9068
21	21	14928.2	0.00975426	95.9068
22	22	14928.2	0.00975426	95.9068
23	23	14928.2	0.00975426	95.9068
24	24	14928.2	0.00975426	95.9068
25	25	14928.2	0.00975426	95.9068
26	26	14928.2	0.00975426	95.9068
27	27	14928.2	0.00975426	95.9068
28	28	14928.2	0.00975426	95.9068
29	29	14928.2	0.00975426	95.9068
30	30	14928.2	0.00975426	95.9068
...	...	...	...	...

2.23 Exportações e importações

Objetivo econômico A função `Bit.set_rotw_import_export!(model)` calcula a demanda externa pelas exportações nacionais e a oferta de produtos importados.

Equações

Demanda agregada por exportações A demanda externa segue um processo autorregressivo em logaritmos:

$$C_{E,t} = \exp(\alpha_E \log C_{E,t-1} + \beta_E + \varepsilon_{E,t})$$

O orçamento é dividido igualmente entre os L parceiros externos:

$$C_l^d = \frac{C_{E,t}}{L} \left(\sum_g c_{E,g} \bar{P}_g \right) (1 + \pi^e)$$

Oferta agregada de importações A oferta de importações também segue um processo autorregressivo:

$$Y_{I,t} = \exp(\alpha_I \log Y_{I,t-1} + \beta_I + \varepsilon_{I,t})$$

A oferta importada de cada produto é:

$$Y_{m,g} = c_{I,g} Y_{I,t}$$

O preço dos produtos importados acompanha o índice de preços doméstico e a inflação esperada:

$$P_{m,g} = \bar{P}_g (1 + \pi^e)$$

Definição das variáveis

Símbolo	Código	Significado
$C_{E,t}$	<code>model.rotw.C_E</code>	Demanda externa agregada por exportações
$C_{E,t-1}$	<code>exports_before</code>	Demanda externa do período anterior
α_E	<code>model.rotw.alpha_E</code>	Coefficiente autorregressivo das exportações
β_E	<code>model.rotw.beta_E</code>	Constante do processo das exportações
$\varepsilon_{E,t}$	<code>model.agg.epsilon_E</code>	Choque sobre a demanda por exportações
L	<code>export_partner_count</code>	Número de parceiros externos
C_l^d	<code>model.rotw.C_d_l[1]</code>	Orçamento de exportação do parceiro l
$c_{E,g}$	<code>model.prop.c_E_g[g]</code>	Coefficiente de demanda externa pelo produto g
$Y_{I,t}$	<code>model.rotw.Y_I</code>	Oferta agregada de importações

Símbolo	Código	Significado
$Y_{I,t-1}$	<code>imports_before</code>	Oferta agregada de importações do período anterior
α_I	<code>model.rotw.alpha_I</code>	Coefficiente autorregressivo das importações
β_I	<code>model.rotw.beta_I</code>	Constante do processo das importações
$\varepsilon_{I,t}$	<code>model.agg.epsilon_I</code>	Choque sobre a oferta de importações
$Y_{m,g}$	<code>model.rotw.Y_m[g]</code>	Oferta importada do produto g
$c_{I,g}$	<code>model.prop.c_I_g[g]</code>	Participação do produto g nas importações
$P_{m,g}$	<code>model.rotw.P_m[g]</code>	Preço do produto importado g
$\bar{P}_g$	<code>model.agg.P_bar_g[g]</code>	Índice de preços doméstico do produto g
π^e	<code>model.agg.pi_e</code>	Taxa de inflação esperada

Valores desejados, esperados e realizados `C_d_l` representa a demanda externa planejada e `Y_m` a oferta importada disponível. Exportações e importações realizadas ainda serão determinadas pelo pareamento no mercado de bens.

Campos atualizados e usos posteriores A função atualiza `model.rotw.C_E`, `model.rotw.C_d_l`, `model.rotw.Y_I`, `model.rotw.Y_m` e `model.rotw.P_m`. As exportações e importações efetivamente realizadas serão determinadas posteriormente pelo mercado de bens.

Aleatoriedade Os choques $\varepsilon_{E,t}$ e $\varepsilon_{I,t}$ já foram sorteados por `Bit.set_epsilon!(model)`. Esta função não realiza novos sorteios; reexecutá-la reaplica os mesmos choques aos níveis que ela própria acabou de atualizar.

```
[22]: rest_of_world = model.rotw

exports_before = rest_of_world.C_E
imports_before = rest_of_world.Y_I
export_partner_count = length(rest_of_world.C_d_l)

expected_exports = exp(
    rest_of_world.alpha_E * log(exports_before) +
    rest_of_world.beta_E +
    model.agg.epsilon_E
)

export_price_factor =
    sum(model.prop.c_E_g .* model.agg.P_bar_g)

expected_export_budget =
```

```

fill(
    expected_exports /
    export_partner_count *
    export_price_factor *
    (1 + model.agg.pi_e),
    export_partner_count,
)

expected_imports = exp(
    rest_of_world.alpha_I * log(imports_before) +
    rest_of_world.beta_I +
    model.agg.epsilon_I
)

expected_import_supply =
    model.prop.c_I_g .* expected_imports

expected_import_prices =
    model.agg.P_bar_g .* (1 + model.agg.pi_e)

Bit.set_rotw_import_export!(model)

@assert isapprox(rest_of_world.C_E, expected_exports)
@assert isapprox(rest_of_world.Y_I, expected_imports)
@assert all(isapprox.(rest_of_world.C_d_l, expected_export_budget))
@assert all(isapprox.(rest_of_world.Y_m, expected_import_supply))
@assert all(isapprox.(rest_of_world.P_m, expected_import_prices))

trade_summary = DataFrame(
    metric = [
        "Aggregate export demand",
        "Aggregate import supply",
        "Export shock",
        "Import shock",
    ],
    value = [
        rest_of_world.C_E,
        rest_of_world.Y_I,
        model.agg.epsilon_E,
        model.agg.epsilon_I,
    ],
)

imports_by_product = DataFrame(
    product_id = eachindex(rest_of_world.Y_m),
    import_supply = rest_of_world.Y_m,
    import_price = rest_of_world.P_m,

```

```
)

display(trade_summary)
imports_by_product
```

[22] :

	metric	value
	String	Float64
1	Aggregate export demand	34686.4
2	Aggregate import supply	33307.1
3	Export shock	0.0127531
4	Import shock	0.00277058

	product_id	import_supply	import_price
	Int64	Float64	Float64
1	1	559.916	1.00223
2	2	136.14	1.00223
3	3	12.2856	1.00223
4	4	1937.95	1.00223
5	5	1630.41	1.00223
6	6	1561.82	1.00223
7	7	336.454	1.00223
8	8	535.123	1.00223
9	9	12.4475	1.00223
10	10	1365.89	1.00223
11	11	2906.92	1.00223
12	12	1000.47	1.00223
13	13	1006.14	1.00223
14	14	435.707	1.00223
15	15	1903.97	1.00223
16	16	1092.04	1.00223
17	17	2035.91	1.00223
18	18	1332.64	1.00223
19	19	3001.95	1.00223
20	20	2815.55	1.00223
21	21	421.093	1.00223
22	22	977.131	1.00223
23	23	178.891	1.00223
24	24	217.497	1.00223
25	25	0.0672703	1.00223
26	26	388.693	1.00223
27	27	304.888	1.00223
28	28	13.6626	1.00223
29	29	81.4499	1.00223
30	30	0.0	1.00223
...	...	...	...

2.24 Pareamento nos mercados de bens

Objetivo econômico A função `Bit.search_and_matching!(model; parallel = false)` realiza as compras e vendas de todos os produtos da economia.

Ela reúne dois mercados:

1. o mercado entre empresas, no qual são comprados insumos intermediários e bens de capital;
2. o mercado varejista, no qual compram as famílias, o governo e os parceiros externos.

O algoritmo é executado separadamente para cada produto g .

Equações

Oferta disponível A oferta de uma empresa doméstica é formada pela produção atual e pelo estoque acumulado:

$$S_i^f = Y_i + S_i$$

A oferta de um produto importado é:

$$S_{m,g}^f = Y_{m,g}$$

Demanda das empresas A demanda da empresa i pelo produto g combina insumos intermediários e investimento produtivo:

$$D_{i,g}^F = a_{g,G_i} DM_i^d + b_g^{CF} I_i^d$$

A primeira parcela representa os insumos necessários à produção. A segunda representa os bens de capital desejados pela empresa.

Demanda do mercado varejista A demanda da família h pelo produto g é:

$$D_{h,g}^R = b_g^{HH} C_h^d + b_g^{CFH} I_h^d$$

A demanda dos parceiros externos é:

$$D_{l,g}^R = c_g^E C_l^d$$

A demanda dos governos locais é:

$$D_{j,g}^R = c_g^G C_j^d$$

Os coeficientes de consumo são normalizados pelo índice de preços de cada produto antes do pareamento.

Escolha dos fornecedores Os compradores são visitados em ordem aleatória. Para cada compra, um fornecedor é sorteado utilizando um peso que combina preço e tamanho da oferta:

$$\omega_f = \frac{\exp(-2P_f)}{\sum_r \exp(-2P_r)} + \frac{S_f}{\sum_r S_r}$$

Assim, fornecedores maiores e com preços menores possuem maior probabilidade de serem escolhidos.

A compra continua até que a demanda do comprador seja atendida ou a oferta disponível termine. Quando necessário, uma segunda rodada considera a capacidade produtiva adicional das empresas.

Vendas realizadas As vendas de cada empresa doméstica são limitadas por sua oferta e pela demanda recebida:

$$Q_i = \min(Y_i + S_i, Q_i^d)$$

As vendas dos produtos importados são:

$$Q_{m,g} = \min(Y_{m,g}, Q_{m,g}^d)$$

Definição das variáveis

Símbolo	Código	Significado
g	índice do produto	Produto negociado
i	índice da empresa	Empresa doméstica
f	índice do fornecedor	Empresa doméstica ou fornecedor estrangeiro
h	índice da família	Família compradora
j	índice do governo local	Governo comprador
l	índice do parceiro externo	Comprador estrangeiro
Y_i	<code>model.firms.Y_i[i]</code>	Produção atual da empresa
S_i	<code>model.firms.S_i[i]</code>	Estoque de produtos da empresa
P_f	preços domésticos e importados	Preço do fornecedor
DM_i^d	<code>model.firms.DM_d_i[i]</code>	Demanda desejada por insumos
I_i^d	<code>model.firms.I_d_i[i]</code>	Investimento produtivo desejado
a_{g,G_i}	<code>model.prop.a_sg[g, G_i]</code>	Necessidade do produto g como insumo
b_g^{CF}	<code>model.prop.b_CF_g[g]</code>	Participação do produto g no investimento empresarial
C_h^d	orçamento de consumo	Consumo desejado da família

Símbolo	Código	Significado
I_h^d	orçamento de investimento	Investimento habitacional desejado
b_g^{HH}	<code>model.prop.b_HH_g[g]</code>	Participação do produto g no consumo familiar
b_g^{CFH}	<code>model.prop.b_CFH_g[g]</code>	Participação do produto g no investimento habitacional
c_g^G	<code>model.prop.c_G_g[g]</code>	Participação do produto g no consumo público
c_g^E	<code>model.prop.c_E_g[g]</code>	Participação do produto g nas exportações
Q_i^d	<code>model.firms.Q_d_i[i]</code>	Demanda total recebida pela empresa
Q_i	<code>model.firms.Q_i[i]</code>	Vendas realizadas pela empresa
$Y_{m,g}$	<code>model.rotw.Y_m[g]</code>	Oferta disponível de importações
$Q_{m,g}^d$	<code>model.rotw.Q_d_m[g]</code>	Demanda por importações
$Q_{m,g}$	<code>model.rotw.Q_m[g]</code>	Vendas realizadas de importações

Valores desejados, esperados e realizados Os orçamentos e demandas recebidos são desejados. O pareamento transforma esses planos em compras, vendas, consumo, investimento, exportações e importações realizados, possivelmente abaixo do desejado quando a oferta é insuficiente.

Campos atualizados e usos posteriores A função também atualiza o consumo e o investimento realizados pelas famílias, as compras de insumos e capital das empresas, o consumo público, as exportações e diversos preços médios.

O investimento habitacional realizado é acrescentado ao estoque de capital das famílias:

$$K_h^{\text{nov}} = K_h^{\text{anterior}} + I_h$$

Aleatoriedade Como o pareamento contém sorteios, reexecutar a célula pode produzir fornecedores e resultados diferentes. O argumento `parallel = false` mantém o processamento sequencial dos produtos.

```
[23]: firms = model.firms
      rest_of_world = model.rotw
      active_workers = model.w_act
      inactive_workers = model.w_inact
      bank = model.bank

      domestic_supply_before =
        copy(firms.Y_i .+ firms.S_i)
```



```

import_supply_before =
    copy(rest_of_world.Y_m)

active_capital_before =
    copy(active_workers.K_h)

inactive_capital_before =
    copy(inactive_workers.K_h)

firm_owner_capital_before =
    copy(firms.K_h)

bank_owner_capital_before =
    bank.K_h

Bit.search_and_matching!(model; parallel = false)

expected_domestic_sales =
    min.(domestic_supply_before, firms.Q_d_i)

expected_import_sales =
    min.(import_supply_before, rest_of_world.Q_d_m)

@assert all(isapprox.(
    firms.Q_i,
    expected_domestic_sales,
))

@assert all(isapprox.(
    rest_of_world.Q_m,
    expected_import_sales,
))

@assert all(isapprox.(
    active_workers.K_h,
    active_capital_before .+ active_workers.I_h,
))

@assert all(isapprox.(
    inactive_workers.K_h,
    inactive_capital_before .+ inactive_workers.I_h,
))

@assert all(isapprox.(
    firms.K_h,
    firm_owner_capital_before .+ firms.I_h,
))

```

```

@assert isapprox(
    bank.K_h,
    bank_owner_capital_before + bank.I_h,
)

market_summary = DataFrame(
    metric = [
        "Domestic supply",
        "Domestic firm sales",
        "Import supply",
        "Import sales",
        "Household consumption",
        "Household investment",
        "Government consumption",
        "Exports",
    ],
    value = [
        sum(domestic_supply_before),
        sum(firms.Q_i),
        sum(import_supply_before),
        sum(rest_of_world.Q_m),
        sum(active_workers.C_h) +
        sum(inactive_workers.C_h) +
        sum(firms.C_h) +
        bank.C_h,
        sum(active_workers.I_h) +
        sum(inactive_workers.I_h) +
        sum(firms.I_h) +
        bank.I_h,
        model.gov.C_j,
        rest_of_world.C_l,
    ],
)

market_summary

```

[23]:

	metric	value
	String	Float64
1	Domestic supply	1.33838e5
2	Domestic firm sales	1.33827e5
3	Import supply	33307.1
4	Import sales	33286.0
5	Household consumption	34903.0
6	Household investment	2895.83
7	Government consumption	14721.4
8	Exports	34417.7

2.25 Inflação e índice geral de preços

Objetivo econômico A função `Bit.set_inflation_priceindex!(model)` calcula o novo índice geral de preços e a inflação realizada no período.

Equações

Índice geral de preços O índice é uma média dos preços das empresas ponderada por suas produções:

$$\bar{P}_t = \frac{\sum_i P_i Y_i}{\sum_i Y_i}$$

Empresas com maior produção possuem maior peso no índice.

Inflação realizada A inflação é calculada como a variação logarítmica entre o índice atual e o índice anterior:

$$\pi_t = \log \left(\frac{\bar{P}_t}{\bar{P}_{t-1}} \right)$$

Definição das variáveis

Símbolo	Código	Significado
i	índice da empresa	Empresa analisada
P_i	<code>model.firms.P_i[i]</code>	Preço definido pela empresa
Y_i	<code>model.firms.Y_i[i]</code>	Produção realizada pela empresa
$P_i Y_i$	<code>price[i] * production[i]</code>	Valor nominal da produção da empresa
$\bar{P}_{t-1}$	<code>price_index_before</code>	Índice geral de preços anterior
$\bar{P}_t$	<code>model.agg.P_bar</code>	Novo índice geral de preços
π_t	<code>model.agg.pi_[inflation_positivo]</code>	Inflação realizada no período
π^e	<code>model.agg.pi_e</code>	Inflação esperada calculada no início do período

Quando:

$$\bar{P}_t > \bar{P}_{t-1},$$

a inflação é positiva. Quando o novo índice é menor que o anterior, a inflação é negativa.

Valores desejados, esperados e realizados `model.agg.pi_e` continua sendo a inflação esperada do início do período. `model.agg.P_bar` e a nova entrada de `model.agg.pi_` são valores realizados calculados com preços e produções correntes; não há valor desejado nesta função.

Campos atualizados e usos posteriores A função acrescenta a inflação realizada ao histórico `model.agg.pi_` e atualiza `model.agg.P_bar`.

Esse índice utiliza a produção doméstica, Y_i , como ponderação. As quantidades importadas e as vendas efetivamente realizadas não entram diretamente nesse cálculo.

```
[24]: aggregate = model.agg
      firms = model.firms

      price_index_before = aggregate.P_bar
      inflation_history_length_before = length(aggregate.pi_)

      prices = copy(firms.P_i)
      production = copy(firms.Y_i)

      nominal_production =
        sum(prices .* production)

      total_production =
        sum(production)

      expected_price_index =
        nominal_production / total_production

      expected_inflation =
        log(expected_price_index / price_index_before)

      inflation_position =
        model.prop.T_prime + aggregate.t

      Bit.set_inflation_priceindex!(model)

      @assert isapprox(
        aggregate.P_bar,
        expected_price_index,
      )

      @assert isapprox(
        aggregate.pi_[inflation_position],
        expected_inflation,
      )

      @assert length(aggregate.pi_) ==
        inflation_history_length_before + 1

      price_contributions = DataFrame(
        firm_id = firms.ID,
        price = prices,
```

```

production = production,
nominal_production = prices .* production,
production_weight = production ./ total_production,
)

inflation_summary = DataFrame(
    metric = [
        "Previous price index",
        "Current price index",
        "Realized inflation",
        "Expected inflation",
    ],
    value = [
        price_index_before,
        aggregate.P_bar,
        aggregate.pi_[inflation_position],
        aggregate.pi_e,
    ],
)

display(inflation_summary)
first(price_contributions, 10)

```

	metric	value
	String	Float64
1	Previous price index	1.0
2	Current price index	1.00223
3	Realized inflation	0.00222896
4	Expected inflation	0.00223145

[24]:

	firm_id	price	production	nominal_production	production_weight
	Int64	Float64	Float64	Float64	Float64
1	1	1.00223	32.2737	32.3457	0.00024114
2	2	1.00223	10.7579	10.7819	8.038e-5
3	3	1.00223	10.7579	10.7819	8.038e-5
4	4	1.00223	10.7579	10.7819	8.038e-5
5	5	1.00223	10.7579	10.7819	8.038e-5
6	6	1.00223	10.7579	10.7819	8.038e-5
7	7	1.00223	10.7579	10.7819	8.038e-5
8	8	1.00223	21.5158	21.5638	0.00016076
9	9	1.00223	10.7579	10.7819	8.038e-5
10	10	1.00223	10.7579	10.7819	8.038e-5

2.26 Índices setoriais de preços

Objetivo econômico A função `Bit.set_sector_specific_priceindex!(model)` calcula um índice de preços para cada produto ou setor g . O índice combina vendas domésticas e importadas, ponderando cada preço pela quantidade efetivamente vendida.

A implementação usa `model.firms.Q_i`, e não `model.firms.Y_i` como sugere a docstring simplificada. Portanto, o peso das empresas corresponde às vendas realizadas, não à produção.

Equações

Cálculo do índice Para cada setor, o valor das vendas domésticas é:

$$V_g^D = \sum_{i:G_i=g} P_i Q_i$$

O valor das vendas importadas é:

$$V_g^M = P_g^m Q_g^m$$

A quantidade total vendida no setor é:

$$Q_g = \sum_{i:G_i=g} Q_i + Q_g^m$$

O índice setorial resulta da média ponderada:

$$\bar{P}_g = \frac{V_g^D + V_g^M}{Q_g} = \frac{\sum_{i:G_i=g} P_i Q_i + P_g^m Q_g^m}{\sum_{i:G_i=g} Q_i + Q_g^m}$$

Assim, preços associados a volumes maiores de vendas recebem maior peso.

Definição das variáveis

Símbolo	Código	Significado
g	<code>1:model.prop.G</code>	Produto ou setor
G_i	<code>model.firms.G_i[i]</code>	Setor principal da empresa i
P_i	<code>model.firms.P_i[i]</code>	Preço definido pela empresa
Q_i^s	<code>model.firms.Q_s_i[i]</code>	Quantidade-alvo desejada pela empresa
Y_i	<code>model.firms.Y_i[i]</code>	Produção realizada
Q_i^d	<code>model.firms.Q_d_i[i]</code>	Demanda dirigida à empresa
Q_i	<code>model.firms.Q_i[i]</code>	Venda doméstica realizada
P_g^m	<code>model.rotw.P_m[g]</code>	Preço das importações do setor
Y_g^m	<code>model.rotw.Y_m[g]</code>	Oferta de importações
$Q_g^{d,m}$	<code>model.rotw.Q_d_m[g]</code>	Demanda por importações
Q_g^m	<code>model.rotw.Q_m[g]</code>	Venda importada realizada
$\bar{P}_g$	<code>model.agg.P_bar_g[g]</code>	Índice de preços do setor

Valores desejados, esperados e realizados A quantidade Q_{s_i} é a meta desejada pela empresa. A produção realizada Y_i pode ser menor por restrições de trabalho, capital ou insumos. Depois do encontro entre oferta e demanda, as vendas realizadas são:

$$Q_i = \min(Y_i + S_i, Q_i^d)$$

$$Q_g^m = \min(Y_g^m, Q_g^{d,m})$$

As expectativas `model.agg.gamma_e` e `model.agg.pi_e` influenciaram anteriormente as decisões de produção e os preços domésticos e importados. Elas não entram diretamente nesta função. O índice é realizado porque usa os preços vigentes e as quantidades efetivamente vendidas.

Na célula de código, `expected_sector_price_index` significa o resultado esperado do teste, calculado antes da chamada da função. Não representa uma expectativa econômica dos agentes.

Campos atualizados e usos posteriores A função atualiza somente `model.agg.P_bar_g`, substituindo cada elemento do vetor pelo índice do respectivo setor.

Ainda neste período, esses valores serão usados por:

- `Bit.set_capital_formation_priceindex!(model)`, no índice de preços do capital;
- `Bit.set_households_priceindex!(model)`, no índice de preços das famílias;
- `Bit.set_firms_equity!(model)`, na avaliação dos estoques de insumos das empresas.

No período seguinte, também influenciarão os preços definidos pelas empresas, o orçamento do governo, o comércio exterior e a distribuição da demanda no mercado de bens.

Se não houver nenhuma venda doméstica nem importada em um setor, o denominador será zero. A implementação não possui um valor alternativo e, nesse caso, armazena NaN.

A função não realiza sorteios. Entretanto, suas quantidades foram produzidas anteriormente pelo processo aleatório de busca e matching. Reexecutar somente esta célula mantém o resultado; reexecutar as células anteriores de matching pode alterá-lo.

```
[25]: aggregate = model.agg
      firms = model.firms
      rest_of_world = model.rotw

      sector_price_index_before = copy(aggregate.P_bar_g)
      firm_prices = copy(firms.P_i)
      firm_sales = copy(firms.Q_i)
      firm_sectors = copy(firms.G_i)
      import_prices = copy(rest_of_world.P_m)
      import_sales = copy(rest_of_world.Q_m)

      sector_count = model.prop.G
      domestic_quantity = zeros(eltype(firm_sales), sector_count)
      domestic_value = zeros(eltype(firm_sales), sector_count)
```

```

for sector in 1:sector_count
    in_sector = firm_sectors .== sector
    domestic_quantity[sector] = sum(firm_sales[in_sector])
    domestic_value[sector] =
        sum(firm_prices[in_sector] .* firm_sales[in_sector])
end

import_value = import_prices .* import_sales
total_quantity = domestic_quantity .+ import_sales

expected_sector_price_index =
    (domestic_value .+ import_value) ./ total_quantity

Bit.set_sector_specific_priceindex!(model)

@assert all(isapprox.(
    aggregate.P_bar_g,
    expected_sector_price_index;
    nans = true,
))

sector_summary = DataFrame(
    sector = 1:sector_count,
    previous_price_index = sector_price_index_before,
    domestic_quantity = domestic_quantity,
    import_quantity = import_sales,
    total_quantity = total_quantity,
    expected_price_index = expected_sector_price_index,
    actual_price_index = aggregate.P_bar_g,
    zero_total_quantity = iszero.(total_quantity),
)

first(sector_summary, min(10, nrow(sector_summary)))

```

[25]:

	sector	previous_price_index	domestic_quantity	import_quantity	total_quantity	
	Int64	Float64	Float64	Float64	Float64	
1	1	1.0	1320.52	559.916	1880.44	...
2	2	1.0	535.292	136.14	671.432	...
3	3	1.0	10.2333	12.2856	22.5189	...
4	4	1.0	433.185	1920.12	2353.31	...
5	5	1.0	4045.44	1630.41	5675.85	...
6	6	1.0	702.617	1559.98	2262.6	...
7	7	1.0	1676.48	336.454	2012.94	...
8	8	1.0	1401.53	535.123	1936.65	...
9	9	1.0	637.077	12.4475	649.524	...
10	10	1.0	949.109	1365.89	2315.0	...

2.27 Índice de preços da formação de capital

Objetivo econômico A função `Bit.set_capital_formation_priceindex!(model)` calcula o nível de preços da cesta de bens utilizada pelas empresas na formação de capital.

Equações

Pesos da cesta Os pesos `model.prop.b_CF_g` são calculados durante a calibração a partir da formação bruta de capital fixo não residencial:

$$b_g^{CF} = \frac{F_g^{CF}}{\sum_h F_h^{CF}}$$

Consequentemente, os pesos somam aproximadamente um:

$$\sum_g b_g^{CF} \approx 1$$

Esses pesos representam a composição estrutural do investimento das empresas. Eles são diferentes de `b_CFH_g`, que descreve a formação de capital das famílias em habitação.

Cálculo do índice O índice é uma média ponderada dos índices setoriais calculados na etapa anterior:

$$\bar{P}_{CF} = \sum_{g=1}^G b_g^{CF} \bar{P}_g$$

A contribuição de cada setor é:

$$C_g^{CF} = b_g^{CF} \bar{P}_g$$

Setores com maior participação na cesta de investimento exercem maior influência sobre o índice.

Definição das variáveis

Símbolo	Código	Significado
g	<code>1:model.prop.G</code>	Produto ou setor
F_g^{CF}	dado utilizado na calibração	Formação de capital não residencial do setor
b_g^{CF}	<code>model.prop.b_CF_g[g]</code>	Peso fixo do setor na cesta de capital das empresas
$\bar{P}_g$	<code>model.agg.P_bar_g[g]</code>	Índice realizado de preços do setor
C_g^{CF}	<code>capital_formation_weights[g]</code> <code>* sector_price_indices[g]</code>	Contribuição ponderada do setor

Símbolo	Código	Significado
$\bar{P}_{CF}$	<code>model.agg.P_bar_CF</code>	Índice agregado de preços da formação de capital
I_i^d	<code>model.firms.I_d_i[i]</code>	Investimento desejado pela empresa
I_i	<code>model.firms.I_i[i]</code>	Investimento efetivamente adquirido
$P_{CF,i}$	<code>model.firms.P_CF_i[i]</code>	Preço médio efetivamente pago pela empresa em seus investimentos

Valores desejados, esperados e realizados O investimento desejado $I_{d,i}$ foi decidido anteriormente e distribuído entre os setores usando `b_CF_g`. Depois da busca e matching, I_i e $P_{CF,i}$ registram as compras efetivamente realizadas por cada empresa.

Esta função não utiliza $I_{d,i}$, I_i ou $P_{CF,i}$ como pesos. Ela aplica a cesta fixa `b_CF_g` aos índices setoriais realizados P_{bar_g} .

As expectativas $\pi_{e,i}$ e $K_{e,i}$ também não entram diretamente no cálculo. Na célula de código, `expected_capital_formation_price_index` representa apenas o resultado esperado do teste, e não uma expectativa econômica dos agentes.

Campos atualizados e usos posteriores A função atualiza somente o escalar `model.agg.P_bar_CF`.

Ainda neste período, esse índice será usado por `Bit.set_firms_equity!(model)` para avaliar o estoque de capital das empresas:

$$\text{valor do capital}_i = \bar{P}_{CF} K_i$$

No início do período seguinte, ele também será usado:

- por `Bit.finance_insolvent_firms!(model)`, na avaliação do capital dado como garantia;
- por `Bit.set_firms_expectations_and_decisions!(model)`, no custo do capital e no valor esperado do estoque de capital.

Não há denominador nem sorteio nesta função. Entretanto, os índices P_{bar_g} dependem das vendas obtidas anteriormente pelo matching aleatório. Reexecutar somente esta célula não altera o resultado; reexecutar as etapas anteriores do mercado pode alterá-lo.

Se algum P_{bar_g} for NaN, inclusive em um setor com peso zero, a multiplicação e a soma da implementação também produzirão NaN.

```
[26]: aggregate = model.agg

capital_formation_price_index_before =
    aggregate.P_bar_CF

sector_price_indices =
```

```

        copy(aggregate.P_bar_g)

capital_formation_weights =
    copy(model.prop.b_CF_g)

expected_capital_formation_price_index =
    sum(capital_formation_weights .* sector_price_indices)

Bit.set_capital_formation_priceindex!(model)

@assert isapprox(
    aggregate.P_bar_CF,
    expected_capital_formation_price_index;
    nans = true,
)

capital_formation_summary = DataFrame(
    metric = [
        "Previous capital formation price index",
        "Computed capital formation price index",
        "Stored capital formation price index",
        "Sum of sector weights",
    ],
    value = [
        capital_formation_price_index_before,
        expected_capital_formation_price_index,
        aggregate.P_bar_CF,
        sum(capital_formation_weights),
    ],
)

sector_contributions = DataFrame(
    sector = eachindex(sector_price_indices),
    sector_price_index = sector_price_indices,
    capital_formation_weight = capital_formation_weights,
    weighted_contribution =
        capital_formation_weights .* sector_price_indices,
)

display(capital_formation_summary)

first(
    sort(
        sector_contributions,
        :capital_formation_weight;
        rev = true,
    ),

```

```
min(10, nrow(sector_contributions)),
)
```

	metric	value
	String	Float64
1	Previous capital formation price index	1.0
2	Computed capital formation price index	1.00223
3	Stored capital formation price index	1.00223
4	Sum of sector weights	1.0

[26] :

	sector	sector_price_index	capital_formation_weight	weighted_contribution
	Int64	Float64	Float64	Float64
1	27	1.00223	0.299326	0.299994
2	47	1.00223	0.138001	0.138309
3	19	1.00223	0.115503	0.115761
4	40	1.00223	0.0865366	0.0867297
5	20	1.00223	0.0709726	0.0711309
6	17	1.00223	0.045953	0.0460556
7	23	1.00223	0.0419963	0.04209
8	29	1.00223	0.0391112	0.0391985
9	22	1.00223	0.0310164	0.0310856
10	46	1.00223	0.0212671	0.0213146

2.28 Índice de preços do consumo das famílias

Objetivo econômico A função `Bit.set_households_priceindex!(model)` calcula o índice de preços de uma cesta representativa do consumo das famílias.

Equações

Pesos da cesta Os pesos são definidos durante a calibração a partir do consumo observado de cada setor:

$$b_g^{HH} = \frac{C_g^{HH}}{\sum_h C_h^{HH}}$$

Logo:

$$\sum_{g=1}^G b_g^{HH} \approx 1$$

Esses pesos permanecem fixos durante a simulação e representam a composição estrutural do consumo das famílias.

Cálculo do índice O índice combina os índices setoriais atualizados na etapa anterior:

$$\bar{P}_{HH} = \sum_{g=1}^G b_g^{HH} \bar{P}_g$$

A contribuição de cada setor é:

$$C_g^P = b_g^{HH} \bar{P}_g$$

Assim, uma variação de preços em um setor com grande participação no consumo exerce maior impacto sobre o índice.

Definição das variáveis

Símbolo	Código	Significado
g	<code>1:model.prop.G</code>	Produto ou setor
C_g^{HH}	dado usado na calibração	Consumo das famílias no setor
b_g^{HH}	<code>model.prop.b_HH_g[g]</code>	Peso fixo do setor na cesta de consumo
$\bar{P}_g$	<code>model.agg.P_bar_g[g]</code>	Índice realizado de preços do setor
C_g^P	<code>household_consumption_weights[g]</code> <code>* sector_price_indices[g]</code>	Contribuição do setor ao índice
$\bar{P}_{HH}$	<code>model.agg.P_bar_HH</code>	Índice de preços do consumo das famílias
C_h^d	<code>C_d_h</code> dos diferentes tipos de família	Orçamento de consumo desejado
C_h	<code>C_h</code> dos diferentes tipos de família	Consumo efetivamente realizado
$\bar{P}_h$	<code>model.agg.P_bar_h</code>	Preço agregado efetivamente pago no matching
π^e	<code>model.agg.pi_e</code>	Inflação esperada

Valores desejados, esperados e realizados O consumo desejado `C_d_h` é definido anteriormente a partir da renda esperada. A inflação esperada `pi_e` participa dessa formação de renda e orçamento, mas não entra diretamente nesta função.

O consumo `C_h` e o índice `P_bar_h` são resultados efetivos do matching entre famílias e vendedores. Eles podem refletir restrições de oferta e os fornecedores efetivamente encontrados.

Por outro lado, `P_bar_HH` aplica uma cesta fixa `b_HH_g` aos índices setoriais realizados `P_bar_g`. Portanto, ele não é ponderado pelo consumo efetivamente comprado no trimestre.

Na célula de código, `expected_households_price_index` representa o resultado esperado do teste, não uma expectativa econômica dos agentes.

Campos atualizados e usos posteriores A função atualiza somente `model.agg.P_bar_HH`.

Ainda neste período, o novo índice será usado:

- em `Bit.set_firms_profits!(model)`, para calcular os custos nominais do trabalho;
- nas funções `Bit.set_households_income_...!`, para converter salários e benefícios em renda nominal;
- em `Bit.set_gov_revenues!(model)` e `Bit.set_gov_loans!(model)`;
- em `Bit.set_firms_deposits!(model)`, novamente nos custos do trabalho.

No período seguinte, será usado na formação dos preços das empresas e no cálculo da renda esperada e dos orçamentos das famílias.

A função não possui denominador nem realiza sorteios. Contudo, `P_bar_g` depende das vendas obtidas anteriormente pelo matching aleatório. Reexecutar somente esta célula não altera o resultado; reexecutar as etapas anteriores do mercado pode alterá-lo.

Se algum índice setorial for NaN, a soma também produzirá NaN, mesmo que o peso correspondente seja zero, pois em Julia `0 * NaN` resulta em NaN.

```
[27]: aggregate = model.agg

households_price_index_before =
    aggregate.P_bar_HH

sector_price_indices =
    copy(aggregate.P_bar_g)

household_consumption_weights =
    copy(model.prop.b_HH_g)

expected_households_price_index =
    sum(household_consumption_weights .* sector_price_indices)

Bit.set_households_priceindex!(model)

@assert isapprox(
    aggregate.P_bar_HH,
    expected_households_price_index;
    nans = true,
)

households_price_summary = DataFrame(
    metric = [
        "Previous households price index",
        "Computed households price index",
        "Stored households price index",
        "Sum of consumption weights",
    ],
    value = [
```

```

        households_price_index_before,
        expected_households_price_index,
        aggregate.P_bar_HH,
        sum(household_consumption_weights),
    ],
)

sector_contributions = DataFrame(
    sector = eachindex(sector_price_indices),
    sector_price_index = sector_price_indices,
    consumption_weight = household_consumption_weights,
    weighted_contribution =
        household_consumption_weights .* sector_price_indices,
)

display(households_price_summary)

first(
    sort(
        sector_contributions,
        :consumption_weight;
        rev = true,
    ),
    min(10, nrow(sector_contributions)),
)

```

	metric	value
	String	Float64
1	Previous households price index	1.0
2	Computed households price index	1.00223
3	Stored households price index	1.00223
4	Sum of consumption weights	1.0

[27]:

	sector	sector_price_index	consumption_weight	weighted_contribution
	Int64	Float64	Float64	Float64
1	44	1.00223	0.174339	0.174728
2	30	1.00223	0.1216	0.121871
3	36	1.00223	0.117678	0.117941
4	5	1.00223	0.063051	0.0631917
5	29	1.00223	0.0405461	0.0406365
6	56	1.00223	0.0351451	0.0352235
7	6	1.00223	0.030682	0.0307505
8	24	1.00223	0.0264915	0.0265506
9	31	1.00223	0.0260628	0.0261209
10	28	1.00223	0.0256879	0.0257453

2.29 Atualização dos estoques das empresas

Objetivo econômico A função `Bit.set_firms_stocks!(model)` fecha os estoques das empresas após a produção, as compras de insumos, os investimentos e as vendas realizadas no período.

Equações

Estoque de capital O capital produtivo sofre depreciação proporcional à produção e recebe o investimento realizado:

$$K_{i,t} = K_{i,t-1} - \frac{\delta_i}{\kappa_i} Y_i + I_i$$

Estoque de insumos intermediários A produção consome insumos de acordo com sua produtividade, enquanto as compras realizadas recompõem o estoque:

$$M_{i,t} = M_{i,t-1} - \frac{Y_i}{\beta_i} + DM_i$$

Estoque de produtos acabados A variação do estoque é a diferença entre produção e vendas:

$$DS_i = Y_i - Q_i$$

O estoque final é:

$$S_{i,t} = S_{i,t-1} + DS_i = S_{i,t-1} + Y_i - Q_i$$

Se $Q_i > Y_i$, então $DS_i < 0$: a empresa vendeu parte do estoque acumulado anteriormente. Uma variação negativa não representa, por si só, um erro.

Definição das variáveis

Símbolo	Código	Significado
$K_{i,t-1}$	<code>capital_before[i]</code>	Capital produtivo antes da atualização
δ_i	<code>model.firms.delta_i[i]</code>	Taxa de depreciação do capital
κ_i	<code>model.firms.kappa_i[i]</code>	Produtividade do capital
I_i	<code>model.firms.I_i[i]</code>	Investimento realizado
$M_{i,t-1}$	<code>materials_before[i]</code>	Estoque anterior de insumos intermediários
β_i	<code>model.firms.beta_i[i]</code>	Produtividade dos insumos intermediários
DM_i	<code>model.firms.DM_i[i]</code>	Compra realizada de insumos
Y_i	<code>model.firms.Y_i[i]</code>	Produção realizada
Q_i	<code>model.firms.Q_i[i]</code>	Venda realizada

Símbolo	Código	Significado
DS_i	<code>model.firms.DS_i[i]</code>	Variação do estoque de produtos acabados
$S_{i,t-1}$	<code>inventories_before[i]</code>	Estoque anterior de produtos acabados
$S_{i,t}$	<code>model.firms.S_i[i]</code>	Novo estoque de produtos acabados

Valores desejados, esperados e realizados Q_s_i , I_d_i e DM_d_i são, respectivamente, a produção-alvo, o investimento desejado e a compra desejada de insumos. K_e_i representa o valor esperado do capital. Esses valores não são usados diretamente nesta função.

A atualização utiliza os resultados realizados:

- Y_i : produção limitada por trabalho, capital e insumos;
- Q_i : vendas obtidas no mercado;
- I_i : investimento efetivamente adquirido;
- DM_i : insumos efetivamente adquiridos.

Portanto, os novos estoques registram o resultado contábil efetivo do período, e não os planos formulados pelas empresas.

Na célula de código, os nomes iniciados por `expected_` representam resultados calculados para testar a implementação, não expectativas econômicas.

Campos atualizados e usos posteriores A função atualiza:

- `model.firms.K_i`;
- `model.firms.M_i`;
- `model.firms.DS_i`;
- `model.firms.S_i`.

Ainda neste período, `DS_i` será usado em `Bit.set_firms_profits!(model)`. Posteriormente, K_i , M_i e S_i serão avaliados em `Bit.set_firms_equity!(model)`.

No período seguinte:

- K_i e M_i limitarão a produção;
- S_i integrará a oferta disponível no mercado;
- K_i será usado na resolução de insolvências e nas decisões de investimento.

A função não realiza sorteios, mas Q_i , I_i e DM_i resultam do matching aleatório executado anteriormente.

Os denominadores são `kappa_i` e `beta_i`. A função não possui proteção contra valores zero: 0/0 produz `NaN` e uma quantidade positiva dividida por zero produz `Inf`.

Esta atualização não é idempotente. Reexecutar a célula aplica novamente os mesmos fluxos aos estoques e altera o modelo outra vez.

```

[28]: firms = model.firms

capital_before = copy(firms.K_i)
materials_before = copy(firms.M_i)
inventories_before = copy(firms.S_i)
inventory_change_before = copy(firms.DS_i)

production = copy(firms.Y_i)
sales = copy(firms.Q_i)
investment = copy(firms.I_i)
materials_purchased = copy(firms.DM_i)

depreciation_rates = copy(firms.delta_i)
capital_productivity = copy(firms.kappa_i)
materials_productivity = copy(firms.beta_i)

capital_depreciation =
    depreciation_rates ./ capital_productivity .* production

materials_used =
    production ./ materials_productivity

expected_capital =
    capital_before .- capital_depreciation .+ investment

expected_materials =
    materials_before .- materials_used .+ materials_purchased

expected_inventory_change =
    production .- sales

expected_inventories =
    inventories_before .+ expected_inventory_change

Bit.set_firms_stocks!(model)

@assert all(isapprox.(
    firms.K_i,
    expected_capital;
    nans = true,
))

@assert all(isapprox.(
    firms.M_i,
    expected_materials;
    nans = true,
))

```

```

@assert all(isapprox.(
    firms.DS_i,
    expected_inventory_change;
    nans = true,
))

@assert all(isapprox.(
    firms.S_i,
    expected_inventories;
    nans = true,
))

stock_summary = DataFrame(
    stock = [
        "Productive capital",
        "Intermediate materials",
        "Finished-goods inventories",
    ],
    opening = [
        sum(capital_before),
        sum(materials_before),
        sum(inventories_before),
    ],
    inflow = [
        sum(investment),
        sum(materials_purchased),
        sum(production),
    ],
    outflow = [
        sum(capital_depreciation),
        sum(materials_used),
        sum(sales),
    ],
    expected_closing = [
        sum(expected_capital),
        sum(expected_materials),
        sum(expected_inventories),
    ],
    actual_closing = [
        sum(firms.K_i),
        sum(firms.M_i),
        sum(firms.S_i),
    ],
)

stock_summary

```

[28] :

	stock	opening	inflow	outflow	expected_closing	actual_closing
	String	Float64	Float64	Float64	Float64	Float64
1	Productive capital	750826.0	12695.3	12695.3	750826.0	750826.0
2	Intermediate materials	80090.1	67673.2	67673.2	80090.1	80090.1
3	Finished-goods inventories	0.0	1.33838e5	1.33827e5	11.0423	11.0423

2.30 Lucro realizado das empresas

Objetivo econômico A função `Bit.set_firms_profits!(model)` calcula o lucro contábil realizado de cada empresa depois da produção, das vendas e da atualização dos estoques.

Equações

Receita da produção A receita inclui as vendas e a variação do estoque de produtos acabados:

$$R_i^Y = P_i Q_i + P_i DS_i = P_i (Q_i + DS_i)$$

Como a função anterior definiu $DS_i = Y_i - Q_i$:

$$R_i^Y = P_i Y_i$$

Assim, produtos não vendidos aumentam o valor dos estoques, enquanto vendas de estoques antigos reduzem essa parcela.

A empresa também recebe juros sobre depósitos positivos:

$$R_i^D = \bar{r}[D_i]^+$$

onde:

$$[x]^+ = \max(0, x)$$

Custos da empresa O custo do trabalho é:

$$C_i^W = (1 + \tau_{SIF})w_i N_i \bar{P}_{HH}$$

O custo dos insumos intermediários consumidos na produção é:

$$C_i^M = \frac{Y_i}{\beta_i} \bar{P}_i$$

O custo da depreciação do capital é:

$$C_i^K = \frac{\delta_i}{\kappa_i} Y_i P_{CF,i}$$

Os impostos líquidos sobre produtos e produção são:

$$T_i^Y = \tau_{Y,i} P_i Y_i$$

$$T_i^K = \tau_{K,i} P_i Y_i$$

Os juros são cobrados sobre os empréstimos e sobre eventuais depósitos negativos:

$$C_i^L = r (L_i + [-D_i]^+)$$

Lucro realizado O lucro final é:

$$\Pi_i = R_i^Y + R_i^D - C_i^W - C_i^M - C_i^K - T_i^Y - T_i^K - C_i^L$$

Taxas líquidas de imposto negativas funcionam como subsídios e aumentam o lucro.

Definição das variáveis

Símbolo	Código	Significado
Π_i	<code>model.firms.Pi_i[i]</code>	Lucro realizado da empresa
P_i	<code>model.firms.P_i[i]</code>	Preço do produto da empresa
Q_i	<code>model.firms.Q_i[i]</code>	Venda realizada
DS_i	<code>model.firms.DS_i[i]</code>	Variação do estoque de produtos acabados
D_i	<code>model.firms.D_i[i]</code>	Depósitos antes do fechamento financeiro
L_i	<code>model.firms.L_i[i]</code>	Empréstimos antes da amortização
$\bar{r}$	<code>model.cb.r_bar</code>	Taxa paga sobre depósitos positivos
r	<code>model.bank.r</code>	Taxa cobrada sobre crédito
w_i	<code>model.firms.w_i[i]</code>	Salário real pago pela empresa
N_i	<code>model.firms.N_i[i]</code>	Emprego realizado
$\bar{P}_{HH}$	<code>model.agg.P_bar_HH</code>	Índice usado para converter salários em valores nominais
β_i	<code>model.firms.beta_i[i]</code>	Produtividade dos insumos intermediários
$\bar{P}_i$	<code>model.firms.P_bar_i[i]</code>	Preço médio dos insumos da empresa
δ_i	<code>model.firms.delta_i[i]</code>	Taxa de depreciação
κ_i	<code>model.firms.kappa_i[i]</code>	Produtividade do capital
$P_{CF,i}$	<code>model.firms.P_CF_i[i]</code>	Preço dos bens de capital da empresa

Símbolo	Código	Significado
$\tau_{Y,i}$	<code>model.firms.tau_Y_i[i]</code>	Imposto líquido sobre produtos
$\tau_{K,i}$	<code>model.firms.tau_K_i[i]</code>	Imposto líquido sobre a produção

Valores desejados, esperados e realizados Pi_e_i é o lucro esperado calculado no início do período a partir do lucro anterior, da inflação esperada e do crescimento esperado:

$$\Pi_i^e = \Pi_{i,t-1}(1 + \pi^e)(1 + \gamma^e)$$

Ele foi usado nas decisões de crédito e produção, mas não entra no cálculo atual.

Os valores desejados, como Q_s_i , N_d_i , I_d_i e DM_d_i , também não entram diretamente. A função utiliza produção, emprego, vendas, preços e posições financeiras realizados.

Na célula de código, `expected_profits` significa o resultado esperado do teste, não o lucro esperado economicamente pelos agentes.

Campos atualizados e usos posteriores A função atualiza somente `model.firms.Pi_i`.

O lucro realizado será utilizado:

- por `Bit.set_households_income_firms!(model)`, no cálculo dos dividendos dos proprietários;
- por `Bit.set_gov_revenues!(model)`, no cálculo dos impostos sobre lucros e rendimentos de capital;
- por `Bit.set_firms_deposits!(model)`, nos pagamentos de impostos e dividendos;
- no período seguinte, para formar `model.firms.Pi_e_i`.

O auxiliar `Bit.pos` transforma valores negativos e também `NaN` em zero. Portanto, um depósito `NaN` não gera receita nem cobrança de juros sobre saldo negativo. Outros valores `NaN` continuam se propagando pela fórmula.

Os termos $Y_i / \text{beta_i}$ e $\text{delta_i} / \text{kappa_i}$ não possuem proteção contra denominadores iguais a zero. A função não realiza sorteios, mas seus resultados dependem de emprego, vendas e preços provenientes dos matchings anteriores.

Reexecutar apenas esta célula, sem alterar os demais campos, produz o mesmo lucro.

```
[29]: firms = model.firms

profits_before = copy(firms.Pi_i)

prices = copy(firms.P_i)
production = copy(firms.Y_i)
sales = copy(firms.Q_i)
inventory_changes = copy(firms.DS_i)
```

```

deposits_before = copy(firms.D_i)
loans_before = copy(firms.L_i)

wages = copy(firms.w_i)
employment = copy(firms.N_i)

materials_productivity = copy(firms.beta_i)
materials_price_indices = copy(firms.P_bar_i)

depreciation_rates = copy(firms.delta_i)
capital_productivity = copy(firms.kappa_i)
capital_price_indices = copy(firms.P_CF_i)

product_tax_rates = copy(firms.tau_Y_i)
production_tax_rates = copy(firms.tau_K_i)

loan_rate = model.bank.r
deposit_rate = model.cb.r_bar
household_price_index = model.agg.P_bar_HH
employer_contribution_rate = model.prop.tau_SIF

interest_bearing_deposits =
    Bit.pos(deposits_before)

overdraft_balances =
    Bit.pos(-deposits_before)

sales_and_inventory_value =
    prices .* sales .+
    prices .* inventory_changes

deposit_interest_income =
    deposit_rate .* interest_bearing_deposits

labour_cost =
    (1.0 + employer_contribution_rate) .*
    wages .* employment .* household_price_index

intermediate_input_cost =
    1.0 ./ materials_productivity .*
    materials_price_indices .* production

capital_depreciation_cost =
    depreciation_rates ./ capital_productivity .*
    capital_price_indices .* production

product_net_taxes =

```

```

    product_tax_rates .* prices .* production

production_net_taxes =
    production_tax_rates .* prices .* production

loan_interest_cost =
    loan_rate .* (loans_before .+ overdraft_balances)

expected_profits =
    sales_and_inventory_value .+
    deposit_interest_income .-
    labour_cost .-
    intermediate_input_cost .-
    capital_depreciation_cost .-
    product_net_taxes .-
    production_net_taxes .-
    loan_interest_cost

Bit.set_firms_profits!(model)

@assert all(isapprox.(
    firms.Pi_i,
    expected_profits;
    nans = true,
))

profit_summary = DataFrame(
    component = [
        "Previous stored profit",
        "Sales and inventory change",
        "Deposit interest",
        "Labour cost",
        "Intermediate inputs",
        "Capital depreciation",
        "Product net taxes",
        "Production net taxes",
        "Loan and overdraft interest",
        "Computed profit",
        "Stored profit",
    ],
    amount = [
        sum(profits_before),
        sum(sales_and_inventory_value),
        sum(deposit_interest_income),
        -sum(labour_cost),
        -sum(intermediate_input_cost),
        -sum(capital_depreciation_cost),
    ],
)

```



```

        -sum(product_net_taxes),
        -sum(production_net_taxes),
        -sum(loan_interest_cost),
        sum(expected_profits),
        sum(firms.Pi_i),
    ],
)

profit_summary

```

[29] :

	component	amount
	String	Float64
1	Previous stored profit	10175.1
2	Sales and inventory change	1.34137e5
3	Deposit interest	88.28
4	Labour cost	-34219.4
5	Intermediate inputs	-67824.2
6	Capital depreciation	-12723.7
7	Product net taxes	-1652.06
8	Production net taxes	-974.435
9	Loan and overdraft interest	-6716.02
10	Computed profit	10115.1
11	Stored profit	10115.1

2.31 Lucro realizado do banco

Objetivo econômico A função `Bit.set_bank_profits!(model)` calcula o lucro do banco a partir dos juros recebidos sobre crédito, dos juros pagos sobre depósitos e de sua posição residual de balanço.

Equações

Juros recebidos à taxa bancária O banco recebe a taxa r sobre os empréstimos das empresas e sobre depósitos negativos, que funcionam como crédito:

$$B_r = \sum_i L_i + \sum_i [-D_i]^+ + \sum_h [-D_h]^+$$

$$R_r = rB_r$$

onde:

$$[x]^+ = \max(0, x)$$

O vetor D_h reúne os depósitos dos trabalhadores ativos, inativos, proprietários de empresas e proprietário do banco.

Resultado associado à taxa básica O banco paga a taxa básica sobre depósitos positivos de empresas e famílias:

$$C_{\bar{r}} = \bar{r} \left(\sum_i [D_i]^+ + \sum_h [D_h]^+ \right)$$

A posição residual D_k entra diretamente na fórmula:

$$R_{\bar{r}} = \bar{r} \left(D_k - \sum_i [D_i]^+ - \sum_h [D_h]^+ \right)$$

Se $D_k > 0$, essa posição gera receita. Se $D_k < 0$, gera despesa.

Lucro do banco O lucro realizado é:

$$\Pi_k = r \left(\sum_i L_i + \sum_i [-D_i]^+ + \sum_h [-D_h]^+ \right) + \bar{r} \left(D_k - \sum_i [D_i]^+ - \sum_h [D_h]^+ \right)$$

Definição das variáveis

Símbolo	Código	Significado
Π_k	<code>model.bank.Pi_k</code>	Lucro realizado do banco
r	<code>model.bank.r</code>	Taxa cobrada pelo banco
$\bar{r}$	<code>model.cb.r_bar</code>	Taxa básica definida pelo banco central
L_i	<code>model.firms.L_i[i]</code>	Empréstimo vigente da empresa
D_i	<code>model.firms.D_i[i]</code>	Depósito corporativo da empresa
D_h	depósitos D_h dos diferentes tipos de família	Depósito das famílias
$[-D_i]^+$	<code>firm_overdrafts[i]</code>	Saldo negativo da empresa tratado como crédito
$[-D_h]^+$	<code>household_overdrafts[h]</code>	Saldo negativo da família tratado como crédito
$[D_i]^+$	<code>positive_firm_deposits[i]</code>	Depósito corporativo remunerado
$[D_h]^+$	<code>positive_household_deposits[h]</code>	Depósito familiar remunerado
D_k	<code>model.bank.D_k</code>	Posição residual de balanço do banco
Π_k^e	<code>model.bank.Pi_e_k</code>	Lucro esperado do banco

Valores desejados, esperados e realizados O lucro esperado foi calculado anteriormente como:

$$\Pi_k^e = \Pi_{k,t-1}(1 + \pi^e)(1 + \gamma^e)$$

Ele foi usado para formar o orçamento do proprietário do banco, mas não participa do cálculo atual.

Os empréstimos desejados `DL_d_i` e os novos empréstimos obtidos `DL_i` também não entram diretamente. Neste ponto do passo, `L_i`, `D_i`, `D_h` e `D_k` ainda são os saldos vigentes antes das atualizações financeiras finais. Os novos empréstimos e depósitos serão incorporados posteriormente.

Na célula de código, `expected_bank_profit` representa o resultado esperado do teste, não a expectativa econômica `Pi_e_k`.

Campos atualizados e usos posteriores A função atualiza somente `model.bank.Pi_k`.

O resultado será usado:

- imediatamente por `Bit.set_bank_equity!(model)`;
- por `Bit.set_households_income_bank!(model)`, no cálculo da renda do proprietário;
- por `Bit.set_gov_revenues!(model)`, nos impostos sobre lucros e rendimentos de capital;
- no período seguinte, por `Bit.set_bank_expected_profits!(model)`.

Não há divisões nem sorteios nesta função. Saldos iguais a zero não geram juros. Os empréstimos `L_i` entram diretamente na soma, sem aplicação de parte positiva; portanto, um valor negativo reduziria a base de juros.

A implementação usa `max`, e não `Bit.pos`. Assim, um depósito NaN se propaga para o lucro. Reexecutar somente esta célula, sem alterar os saldos ou as taxas, produz o mesmo resultado.

```
[30]: bank = model.bank
      firms = model.firms

      bank_profit_before = bank.Pi_k

      firm_loans = copy(firms.L_i)
      firm_deposits = copy(firms.D_i)

      active_household_deposits =
          copy(model.w_act.D_h)

      inactive_household_deposits =
          copy(model.w_inact.D_h)

      firm_owner_deposits =
          copy(firms.D_h)

      bank_owner_deposit =
          bank.D_h
```

```

household_deposits = [
    active_household_deposits
    inactive_household_deposits
    firm_owner_deposits
    bank_owner_deposit
]

bank_position_before = bank.D_k
bank_loan_rate = bank.r
base_rate = model.cb.r_bar
zero_balance = zero(bank_position_before)

firm_overdrafts =
    max.(zero_balance, -firm_deposits)

household_overdrafts =
    max.(zero_balance, -household_deposits)

positive_firm_deposits =
    max.(zero_balance, firm_deposits)

positive_household_deposits =
    max.(zero_balance, household_deposits)

bank_rate_base =
    sum(firm_loans) +
    sum(firm_overdrafts) +
    sum(household_overdrafts)

base_rate_base =
    bank_position_before -
    sum(positive_firm_deposits) -
    sum(positive_household_deposits)

expected_bank_profit =
    bank_loan_rate * bank_rate_base +
    base_rate * base_rate_base

Bit.set_bank_profits!(model)

@assert isapprox(
    bank.Pi_k,
    expected_bank_profit;
    nans = true,
)

```

```

profit_summary = DataFrame(
    component = [
        "Previous stored profit",
        "Interest on firm loans",
        "Interest on firm overdrafts",
        "Interest on household overdrafts",
        "Return on bank balancing position",
        "Interest paid on firm deposits",
        "Interest paid on household deposits",
        "Computed bank profit",
        "Stored bank profit",
    ],
    amount = [
        bank_profit_before,
        bank_loan_rate * sum(firm_loans),
        bank_loan_rate * sum(firm_overdrafts),
        bank_loan_rate * sum(household_overdrafts),
        base_rate * bank_position_before,
        -base_rate * sum(positive_firm_deposits),
        -base_rate * sum(positive_household_deposits),
        expected_bank_profit,
        bank.Pi_k,
    ],
)

profit_summary

```

[30]:

	component	amount
	String	Float64
1	Previous stored profit	6476.29
2	Interest on firm loans	6716.02
3	Interest on firm overdrafts	0.0
4	Interest on household overdrafts	0.0
5	Return on bank balancing position	206.504
6	Interest paid on firm deposits	-88.28
7	Interest paid on household deposits	-359.074
8	Computed bank profit	6475.17
9	Stored bank profit	6475.17

2.32 Atualização do patrimônio líquido do banco

Objetivo econômico A função `Bit.set_bank_equity!(model)` incorpora ao patrimônio do banco o lucro realizado após o pagamento do imposto corporativo e dos dividendos.

Equações

Lucro sujeito a distribuições Impostos e dividendos incidem apenas sobre a parte positiva do lucro:

$$\Pi_k^+ = \max(0, \Pi_k)$$

O imposto corporativo é:

$$T_k = \tau_{FIRM} \Pi_k^+$$

Os dividendos correspondem a uma parcela do lucro positivo após o imposto corporativo:

$$DIV_k = \theta_{DIV} (1 - \tau_{FIRM}) \Pi_k^+$$

Variação do patrimônio A parcela do lucro incorporada ao patrimônio é:

$$\Delta E_k = \Pi_k - T_k - DIV_k$$

Portanto:

$$E_{k,t} = E_{k,t-1} + \Delta E_k$$

A regra também pode ser escrita como:

$$\Delta E_k = \begin{cases} (1 - \tau_{FIRM})(1 - \theta_{DIV})\Pi_k, & \text{se } \Pi_k > 0, \\ \Pi_k, & \text{se } \Pi_k \leq 0. \end{cases}$$

Um prejuízo reduz integralmente o patrimônio porque não há imposto nem pagamento de dividendos sobre lucros não positivos.

Definição das variáveis

Símbolo	Código	Significado
$E_{k,t-1}$	bank_equity_before	Patrimônio do banco antes da atualização
$E_{k,t}$	model.bank.E_k	Patrimônio após a atualização
Π_k	model.bank.Pi_k	Lucro realizado do banco
Π_k^+	positive_bank_profit	Parte positiva do lucro
τ_{FIRM}	model.prop.tau_FIRM	Alíquota do imposto corporativo
θ_{DIV}	model.prop.theta_DIV	Parcela distribuída como dividendos
T_k	corporate_tax	Imposto corporativo
DIV_k	dividend_payment	Dividendos pagos pelo banco
ΔE_k	expected_equity_change	Lucro retido ou prejuízo incorporado ao patrimônio

Valores desejados, esperados e realizados `model.bank.Pi_e_k` é o lucro esperado calculado no início do período. Ele foi utilizado para definir o orçamento desejado de consumo e investimento do proprietário do banco, mas não entra nesta atualização.

A função utiliza exclusivamente o lucro realizado `model.bank.Pi_k`, calculado na etapa anterior. Impostos e dividendos são então determinados por regras fixas, sem novas decisões ou expectativas.

Na célula de código, `expected_bank_equity` representa o resultado esperado do teste, não uma expectativa econômica do banco.

Campos atualizados e usos posteriores A função atualiza somente `model.bank.E_k`.

Ainda neste período, o novo patrimônio será usado por `Bit.set_bank_deposits!(model)` no fechamento da posição residual `D_k` do banco.

No período seguinte:

- `Bit.finance_insolvent_firms!(model)` poderá reduzir `E_k` ao absorver perdas de empresas insolventes;
- `Bit.search_and_matching_credit!(model)` usará `E_k` para limitar a capacidade total de concessão de crédito.

A concessão de crédito deste período já ocorreu usando o patrimônio anterior. Esta atualização não altera retroativamente os empréstimos concedidos.

Não há divisões nem sorteios. Lucro igual a zero não altera o patrimônio, e um prejuízo pode tornar `E_k` negativo porque a função não aplica nenhum piso.

A implementação usa `max`, portanto um lucro NaN produz patrimônio NaN.

Esta função não é idempotente: reexecutar a célula adiciona novamente a mesma variação ao patrimônio.

```
[31]: bank = model.bank

bank_equity_before = bank.E_k
realized_bank_profit = bank.Pi_k

dividend_payout_rate =
    model.prop.theta_DIV

corporate_tax_rate =
    model.prop.tau_FIRM

positive_bank_profit =
    max(
        zero(realized_bank_profit),
        realized_bank_profit,
    )

corporate_tax =
    corporate_tax_rate * positive_bank_profit
```

```

dividend_payment =
    dividend_payout_rate *
    (1 - corporate_tax_rate) *
    positive_bank_profit

expected_equity_change =
    realized_bank_profit -
    corporate_tax -
    dividend_payment

expected_bank_equity =
    bank_equity_before +
    expected_equity_change

Bit.set_bank_equity!(model)

@assert isapprox(
    bank.E_k,
    expected_bank_equity;
    nans = true,
)

equity_summary = DataFrame(
    metric = [
        "Opening bank equity",
        "Realized bank profit",
        "Profit subject to distributions",
        "Corporate tax",
        "Dividend payment",
        "Equity change",
        "Computed closing equity",
        "Stored closing equity",
    ],
    value = [
        bank_equity_before,
        realized_bank_profit,
        positive_bank_profit,
        corporate_tax,
        dividend_payment,
        expected_equity_change,
        expected_bank_equity,
        bank.E_k,
    ],
)

equity_summary

```


[31]:

	metric	value
	String	Float64
1	Opening bank equity	89460.0
2	Realized bank profit	6475.17
3	Profit subject to distributions	6475.17
4	Corporate tax	498.665
5	Dividend payment	4696.38
6	Equity change	1280.12
7	Computed closing equity	90740.1
8	Stored closing equity	90740.1

2.33 Renda realizada dos trabalhadores ativos

Objetivo econômico A função `Bit.set_households_income_act!(model)` calcula a renda disponível dos trabalhadores economicamente ativos. Esse grupo inclui tanto empregados quanto desempregados.

Equações

Renda dos empregados Um trabalhador está empregado quando:

$$O_h \neq 0$$

Sua renda é formada pelo salário após a contribuição do trabalhador e o imposto de renda, acrescido do benefício social geral:

$$Y_h = [w_h (1 - \tau_{SIW} - \tau_{INC}(1 - \tau_{SIW})) + sb_{other}] \bar{P}_{HH}$$

O fator sobre o salário também pode ser escrito como:

$$1 - \tau_{SIW} - \tau_{INC}(1 - \tau_{SIW}) = (1 - \tau_{SIW})(1 - \tau_{INC})$$

Portanto, o imposto de renda incide sobre o salário depois da contribuição do trabalhador.

Renda dos desempregados Um trabalhador está desempregado quando:

$$O_h = 0$$

Nesse caso, sua renda é:

$$Y_h = (\theta_{UB} w_h + sb_{other}) \bar{P}_{HH}$$

O valor `w_h` funciona como salário de referência para o benefício de desemprego, e `theta_UB` é sua taxa de reposição.

Definição das variáveis

Símbolo	Código	Significado
h	índice do trabalhador	Trabalhador analisado
Y_h	<code>model.w_act.Y_h[h]</code>	Renda disponível realizada
w_h	<code>model.w_act.w_h[h]</code>	Salário atual ou de referência
O_h	<code>model.w_act.O_h[h]</code>	Empresa empregadora; zero indica desemprego
τ_{SIW}	<code>model.prop.tau_SIW</code>	Contribuição social paga pelo trabalhador
τ_{INC}	<code>model.prop.tau_INC</code>	Alíquota do imposto de renda
θ_{UB}	<code>model.prop.theta_UB</code>	Taxa de reposição do benefício de desemprego
sb_{other}	<code>model.gov.sb_other</code>	Benefício social geral
$\bar{P}_{HH}$	<code>model.agg.P_bar_HH</code>	Índice usado para converter valores reais em nominais
π^e	<code>model.agg.pi_e</code>	Inflação esperada

Valores desejados, esperados e realizados Anteriormente, os orçamentos desejados de consumo e investimento foram calculados usando `households_income_act(model; expected=true)`. Nesse caso, a fórmula da renda recebe o fator adicional:

$$1 + \pi^e$$

Essa renda esperada ajudou a determinar $C_{d,h}$ e $I_{d,h}$, mas não foi armazenada em Y_h .

A chamada atual usa o valor padrão `expected=false`. Portanto:

$$\pi_{\text{usada na função}}^e = 0$$

e a renda é calculada com o emprego, o salário, os benefícios e o índice de preços realizados. Na célula de código, `expected_income` significa o resultado esperado do teste, não a expectativa econômica usada anteriormente.

Campos atualizados e usos posteriores A função atualiza somente:

- `model.w_act.Y_h`.

Depois que as rendas dos demais tipos de família forem calculadas:

- `Bit.set_gambling_transfers!(model)` poderá descontar apostas da renda de trabalhadores selecionados;
- `Bit.set_households_deposits_act!(model)` usará a renda após esses descontos para atualizar os depósitos.

A função não possui denominadores nem realiza sorteios. Trabalhadores com `O_h == 0` são tratados como desempregados, sem exigir que todos estejam empregados. Valores negativos ou NaN não são limitados e se propagam para a renda.

O resultado depende do matching aleatório do mercado de trabalho realizado anteriormente. Re-executar somente esta célula antes das transferências do mercado de apostas produz o mesmo resultado. Reexecutá-la depois dessas transferências sobrescreveria seus descontos sobre Y_h .

```
[32]: workers = model.w_act

income_before = copy(workers.Y_h)
wages = copy(workers.w_h)
occupations = copy(workers.O_h)

worker_contribution_rate =
    model.prop.tau_SIW

income_tax_rate =
    model.prop.tau_INC

unemployment_replacement_rate =
    model.prop.theta_UB

other_social_benefit =
    model.gov.sb_other

household_price_index =
    model.agg.P_bar_HH

expected_income =
    zeros(eltype(income_before), length(wages))

for household in eachindex(wages)
    if occupations[household] != 0
        expected_income[household] =
            (
                wages[household] *
                (
                    1 -
                    worker_contribution_rate -
                    income_tax_rate *
                    (1 - worker_contribution_rate)
                ) +
                other_social_benefit
            ) *
            household_price_index
    else
        expected_income[household] =
            (
                unemployment_replacement_rate *
                wages[household] +
```

```

        other_social_benefit
    ) *
    household_price_index
end
end

Bit.set_households_income_act!(model)

@assert all(isapprox.(
    workers.Y_h,
    expected_income;
    nans = true,
))

income_summary = DataFrame(
    household_id = workers.ID,
    employment_status = ifelse.(
        occupations .!= 0,
        "Employed",
        "Unemployed",
    ),
    occupation = occupations,
    reference_wage = wages,
    previous_income = income_before,
    computed_income = expected_income,
    stored_income = workers.Y_h,
)

first(
    income_summary,
    min(10, nrow(income_summary)),
)

```

[32]:

	household_id	employment_status	occupation	reference_wage	previous_income	computed_income
	Int64	String	Int64	Float64	Float64	Float64
1	1	Employed	1	0.268112	0.766128	0.763288
2	2	Employed	1	0.268112	0.766128	0.763288
3	3	Employed	1	0.268112	0.766128	0.763288
4	4	Employed	2	0.268112	0.766128	0.763288
5	5	Employed	3	0.268112	0.766128	0.763288
6	6	Employed	4	0.268112	0.766128	0.763288
7	7	Employed	5	0.268112	0.766128	0.763288
8	8	Employed	6	0.268112	0.766128	0.763288
9	9	Employed	7	0.268112	0.766128	0.763288
10	10	Employed	8	0.268112	0.766128	0.763288

2.34 Renda realizada dos trabalhadores inativos

Objetivo econômico A função `Bit.set_households_income_inact!(model)` calcula a renda disponível dos trabalhadores inativos, como aposentados e outras pessoas fora da população economicamente ativa.

Equações

Composição da renda Cada trabalhador inativo recebe dois benefícios:

- `sb_inact`: benefício específico dos inativos;
- `sb_other`: benefício social geral pago a todas as famílias.

A renda realizada é:

$$Y_h = (sb_{inact} + sb_{other}) \bar{P}_{HH}$$

Como os benefícios e o índice de preços são escalares, todos os trabalhadores inativos recebem o mesmo valor nesta função.

Definição das variáveis

Símbolo	Código	Significado
h	índice do trabalhador	Trabalhador inativo analisado
H_{inact}	<code>length(model.w_inact)</code>	Número de trabalhadores inativos
Y_h	<code>model.w_inact.Y_h[h]</code>	Renda disponível realizada
sb_{inact}	<code>model.gov.sb_inact</code>	Benefício específico dos inativos
sb_{other}	<code>model.gov.sb_other</code>	Benefício social geral
$\bar{P}_{HH}$	<code>model.agg.P_bar_HH</code>	Índice usado para converter benefícios reais em nominais
π^e	<code>model.agg.pi_e</code>	Inflação esperada
C_h^d	<code>model.w_inact.C_d_h[h]</code>	Orçamento desejado de consumo
I_h^d	<code>model.w_inact.I_d_h[h]</code>	Orçamento desejado de investimento residencial

Valores desejados, esperados e realizados Anteriormente, os orçamentos desejados foram calculados usando a renda esperada:

$$Y_h^e = (sb_{inact} + sb_{other}) \bar{P}_{HH} (1 + \pi^e)$$

Essa renda esperada, após o desconto antecipado de eventuais apostas, foi usada para determinar `C_d_h` e `I_d_h`.

A chamada atual usa `expected=false`. Portanto, não aplica o fator de inflação esperada:

$$Y_h = (sb_{inact} + sb_{other}) \bar{P}_{HH}$$

Os próprios benefícios já foram atualizados anteriormente por `Bit.set_gov_social_benefits!(model)` usando o crescimento esperado. A renda realizada não depende de emprego, salário, consumo desejado ou consumo efetivamente realizado.

Na célula de código, `expected_income` representa o resultado esperado do teste, não a expectativa econômica usada nos orçamentos.

Campos atualizados e usos posteriores A função atualiza somente:

- `model.w_inact.Y_h`.

Posteriormente:

- `Bit.set_gambling_transfers!(model)` poderá descontar apostas da renda de inativos selecionados;
- `Bit.set_households_deposits_inact!(model)` usará a renda após esses descontos para atualizar seus depósitos.

Não há denominadores nem sorteios nesta função. Se não houver trabalhadores inativos, a função apenas atualiza um vetor vazio. Valores negativos ou NaN nos benefícios ou no índice de preços propagam-se para a renda.

O resultado depende indiretamente das expectativas que atualizaram os benefícios e do matching que determinou `P_bar_HH`. Reexecutar somente esta célula antes das transferências de apostas produz o mesmo resultado. Reexecutá-la depois dessas transferências sobrescreveria seus descontos sobre `Y_h`.

```
[33]: inactive_workers = model.w_inact

income_before =
    copy(inactive_workers.Y_h)

inactive_household_count =
    length(inactive_workers)

inactive_social_benefit =
    model.gov.sb_inact

other_social_benefit =
    model.gov.sb_other

household_price_index =
    model.agg.P_bar_HH

expected_income_per_household =
    (
        inactive_social_benefit +
        other_social_benefit
```

```

    ) *
    household_price_index

expected_income =
    fill(
        expected_income_per_household,
        inactive_household_count,
    )

Bit.set_households_income_inact!(model)

@assert all(isapprox.(
    inactive_workers.Y_h,
    expected_income;
    nans = true,
))

income_summary = DataFrame(
    metric = [
        "Inactive households",
        "Inactive social benefit per household",
        "Other social benefit per household",
        "Households price index",
        "Previous total income",
        "Computed income per household",
        "Computed total income",
        "Stored total income",
    ],
    value = [
        inactive_household_count,
        inactive_social_benefit,
        other_social_benefit,
        household_price_index,
        sum(income_before),
        expected_income_per_household,
        sum(expected_income),
        sum(inactive_workers.Y_h),
    ],
)

income_summary

```

[33]:

	metric	value
	String	Float64
1	Inactive households	4130.0
2	Inactive social benefit per household	2.2252
3	Other social benefit per household	0.586788
4	Households price index	1.00223
5	Previous total income	11682.8
6	Computed income per household	2.81827
7	Computed total income	11639.4
8	Stored total income	11639.4

2.35 Renda realizada dos proprietários das empresas

Objetivo econômico A função `Bit.set_households_income_firms!(model)` calcula a renda disponível do proprietário associado a cada empresa. Essa renda combina dividendos líquidos e o benefício social geral.

A função altera a renda do proprietário, `firms.Y_h`, e não o lucro da empresa, `firms.Pi_i`.

Equações

Dividendos líquidos Somente lucros positivos geram dividendos:

$$\Pi_i^+ = \max(0, \Pi_i)$$

Primeiro incide o imposto corporativo. Depois, uma parcela do lucro restante é distribuída e sofre imposto de renda:

$$Y_i^{DIV} = \theta_{DIV}(1 - \tau_{INC})(1 - \tau_{FIRM})\Pi_i^+$$

Se a empresa tiver lucro igual a zero ou prejuízo, essa parcela será zero.

Benefício social Cada proprietário também recebe o benefício social geral convertido em valor nominal:

$$Y_i^{SB} = sb_{other}\bar{P}_{HH}$$

A renda realizada é:

$$Y_i^h = Y_i^{DIV} + Y_i^{SB}$$

ou:

$$Y_i^h = \theta_{DIV}(1 - \tau_{INC})(1 - \tau_{FIRM})\max(0, \Pi_i) + sb_{other}\bar{P}_{HH}$$

Definição das variáveis

Símbolo	Código	Significado
i	índice da empresa	Empresa e proprietário associados
Y_i^h	<code>model.firms.Y_h[i]</code>	Renda disponível realizada do proprietário
Π_i	<code>model.firms.Pi_i[i]</code>	Lucro realizado da empresa
Π_i^+	<code>positive_profits[i]</code>	Parte positiva do lucro
θ_{DIV}	<code>model.prop.theta_DIV</code>	Parcela do lucro distribuída como dividendos
τ_{FIRM}	<code>model.prop.tau_FIRM</code>	Alíquota do imposto corporativo
τ_{INC}	<code>model.prop.tau_INC</code>	Alíquota do imposto de renda
sb_{other}	<code>model.gov.sb_other</code>	Benefício social geral
$\bar{P}_{HH}$	<code>model.agg.P_bar_HH</code>	Índice usado para converter o benefício em valor nominal
Π_i^e	<code>model.firms.Pi_e_i[i]</code>	Lucro esperado da empresa
π^e	<code>model.agg.pi_e</code>	Inflação esperada
C_i^d	<code>model.firms.C_d_h[i]</code>	Orçamento desejado de consumo do proprietário
I_i^d	<code>model.firms.I_d_h[i]</code>	Orçamento desejado de investimento residencial

Valores desejados, esperados e realizados Anteriormente, os orçamentos desejados foram calculados com a renda esperada:

$$Y_i^{h,e} = \theta_{DIV}(1 - \tau_{INC})(1 - \tau_{FIRM}) \max(0, \Pi_i^e) + sb_{other} \bar{P}_{HH}(1 + \pi^e)$$

Nesse cálculo anterior, também foram acrescentados os recebimentos esperados do mercado de apostas antes da definição de `C_d_h` e `I_d_h`.

A chamada atual usa `expected=false`. Portanto, utiliza `Pi_i`, não `Pi_e_i`, e não aplica inflação esperada ao benefício. Os recebimentos realizados das apostas ainda não entram nesta função: eles serão adicionados posteriormente.

Na célula de código, `expected_owner_income` representa o resultado esperado do teste, não a expectativa econômica usada nos orçamentos.

Campos atualizados e usos posteriores A função atualiza somente:

- `model.firms.Y_h`.

Posteriormente:

- `Bit.set_gambling_transfers!(model)` adicionará aos proprietários selecionados os recursos transferidos pelos apostadores;
- `Bit.set_households_deposits_firms!(model)` usará a renda após essas transferências para atualizar os depósitos pessoais dos proprietários.

Não há denominadores nem sorteios nesta função. Contudo, Π_i depende de produção e matchings anteriores. Lucros negativos são permitidos e simplesmente não geram dividendos.

A implementação usa `max`, portanto um lucro NaN produz renda NaN.

Reexecutar somente esta célula antes das transferências de apostas produz o mesmo resultado. Reexecutá-la depois dessas transferências apagaria os recebimentos adicionados a `firms.Y_h`.

```
[34]: firms = model.firms

owner_income_before =
    copy(firms.Y_h)

realized_profits =
    copy(firms.Pi_i)

dividend_payout_rate =
    model.prop.theta_DIV

income_tax_rate =
    model.prop.tau_INC

corporate_tax_rate =
    model.prop.tau_FIRM

other_social_benefit =
    model.gov.sb_other

household_price_index =
    model.agg.P_bar_HH

positive_profits =
    max.(
        zero(eltype(realized_profits)),
        realized_profits,
    )

net_dividend_income =
    dividend_payout_rate .*
    (1 - income_tax_rate) .*
    (1 - corporate_tax_rate) .*
    positive_profits

social_benefit_income =
    other_social_benefit *
    household_price_index

expected_owner_income =
```

```

net_dividend_income .+
social_benefit_income

Bit.set_households_income_firms!(model)

@assert all(isapprox.(
    firms.Y_h,
    expected_owner_income;
    nans = true,
))

income_summary = DataFrame(
    firm_owner_id = firms.ID,
    realized_firm_profit = realized_profits,
    positive_firm_profit = positive_profits,
    net_dividend_income = net_dividend_income,
    social_benefit_income = fill(
        social_benefit_income,
        length(firms),
    ),
    previous_income = owner_income_before,
    computed_income = expected_owner_income,
    stored_income = firms.Y_h,
)

first(
    income_summary,
    min(10, nrow(income_summary)),
)

```

[34]:

	firm_owner_id	realized_firm_profit	positive_firm_profit	net_dividend_income	
	Int64	Float64	Float64	Float64	
1	1	3.72421	3.72421	2.1247	...
2	2	1.2414	1.2414	0.708232	...
3	3	1.2414	1.2414	0.708232	...
4	4	1.2414	1.2414	0.708232	...
5	5	1.2414	1.2414	0.708232	...
6	6	1.2414	1.2414	0.708232	...
7	7	1.2414	1.2414	0.708232	...
8	8	2.48281	2.48281	1.41646	...
9	9	1.2414	1.2414	0.708232	...
10	10	1.2414	1.2414	0.708232	...

2.36 Renda realizada do proprietário do banco

Objetivo econômico `Bit.set_households_income_bank!(model)` calcula a renda realizada do domicílio proprietário do banco. Essa renda combina:

1. dividendos distribuídos sobre lucros bancários positivos, líquidos dos impostos corporativo e

- de renda;
2. o benefício social destinado aos demais domicílios, corrigido pelo índice de preços dos consumidores.

Equações A parcela positiva do lucro bancário realizado é:

$$\Pi_k^+ = \max(0, \Pi_k)$$

Os dividendos líquidos recebidos pelo proprietário são:

$$Y_k^{DIV} = \theta_{DIV}(1 - \tau_{INC})(1 - \tau_{FIRM})\Pi_k^+$$

O benefício social realizado é:

$$Y_k^{SB} = sb_{other}\bar{P}_{HH}$$

Portanto, a renda realizada gravada pela função é:

$$Y_k^h = Y_k^{DIV} + Y_k^{SB}$$

Se o lucro bancário for zero ou negativo, a parcela de dividendos será zero, mas o benefício social continuará sendo incluído. Não há divisão nessa fórmula e, portanto, não há denominadores nulos a tratar.

Definição das variáveis

Símbolo	Código	Significado
Y_k^h	<code>model.bank.Y_h</code>	Renda realizada do domicílio proprietário do banco
Π_k	<code>model.bank.Pi_k</code>	Lucro bancário realizado
Π_k^+	<code>positive_bank_profit</code>	Parcela positiva do lucro realizado
Π_k^e	<code>model.bank.Pi_e_k</code>	Lucro bancário esperado
θ_{DIV}	<code>model.prop.theta_DIV</code>	Fração do lucro positivo distribuída como dividendos
τ_{INC}	<code>model.prop.tau_INC</code>	Alíquota do imposto de renda
τ_{FIRM}	<code>model.prop.tau_FIRM</code>	Alíquota do imposto corporativo
sb_{other}	<code>model.gov.sb_other</code>	Benefício social real destinado aos demais domicílios
$\bar{P}_{HH}$	<code>model.agg.P_bar_HH</code>	Índice de preços dos consumidores
π^e	<code>model.agg.pi_e</code>	Inflação esperada, usada somente no cálculo esperado

Valores desejados, esperados e realizados O cálculo **esperado**, executado anteriormente durante a definição dos orçamentos desejados, utiliza o lucro esperado e incorpora a inflação esperada ao benefício:

$$Y_k^{h,e} = \theta_{DIV}(1 - \tau_{INC})(1 - \tau_{FIRM}) \max(0, \Pi_k^e) + sb_{other} \bar{P}_{HH}(1 + \pi^e)$$

Essa renda esperada ajuda a determinar o consumo desejado `model.bank.C_d_h` e o investimento residencial desejado `model.bank.I_d_h`.

A chamada atual usa `expected=false`. Assim, ela substitui o lucro esperado pelo lucro **realizado** `model.bank.Pi_k` e não aplica o fator $(1 + \pi^e)$ ao benefício. Consumo e investimento realizados podem diferir dos valores desejados devido aos resultados dos mercados.

Na célula de código, `expected_owner_income` significa apenas o resultado que esperamos encontrar ao verificar a implementação; não representa a expectativa econômica do proprietário.

Campos atualizados e usos posteriores A função atualiza somente:

- `model.bank.Y_h`

Posteriormente, `Bit.set_households_deposits_bank!(model)` usa essa renda para calcular a variação dos depósitos do proprietário do banco. Esses depósitos também entram, indiretamente, na atualização da posição financeira agregada do banco.

A função não possui sorteio próprio. Com os mesmos valores de entrada, reexecutá-la produz o mesmo resultado. Entretanto, `model.bank.Pi_k` e outros valores anteriores podem refletir choques ou resultados aleatórios dos mercados; reexecutar essas etapas anteriores pode alterar esta renda.

```
[36]: bank = model.bank

# Save the value that will be updated
owner_income_before = bank.Y_h

# Save the inputs used by the realized-income implementation
realized_bank_profit = bank.Pi_k
dividend_payout_rate = model.prop.theta_DIV
income_tax_rate = model.prop.tau_INC
corporate_tax_rate = model.prop.tau_FIRM
other_social_benefit = model.gov.sb_other
household_price_index = model.agg.P_bar_HH

# Reproduce households_income_bank(model; expected = false)
positive_bank_profit =
    max(zero(realized_bank_profit), realized_bank_profit)

net_dividend_income =
    dividend_payout_rate *
    (1 - income_tax_rate) *
    (1 - corporate_tax_rate) *
    positive_bank_profit
```

```

social_benefit_income =
    other_social_benefit *
    household_price_index

expected_owner_income =
    net_dividend_income +
    social_benefit_income

# Execute the model function exactly once
Bit.set_households_income_bank!(model)

@assert isapprox(
    bank.Y_h,
    expected_owner_income;
    nans = true,
)

income_summary = DataFrame(
    component = [
        "Previous bank-owner income",
        "Realized bank profit",
        "Positive bank profit",
        "Net dividend income",
        "Social benefit income",
        "Computed bank-owner income",
        "Stored bank-owner income",
    ],
    value = [
        owner_income_before,
        realized_bank_profit,
        positive_bank_profit,
        net_dividend_income,
        social_benefit_income,
        expected_owner_income,
        bank.Y_h,
    ],
)

income_summary

```

[36]:

	component	value
	String	Float64
1	Previous bank-owner income	3695.37
2	Realized bank profit	6476.29
3	Positive bank profit	6476.29
4	Net dividend income	3694.78
5	Social benefit income	0.590286
6	Computed bank-owner income	3695.37
7	Stored bank-owner income	3695.37

2.37 Transferências do mercado de apostas

Objetivo econômico `Bit.set_gambling_transfers!(model)` transfere uma parcela da renda realizada dos trabalhadores configurados como apostadores para os proprietários de empresas configurados como recebedores.

A transferência não altera o lucro operacional das empresas. Ela aumenta diretamente a renda pessoal Y_h de seus proprietários.

Equações Para cada trabalhador ativo:

$$S_h^{act} = g \mathbf{1}(ID_h \in \mathcal{A}) \max(0, Y_h^{act})$$

Para cada trabalhador inativo:

$$S_h^{inact} = g \mathbf{1}(ID_h \in \mathcal{I}) \max(0, Y_h^{inact})$$

O volume total de apostas é:

$$V^{gambling} = \sum_h S_h^{act} + \sum_h S_h^{inact}$$

Quando as apostas estão habilitadas, cada proprietário configurado recebe:

$$R_i = \mathbf{1}(ID_i \in \mathcal{O}) \frac{V^{gambling}}{|\mathcal{O}|}$$

As rendas realizadas são então atualizadas por:

$$Y_{h,new}^{act} = Y_{h,old}^{act} - S_h^{act}$$

$$Y_{h,new}^{inact} = Y_{h,old}^{inact} - S_h^{inact}$$

$$Y_{i,new}^{owner} = Y_{i,old}^{owner} + R_i$$

Definição das variáveis

Símbolo	Código	Significado
g	<code>model.prop.gambling_income_share</code>	Parteira da renda positiva destinada às apostas
$\mathcal{A}$	<code>model.prop.gambling_active_workers</code>	ID dos trabalhadores ativos apostadores
$\mathcal{I}$	<code>model.prop.gambling_inactive_workers</code>	ID dos trabalhadores inativos apostadores
$\mathcal{O}$	<code>model.prop.gambling_owner_ids</code>	IDs dos proprietários recebedores
S_h^{act}	<code>active_stakes</code>	Aposta de cada trabalhador ativo
S_h^{inact}	<code>inactive_stakes</code>	Aposta de cada trabalhador inativo
R_i	<code>firm_owner_receipts</code>	Valor recebido por cada proprietário selecionado
Y_h^{act}	<code>model.w_act.Y_h</code>	Renda dos trabalhadores ativos
Y_h^{inact}	<code>model.w_inact.Y_h</code>	Renda dos trabalhadores inativos
Y_i^{owner}	<code>model.firms.Y_h</code>	Renda pessoal dos proprietários das empresas
$V^{gambling}$	<code>model.agg.gambling_volume</code>	Volume total apostado no período

Casos nulos e conservação da transferência Somente rendas positivas geram apostas. Um apostador com renda zero ou negativa aposta zero.

Quando `gambling_income_share` é zero, a implementação retorna vetores de transferências nulos antes de calcular a divisão entre proprietários. Assim, uma lista vazia de proprietários não produz divisão por zero nesse caso.

Na inicialização válida, uma participação positiva exige pelo menos um apostador e um proprietário. O volume total é integralmente redistribuído quando todos os IDs de proprietários configurados estão presentes em `model.firms.ID`. A implementação divide pelo número de IDs configurados, mas credita somente IDs encontrados entre as empresas atuais.

Valores desejados, esperados e realizados Anteriormente, os orçamentos **desejados** de consumo e investimento foram calculados a partir de rendas **esperadas**:

- as apostas esperadas foram descontadas da renda esperada dos trabalhadores;
- os recebimentos esperados foram acrescentados à renda esperada dos proprietários;
- esses valores ajudaram a definir `C_d_h` e `I_d_h`.

A função atual usa as rendas **realizadas** já armazenadas em `Y_h`. Ela efetivamente desconta as apostas e credita os proprietários. O consumo e o investimento realizados não são recalculados; a transferência afetará a poupança financeira restante.

Na célula de código, os nomes iniciados por `expected_` representam o resultado esperado do teste, não expectativas econômicas dos agentes.

Campos atualizados e usos posteriores A função atualiza:

- `model.w_act.Y_h`;
- `model.w_inact.Y_h`;
- `model.firms.Y_h`;
- `model.agg.gambling_volume`.

As próximas funções de depósitos usarão as rendas posteriores às transferências:

- `Bit.set_households_deposits_act!(model)`;
- `Bit.set_households_deposits_inact!(model)`;
- `Bit.set_households_deposits_firms!(model)`.

O volume também será registrado posteriormente por `Bit.collect_data!(model)`.

Não há sorteio dentro desta função: os conjuntos de IDs já estão definidos. Contudo, as rendas realizadas podem depender dos matchings aleatórios executados anteriormente.

Atenção: quando as apostas estão habilitadas, esta função não é idempotente. Reexecutar a célula descontará uma nova rodada de apostas das rendas já reduzidas e adicionará novos recebimentos aos proprietários.

```
[37]: # Save every field updated by the function
active_income_before = copy(model.w_act.Y_h)
inactive_income_before = copy(model.w_inact.Y_h)
firm_owner_income_before = copy(model.firms.Y_h)
gambling_volume_before = model.agg.gambling_volume

gambling_share = model.prop.gambling_income_share
active_gambler_ids = Set(model.prop.gambling_active_worker_ids)
inactive_gambler_ids = Set(model.prop.gambling_inactive_worker_ids)
owner_ids = Set(model.prop.gambling_owner_ids)

active_stakes =
    zeros(eltype(active_income_before), length(active_income_before))
inactive_stakes =
    zeros(eltype(inactive_income_before), length(inactive_income_before))
firm_owner_receipts =
    zeros(eltype(firm_owner_income_before), length(firm_owner_income_before))

# Reproduce gambling_transfers using the realized incomes
if !iszero(gambling_share)
    for (index, id) in pairs(model.w_act.ID)
        if id in active_gambler_ids
            active_stakes[index] =
                gambling_share *
```

```

        max(zero(eltype(active_income_before)),  

↪active_income_before[index])
    end
end

for (index, id) in pairs(model.w_inact.ID)
    if id in inactive_gambler_ids
        inactive_stakes[index] =
            gambling_share *
            max(zero(eltype(inactive_income_before)),  

↪inactive_income_before[index])
    end
end

total_stakes = sum(active_stakes) + sum(inactive_stakes)

# A valid positive-share configuration has at least one owner
if !isempty(owner_ids)
    receipt_per_owner = total_stakes / length(owner_ids)

    for (index, id) in pairs(model.firms.ID)
        if id in owner_ids
            firm_owner_receipts[index] = receipt_per_owner
        end
    end
end

end

expected_active_income =
    active_income_before .- active_stakes
expected_inactive_income =
    inactive_income_before .- inactive_stakes
expected_firm_owner_income =
    firm_owner_income_before .+ firm_owner_receipts
expected_gambling_volume =
    sum(active_stakes) + sum(inactive_stakes)

# Execute the model function exactly once
Bit.set_gambling_transfers!(model)

@assert all(isapprox.(
    model.w_act.Y_h,
    expected_active_income;
    nans = true,
))
@assert all(isapprox.(
    model.w_inact.Y_h,

```

```

    expected_inactive_income;
    nans = true,
))
@assert all(isapprox.(
    model.firms.Y_h,
    expected_firm_owner_income;
    nans = true,
))
@assert isapprox(
    model.agg.gambling_volume,
    expected_gambling_volume;
    nans = true,
)

transfer_summary = DataFrame(
    metric = [
        "Previous gambling volume",
        "Active-worker stakes",
        "Inactive-worker stakes",
        "Stored gambling volume",
        "Firm-owner receipts",
    ],
    value = [
        gambling_volume_before,
        sum(active_stakes),
        sum(inactive_stakes),
        model.agg.gambling_volume,
        sum(firm_owner_receipts),
    ],
)

transfer_summary

```

[37]:

	metric	value
	String	Float64
1	Previous gambling volume	0.0
2	Active-worker stakes	0.0
3	Inactive-worker stakes	0.0
4	Stored gambling volume	0.0
5	Firm-owner receipts	0.0

2.38 Atualização dos depósitos dos trabalhadores ativos

Objetivo econômico `Bit.set_households_deposits_act!(model)` atualiza a posição financeira de cada trabalhador ativo depois da renda realizada, das transferências de apostas e das compras efetivamente realizadas.

O campo `D_h` é uma posição líquida: valores positivos representam depósitos; valores negativos representam dívida do domicílio com o banco.

Equações A remuneração dos depósitos positivos é:

$$J_h^{dep} = \bar{r} \max(0, D_{h,old})$$

O custo financeiro da dívida é:

$$J_h^{debt} = r \max(0, -D_{h,old})$$

As despesas realizadas incluem os impostos sobre consumo e formação de capital:

$$E_h^C = (1 + \tau_{VAT})C_h$$

$$E_h^I = (1 + \tau_{CF})I_h$$

A variação dos depósitos é:

$$\Delta D_h = Y_h - E_h^C - E_h^I + J_h^{dep} - J_h^{debt}$$

Finalmente:

$$D_{h,new} = D_{h,old} + \Delta D_h$$

Definição das variáveis

Símbolo	Código	Significado
$D_{h,old}$	<code>active_deposits_before</code>	Depósito ou dívida antes da atualização
$D_{h,new}$	<code>model.w_act.D_h</code>	Nova posição financeira do trabalhador
ΔD_h	<code>expected_deposit_change</code>	Poupança financeira líquida do período
Y_h	<code>model.w_act.Y_h</code>	Renda realizada após as transferências de apostas
C_h	<code>model.w_act.C_h</code>	Consumo realizado
I_h	<code>model.w_act.I_h</code>	Investimento residencial realizado
τ_{VAT}	<code>model.prop.tau_VAT</code>	Alíquota do imposto sobre consumo
τ_{CF}	<code>model.prop.tau_CF</code>	Alíquota sobre formação de capital
$\bar{r}$	<code>model.cb.r_bar</code>	Taxa recebida sobre depósitos positivos
r	<code>model.bank.r</code>	Taxa paga sobre saldos negativos

Símbolo	Código	Significado
J_h^{dep}	<code>deposit_interest</code>	Juros recebidos sobre depósitos
J_h^{debt}	<code>debt_interest</code>	Juros pagos sobre dívidas

Tratamento dos saldos Para cada trabalhador:

- se $D_{h,old} > 0$, ele recebe juros à taxa $\bar{r}$;
- se $D_{h,old} < 0$, ele paga juros à taxa r ;
- se $D_{h,old} = 0$, ambos os componentes de juros são zero.

A fórmula não contém denominadores. Um vetor vazio de trabalhadores ativos também é tratado normalmente pelas operações vetoriais.

Valores desejados, esperados e realizados Os valores **desejados** `C_d_h` e `I_d_h` foram definidos anteriormente a partir da renda esperada, já considerando as apostas esperadas.

Os valores `C_h` e `I_h` são **realizados**: correspondem ao que os trabalhadores conseguiram comprar no mercado. A renda `Y_h` também é realizada e, neste ponto da sequência, já contém o desconto das apostas efetivas.

A nova posição `D_h` é o resíduo financeiro realizado. Portanto, consumo ou investimento abaixo do desejado tende a deixar mais recursos nos depósitos, mantendo constantes os demais componentes.

Na célula de código, `expected_active_deposits` é apenas o resultado esperado da verificação da implementação; não representa uma expectativa econômica dos trabalhadores.

Campos atualizados e usos posteriores A função atualiza somente:

- `model.w_act.D_h`.

Mais adiante no mesmo passo, `Bit.set_bank_deposits!(model)` incluirá esses saldos na posição agregada do banco. Em períodos posteriores, os saldos positivos ou negativos também influenciarão os juros dos trabalhadores e o lucro bancário.

Não há sorteio dentro desta função. Contudo, renda, consumo e investimento realizados podem depender dos matchings aleatórios anteriores.

Atenção: a função não é idempotente. Reexecutar a célula aplicará novamente os fluxos do mesmo período e calculará juros sobre os depósitos já atualizados.

```
[38]: # Save the field that will be updated and all current-period inputs
active_deposits_before = copy(model.w_act.D_h)
active_income = copy(model.w_act.Y_h)
realized_consumption = copy(model.w_act.C_h)
realized_housing_investment = copy(model.w_act.I_h)

vat_rate = model.prop.tau_VAT
capital_formation_tax_rate = model.prop.tau_CF
deposit_rate = model.cb.r_bar
```

```

loan_rate = model.bank.r

zero_balance = zero(eltype(active_deposits_before))

positive_deposits =
    max.(zero_balance, active_deposits_before)
outstanding_debt =
    max.(zero_balance, -active_deposits_before)

consumption_outflow =
    (1 + vat_rate) .* realized_consumption
investment_outflow =
    (1 + capital_formation_tax_rate) .* realized_housing_investment

deposit_interest =
    deposit_rate .* positive_deposits
debt_interest =
    loan_rate .* outstanding_debt

expected_deposit_change =
    active_income .-
    consumption_outflow .-
    investment_outflow .+
    deposit_interest .-
    debt_interest

expected_active_deposits =
    active_deposits_before .+ expected_deposit_change

# Execute the model function exactly once
Bit.set_households_deposits_act!(model)

@assert all(isapprox.(
    model.w_act.D_h,
    expected_active_deposits;
    nans = true,
))

deposit_summary = DataFrame(
    component = [
        "Opening net position",
        "Realized income",
        "Consumption including VAT",
        "Housing investment including tax",
        "Interest on positive deposits",
        "Interest on outstanding debt",
        "Net deposit change",
    ],

```

```

        "Closing net position",
    ],
    value = [
        sum(active_deposits_before),
        sum(active_income),
        sum(consumption_outflow),
        sum(investment_outflow),
        sum(deposit_interest),
        sum(debt_interest),
        sum(expected_deposit_change),
        sum(model.w_act.D_h),
    ],
)

deposit_summary

```

[38]:

	component	value
	String	Float64
1	Opening net position	1.08247e5
2	Realized income	21929.0
3	Consumption including VAT	0.0
4	Housing investment including tax	0.0
5	Interest on positive deposits	178.168
6	Interest on outstanding debt	0.0
7	Net deposit change	22107.2
8	Closing net position	1.30355e5

2.39 Atualização dos depósitos dos trabalhadores inativos

Objetivo econômico `Bit.set_households_deposits_inact!(model)` atualiza a posição financeira de cada trabalhador inativo depois da renda realizada, das transferências de apostas e das compras efetivamente realizadas.

O campo `D_h` é uma posição líquida: valores positivos representam depósitos; valores negativos representam dívida com o banco.

Equações Os juros recebidos sobre depósitos positivos são:

$$J_h^{dep} = \bar{r} \max(0, D_{h,old})$$

Os juros pagos sobre saldos negativos são:

$$J_h^{debt} = r \max(0, -D_{h,old})$$

As despesas realizadas, incluindo impostos, são:

$$E_h^C = (1 + \tau_{VAT})C_h$$

$$E_h^I = (1 + \tau_{CF})I_h$$

A variação da posição financeira é:

$$\Delta D_h = Y_h - E_h^C - E_h^I + J_h^{dep} - J_h^{debt}$$

O novo saldo é:

$$D_{h,new} = D_{h,old} + \Delta D_h$$

Definição das variáveis

Símbolo	Código	Significado
$D_{h,old}$	<code>inactive_deposits_before</code>	Depósito ou dívida antes da atualização
$D_{h,new}$	<code>model.w_inact.D_h</code>	Nova posição financeira do inativo
ΔD_h	<code>expected_deposit_change</code>	Poupança financeira líquida do período
Y_h	<code>model.w_inact.Y_h</code>	Renda realizada após as transferências de apostas
C_h	<code>model.w_inact.C_h</code>	Consumo realizado
I_h	<code>model.w_inact.I_h</code>	Investimento residencial realizado
τ_{VAT}	<code>model.prop.tau_VAT</code>	Alíquota do imposto sobre consumo
τ_{CF}	<code>model.prop.tau_CF</code>	Alíquota sobre formação de capital
$\bar{r}$	<code>model.cb.r_bar</code>	Taxa recebida sobre depósitos positivos
r	<code>model.bank.r</code>	Taxa paga sobre saldos negativos
J_h^{dep}	<code>deposit_interest</code>	Juros recebidos sobre depósitos
J_h^{debt}	<code>debt_interest</code>	Juros pagos sobre dívidas

Tratamento dos saldos Para cada trabalhador inativo:

- se $D_{h,old} > 0$, ele recebe juros à taxa $\bar{r}$;
- se $D_{h,old} < 0$, ele paga juros à taxa r ;
- se $D_{h,old} = 0$, ambos os componentes de juros são zero.

Não há denominadores. Se não existirem trabalhadores inativos, as operações são aplicadas normalmente aos vetores vazios.

Valores desejados, esperados e realizados Os valores **desejados** $C_{d,h}$ e $I_{d,h}$ foram calculados anteriormente com a renda esperada dos inativos, incluindo benefícios esperados e o desconto de apostas esperadas.

Os valores C_h e I_h são **realizados**, pois resultam das quantidades efetivamente compradas no mercado. A renda Y_h também é realizada e já inclui o desconto das apostas efetivas da etapa anterior.

O novo D_h é o resíduo financeiro realizado. Não existe um depósito desejado ou esperado calculado por esta função.

Na célula de código, `expected_inactive_deposits` representa somente o resultado esperado da verificação da implementação, não uma expectativa econômica dos agentes.

Campos atualizados e usos posteriores A função atualiza somente:

- `model.w_inact.D_h`.

Mais adiante no mesmo passo, `Bit.set_bank_deposits!(model)` incluirá esses saldos na posição agregada do banco. Nos períodos seguintes, depósitos positivos e dívidas também influenciarão os juros dos inativos e o lucro bancário.

Não há sorteio dentro desta função. Entretanto, consumo e investimento realizados podem depender dos matchings aleatórios anteriores.

Atenção: a função não é idempotente. Reexecutar a célula aplicará novamente os fluxos do período e calculará juros sobre os saldos já atualizados.

```
[39]: # Save the field that will be updated and all current-period inputs
inactive_deposits_before = copy(model.w_inact.D_h)
inactive_income = copy(model.w_inact.Y_h)
realized_consumption = copy(model.w_inact.C_h)
realized_housing_investment = copy(model.w_inact.I_h)

vat_rate = model.prop.tau_VAT
capital_formation_tax_rate = model.prop.tau_CF
deposit_rate = model.cb.r_bar
loan_rate = model.bank.r

zero_balance = zero(eltype(inactive_deposits_before))

positive_deposits =
    max.(zero_balance, inactive_deposits_before)
outstanding_debt =
    max.(zero_balance, -inactive_deposits_before)

consumption_outflow =
    (1 + vat_rate) .* realized_consumption
investment_outflow =
    (1 + capital_formation_tax_rate) .* realized_housing_investment
```

```

deposit_interest =
    deposit_rate .* positive_deposits
debt_interest =
    loan_rate .* outstanding_debt

expected_deposit_change =
    inactive_income .-
    consumption_outflow .-
    investment_outflow .+
    deposit_interest .-
    debt_interest

expected_inactive_deposits =
    inactive_deposits_before .+ expected_deposit_change

# Execute the model function exactly once
Bit.set_households_deposits_inact!(model)

@assert all(isapprox.(
    model.w_inact.D_h,
    expected_inactive_deposits;
    nans = true,
))

deposit_summary = DataFrame(
    component = [
        "Opening net position",
        "Realized income",
        "Consumption including VAT",
        "Housing investment including tax",
        "Interest on positive deposits",
        "Interest on outstanding debt",
        "Net deposit change",
        "Closing net position",
    ],
    value = [
        sum(inactive_deposits_before),
        sum(inactive_income),
        sum(consumption_outflow),
        sum(investment_outflow),
        sum(deposit_interest),
        sum(debt_interest),
        sum(expected_deposit_change),
        sum(model.w_inact.D_h),
    ],
)

```

```
deposit_summary
```

[39]:

	component	value
	String	Float64
1	Opening net position	57669.1
2	Realized income	11682.8
3	Consumption including VAT	0.0
4	Housing investment including tax	0.0
5	Interest on positive deposits	94.9194
6	Interest on outstanding debt	0.0
7	Net deposit change	11777.7
8	Closing net position	69446.8

2.40 Atualização dos depósitos dos proprietários das empresas

Objetivo econômico `Bit.set_households_deposits_firms!(model)` atualiza a posição financeira pessoal de cada proprietário de empresa após sua renda realizada, os recebimentos das apostas e as compras efetivamente realizadas.

O campo atualizado é `model.firms.D_h`, referente ao domicílio proprietário. Ele é diferente de `model.firms.D_i`, que representa os depósitos operacionais da empresa.

Equações Os juros recebidos sobre depósitos pessoais positivos são:

$$J_i^{dep} = \bar{r} \max(0, D_{i,old}^h)$$

Os juros pagos sobre dívidas pessoais são:

$$J_i^{debt} = r \max(0, -D_{i,old}^h)$$

As despesas realizadas, incluindo impostos, são:

$$E_i^C = (1 + \tau_{VAT})C_i^h$$

$$E_i^I = (1 + \tau_{CF})I_i^h$$

A variação da posição financeira pessoal é:

$$\Delta D_i^h = Y_i^h - E_i^C - E_i^I + J_i^{dep} - J_i^{debt}$$

O novo saldo é:

$$D_{i,new}^h = D_{i,old}^h + \Delta D_i^h$$

Definição das variáveis

Símbolo	Código	Significado
$D_{i,old}^h$	<code>owner_deposits_before</code>	Depósito ou dívida pessoal anterior
$D_{i,new}^h$	<code>model.firms.D_h</code>	Nova posição financeira do proprietário
D_i	<code>model.firms.D_i</code>	Depósito operacional da empresa, não alterado aqui
ΔD_i^h	<code>expected_deposit_change</code>	Poupança financeira pessoal do período
Y_i^h	<code>model.firms.Y_h</code>	Renda realizada do proprietário, incluindo apostas recebidas
C_i^h	<code>model.firms.C_h</code>	Consumo realizado do proprietário
I_i^h	<code>model.firms.I_h</code>	Investimento residencial realizado
τ_{VAT}	<code>model.prop.tau_VAT</code>	Alíquota do imposto sobre consumo
τ_{CF}	<code>model.prop.tau_CF</code>	Alíquota sobre formação de capital
$\bar{r}$	<code>model.cb.r_bar</code>	Taxa recebida sobre depósitos positivos
r	<code>model.bank.r</code>	Taxa paga sobre saldos negativos
J_i^{dep}	<code>deposit_interest</code>	Juros recebidos sobre depósitos pessoais
J_i^{debt}	<code>debt_interest</code>	Juros pagos sobre dívidas pessoais

Tratamento dos saldos Para cada proprietário:

- se $D_{i,old}^h > 0$, ele recebe juros à taxa $\bar{r}$;
- se $D_{i,old}^h < 0$, ele paga juros à taxa r ;
- se $D_{i,old}^h = 0$, os dois componentes de juros são zero.

Não há denominadores. Vetores vazios também são tratados normalmente.

Valores desejados, esperados e realizados Os valores **desejados** `C_d_h` e `I_d_h` foram calculados anteriormente com a renda esperada dos proprietários. Essa renda combinava lucros esperados positivos, benefícios e recebimentos esperados das apostas.

Neste ponto:

- `model.firms.Y_h` contém a renda **realizada**, incluindo as transferências de apostas;
- `model.firms.C_h` e `model.firms.I_h` contêm consumo e investimento **realizados** pelo mercado;

- `model.firms.D_h` passa a registrar o resíduo financeiro efetivamente acumulado.

A função não calcula um depósito desejado ou esperado. Na célula de código, `expected_owner_deposits` significa apenas o resultado esperado da verificação.

Campos atualizados e usos posteriores A função atualiza somente:

- `model.firms.D_h`.

Ela não altera `model.firms.D_i`. Os depósitos operacionais serão atualizados posteriormente por `Bit.set_firms_deposits!(model)`.

Mais adiante, `Bit.set_bank_deposits!(model)` incorporará separadamente os depósitos pessoais dos proprietários e os depósitos operacionais das empresas à posição agregada do banco. Nos períodos seguintes, `D_h` também afetará juros pessoais e lucros bancários.

Não há sorteio dentro desta função, mas renda, consumo, investimento e recebimentos podem depender dos resultados aleatórios anteriores.

Atenção: a função não é idempotente. Reexecutar a célula aplicará novamente os fluxos do período sobre os saldos já atualizados.

```
[40]: # Save the updated field, the related business field, and current-period inputs
owner_deposits_before = copy(model.firms.D_h)
firm_operating_deposits_before = copy(model.firms.D_i)
owner_income = copy(model.firms.Y_h)
realized_consumption = copy(model.firms.C_h)
realized_housing_investment = copy(model.firms.I_h)

vat_rate = model.prop.tau_VAT
capital_formation_tax_rate = model.prop.tau_CF
deposit_rate = model.cb.r_bar
loan_rate = model.bank.r

zero_balance = zero(eltype(owner_deposits_before))

positive_deposits =
    max.(zero_balance, owner_deposits_before)
outstanding_debt =
    max.(zero_balance, -owner_deposits_before)

consumption_outflow =
    (1 + vat_rate) .* realized_consumption
investment_outflow =
    (1 + capital_formation_tax_rate) .* realized_housing_investment

deposit_interest =
    deposit_rate .* positive_deposits
debt_interest =
    loan_rate .* outstanding_debt
```

```

expected_deposit_change =
    owner_income .-
    consumption_outflow .-
    investment_outflow .+
    deposit_interest .-
    debt_interest

expected_owner_deposits =
    owner_deposits_before .+ expected_deposit_change

# Execute the model function exactly once
Bit.set_households_deposits_firms!(model)

@assert all(isapprox.(
    model.firms.D_h,
    expected_owner_deposits;
    nans = true,
))
@assert all(isapprox.(
    model.firms.D_i,
    firm_operating_deposits_before;
    nans = true,
))

deposit_summary = DataFrame(
    component = [
        "Opening owner net position",
        "Realized owner income",
        "Consumption including VAT",
        "Housing investment including tax",
        "Interest on positive deposits",
        "Interest on outstanding debt",
        "Net deposit change",
        "Closing owner net position",
    ],
    value = [
        sum(owner_deposits_before),
        sum(owner_income),
        sum(consumption_outflow),
        sum(investment_outflow),
        sum(deposit_interest),
        sum(debt_interest),
        sum(expected_deposit_change),
        sum(model.firms.D_h),
    ],
)

```

```
deposit_summary
```

[40]:

	component	value
	String	Float64
1	Opening owner net position	35683.3
2	Realized owner income	7228.82
3	Consumption including VAT	0.0
4	Housing investment including tax	0.0
5	Interest on positive deposits	58.7323
6	Interest on outstanding debt	0.0
7	Net deposit change	7287.55
8	Closing owner net position	42970.9

2.41 Atualização do depósito do proprietário do banco

Objetivo econômico `Bit.set_households_deposits_bank!(model)` atualiza a posição financeira pessoal do domicílio proprietário do banco após sua renda e suas compras realizadas.

`model.bank.D_h` pertence ao proprietário do banco. Ele é diferente de `model.bank.D_k`, que representa a posição líquida do próprio banco em seu balanço.

Equações Os juros recebidos sobre um depósito pessoal positivo são:

$$J_k^{dep} = \bar{r} \max(0, D_{k,old}^h)$$

Os juros pagos sobre uma dívida pessoal são:

$$J_k^{debt} = r \max(0, -D_{k,old}^h)$$

As despesas realizadas, incluindo impostos, são:

$$E_k^C = (1 + \tau_{VAT}) C_k^h$$

$$E_k^I = (1 + \tau_{CF}) I_k^h$$

A variação da posição financeira pessoal é:

$$\Delta D_k^h = Y_k^h - E_k^C - E_k^I + J_k^{dep} - J_k^{debt}$$

O novo saldo é:

$$D_{k,new}^h = D_{k,old}^h + \Delta D_k^h$$

Definição das variáveis

Símbolo	Código	Significado
$D_{k,old}^h$	<code>bank_owner_deposit_before</code>	Depósito ou dívida pessoal anterior
$D_{k,new}^h$	<code>model.bank.D_h</code>	Nova posição financeira do proprietário
D_k	<code>model.bank.D_k</code>	Posição líquida do próprio banco, não alterada aqui
ΔD_k^h	<code>expected_deposit_change</code>	Poupança financeira pessoal do período
Y_k^h	<code>model.bank.Y_h</code>	Renda realizada do proprietário do banco
C_k^h	<code>model.bank.C_h</code>	Consumo realizado
I_k^h	<code>model.bank.I_h</code>	Investimento residencial realizado
τ_{VAT}	<code>model.prop.tau_VAT</code>	Alíquota do imposto sobre consumo
τ_{CF}	<code>model.prop.tau_CF</code>	Alíquota sobre formação de capital
$\bar{r}$	<code>model.cb.r_bar</code>	Taxa recebida sobre depósitos positivos
r	<code>model.bank.r</code>	Taxa paga sobre saldos negativos
J_k^{dep}	<code>deposit_interest</code>	Juros recebidos sobre o depósito
J_k^{debt}	<code>debt_interest</code>	Juros pagos sobre a dívida

Tratamento do saldo

- Se $D_{k,old}^h > 0$, o proprietário recebe juros à taxa $\bar{r}$.
- Se $D_{k,old}^h < 0$, ele paga juros à taxa r .
- Se $D_{k,old}^h = 0$, ambos os componentes de juros são zero.

A fórmula é escalar e não possui denominadores.

Valores desejados, esperados e realizados O consumo desejado `C_d_h` e o investimento desejado `I_d_h` foram calculados anteriormente com a renda **esperada** do proprietário, baseada em `model.bank.Pi_e_k` e na inflação esperada.

Nesta etapa:

- `model.bank.Y_h` contém a renda **realizada**, baseada no lucro realizado `model.bank.Pi_k`;
- `model.bank.C_h` e `model.bank.I_h` são consumo e investimento **realizados** no mercado;
- `model.bank.D_h` passa a registrar o resíduo financeiro efetivo.

O proprietário do banco não paga nem recebe as transferências implementadas por `set_gambling_transfers!`; elas envolvem somente trabalhadores e proprietários de empresas.

Na célula de código, `expected_bank_owner_deposit` representa apenas o resultado esperado da verificação, não uma expectativa econômica do proprietário.

Campos atualizados e usos posteriores A função atualiza somente:

- `model.bank.D_h`.

Ela não altera `model.bank.D_k`. Mais adiante no mesmo passo, `Bit.set_bank_deposits!(model)` incluirá `bank.D_h`, junto aos demais depósitos e empréstimos, no cálculo de `bank.D_k`.

Nos períodos posteriores, esse saldo pessoal também influencia os juros do proprietário e o lucro bancário.

Não há sorteio dentro desta função. Entretanto, a renda e as compras realizadas podem depender de lucros e matchings aleatórios anteriores.

Atenção: a função não é idempotente. Reexecutar a célula aplicará novamente os fluxos do período sobre o saldo já atualizado.

[41]: *# Save the updated field, the bank position, and current-period inputs*

```
bank_owner_deposit_before = model.bank.D_h
bank_net_position_before = model.bank.D_k
bank_owner_income = model.bank.Y_h
realized_consumption = model.bank.C_h
realized_housing_investment = model.bank.I_h

vat_rate = model.prop.tau_VAT
capital_formation_tax_rate = model.prop.tau_CF
deposit_rate = model.cb.r_bar
loan_rate = model.bank.r

zero_balance = zero(bank_owner_deposit_before)

positive_deposit =
    max(zero_balance, bank_owner_deposit_before)
outstanding_debt =
    max(zero_balance, -bank_owner_deposit_before)

consumption_outflow =
    (1 + vat_rate) * realized_consumption
investment_outflow =
    (1 + capital_formation_tax_rate) * realized_housing_investment

deposit_interest =
    deposit_rate * positive_deposit
debt_interest =
    loan_rate * outstanding_debt

expected_deposit_change =
    bank_owner_income -
```

```

    consumption_outflow -
    investment_outflow +
    deposit_interest -
    debt_interest

expected_bank_owner_deposit =
    bank_owner_deposit_before + expected_deposit_change

# Execute the model function exactly once
Bit.set_households_deposits_bank!(model)

@assert isapprox(
    model.bank.D_h,
    expected_bank_owner_deposit;
    nans = true,
)
@assert isapprox(
    model.bank.D_k,
    bank_net_position_before;
    nans = true,
)

deposit_summary = DataFrame(
    component = [
        "Opening owner net position",
        "Realized owner income",
        "Consumption including VAT",
        "Housing investment including tax",
        "Interest on positive deposit",
        "Interest on outstanding debt",
        "Net deposit change",
        "Closing owner net position",
    ],
    value = [
        bank_owner_deposit_before,
        bank_owner_income,
        consumption_outflow,
        investment_outflow,
        deposit_interest,
        debt_interest,
        expected_deposit_change,
        model.bank.D_h,
    ],
)

deposit_summary

```

[41]:

	component	value
	String	Float64
1	Opening owner net position	18241.3
2	Realized owner income	3695.37
3	Consumption including VAT	0.0
4	Housing investment including tax	0.0
5	Interest on positive deposit	30.0239
6	Interest on outstanding debt	0.0
7	Net deposit change	3725.39
8	Closing owner net position	21966.7

2.42 Atualização do patrimônio do banco central

Objetivo econômico `Bit.set_central_bank_equity!(model)` acumula no patrimônio do banco central o resultado financeiro do período.

O banco central recebe juros sobre a dívida pública e paga juros sobre a posição positiva do banco comercial junto a ele. A implementação não armazena o lucro em um campo próprio: ele é calculado como valor intermediário e incorporado diretamente a `E_CB`.

Equações A receita de juros sobre a dívida pública é:

$$J_G = r_G L_G$$

O termo de juros associado à posição do banco comercial é:

$$J_k = \bar{r} D_k$$

O lucro do banco central é:

$$\Pi_{CB} = r_G L_G - \bar{r} D_k$$

O patrimônio atualizado é:

$$E_{CB,new} = E_{CB,old} + \Pi_{CB}$$

Definição das variáveis

Símbolo	Código	Significado
$E_{CB,old}$	<code>central_bank_equity_before</code>	Patrimônio anterior do banco central
$E_{CB,new}$	<code>model.cb.E_CB</code>	Patrimônio atualizado do banco central
Π_{CB}	<code>expected_central_bank_profit</code>	Lucro calculado no período
L_G	<code>model.gov.L_G</code>	Estoque atual da dívida pública

Símbolo	Código	Significado
D_k	<code>model.bank.D_k</code>	Posição líquida atual do banco comercial
r_G	<code>model.cb.r_G</code>	Taxa de juros da dívida pública
$\bar{r}$	<code>model.cb.r_bar</code>	Taxa básica de juros
J_G	<code>government_interest_income</code>	Receita de juros da dívida pública
J_k	<code>commercial_bank_interest_term</code>	Termo de juros da posição bancária

Sinais e momento do cálculo A implementação não aplica `max` nem restringe os sinais:

- um $D_k > 0$ gera o termo subtraído $\bar{r}D_k$;
- um $D_k < 0$ transforma essa parcela em receita para o banco central;
- valores negativos de L_G ou das taxas também são usados diretamente.

Não há denominadores.

Nesta posição da sequência, `model.bank.D_k` ainda não incorpora os depósitos familiares que acabaram de ser atualizados. A nova posição bancária será calculada somente mais adiante por `Bit.set_bank_deposits!(model)`.

Valores desejados, esperados e realizados Esta função não possui versões desejada ou esperada. Ela usa diretamente os valores correntes de `L_G`, `D_k`, `r_G` e `r_bar` para calcular o resultado financeiro **realizado**.

Também não existe um campo `Pi_CB` no modelo. O lucro é apenas o incremento entre o patrimônio anterior e o novo patrimônio.

Na célula de código, `expected_central_bank_profit` e `expected_central_bank_equity` são os resultados esperados da verificação, não expectativas econômicas do banco central.

Campos atualizados e usos posteriores A função atualiza somente:

- `model.cb.E_CB`.

Ela não altera `model.gov.L_G` nem `model.bank.D_k`. As etapas posteriores atualizarão a dívida pública, a posição externa e a posição do banco comercial. Esses estoques, juntamente com `E_CB`, compõem a identidade patrimonial verificada por `Bit.get_accounting_identity_banks(model)`.

O patrimônio atualizado também será o saldo inicial acumulado no próximo período.

Não há sorteio dentro desta função. Entretanto, taxas e posições financeiras podem refletir resultados aleatórios das etapas anteriores.

Atenção: a função não é idempotente. Reexecutá-la com os mesmos valores de `L_G` e `D_k` acrescentará o mesmo lucro novamente ao patrimônio.

```
[42]: # Save the updated field and the current financial positions
      central_bank_equity_before = model.cb.E_CB
```

```

government_debt = model.gov.L_G
commercial_bank_position = model.bank.D_k

government_bond_rate = model.cb.r_G
policy_rate = model.cb.r_bar

# Reproduce central_bank_profits and central_bank_equity
government_interest_income =
    government_bond_rate * government_debt
commercial_bank_interest_term =
    policy_rate * commercial_bank_position

expected_central_bank_profit =
    government_interest_income -
    commercial_bank_interest_term

expected_central_bank_equity =
    central_bank_equity_before +
    expected_central_bank_profit

# Execute the model function exactly once
Bit.set_central_bank_equity!(model)

@assert isapprox(
    model.cb.E_CB,
    expected_central_bank_equity;
    nans = true,
)
@assert isapprox(
    model.gov.L_G,
    government_debt;
    nans = true,
)
@assert isapprox(
    model.bank.D_k,
    commercial_bank_position;
    nans = true,
)

equity_summary = DataFrame(
    component = [
        "Opening central-bank equity",
        "Government interest income",
        "Bank-position interest term",
        "Computed central-bank profit",
        "Closing central-bank equity",
    ],

```

```

    value = [
        central_bank_equity_before,
        government_interest_income,
        commercial_bank_interest_term,
        expected_central_bank_profit,
        model.cb.E_CB,
    ],
)

equity_summary

```

[42]:

	component	value
	String	Float64
1	Opening central-bank equity	1.0618e5
2	Government interest income	2089.1
3	Bank-position interest term	208.097
4	Computed central-bank profit	1881.0
5	Closing central-bank equity	1.08061e5

2.43 Receitas realizadas do governo

Objetivo econômico `Bit.set_gov_revenues!(model)` soma as contribuições sociais e os impostos realizados no período. A função usa bases tributárias correntes de salários, consumo, investimento, lucros, produção e exportações.

Equações A massa salarial considerada inclui somente trabalhadores ativos com ocupação diferente de zero:

$$W^{emp} = \sum_{h:O_h \neq 0} w_h$$

O consumo e o investimento residenciais agregados incluem todos os tipos de domicílio:

$$C^H = \sum_h C_h^{act} + \sum_h C_h^{inact} + \sum_i C_i^{owner} + C_k^{owner}$$

$$I^H = \sum_h I_h^{act} + \sum_h I_h^{inact} + \sum_i I_i^{owner} + I_k^{owner}$$

A base positiva de lucros é:

$$\Pi^+ = \sum_i pos(\Pi_i) + pos(\Pi_k)$$

A função auxiliar `pos` é definida economicamente por:

$$pos(x) = \begin{cases} 0, & x < 0 \text{ ou } x = \text{NaN} \\ x, & \text{caso contrário} \end{cases}$$

Os componentes da receita são:

$$\begin{aligned}
T^{SI} &= (\tau_{SIF} + \tau_{SIW})\bar{P}_{HH}W^{emp} \\
T^{LAB} &= \tau_{INC}(1 - \tau_{SIW})\bar{P}_{HH}W^{emp} \\
T^{VAT} &= \tau_{VAT}C^H \\
T^{CAP} &= \tau_{INC}(1 - \tau_{FIRM})\theta_{DIV}\Pi^+ \\
T^{FIRM} &= \tau_{FIRM}\Pi^+ \\
T^{CF} &= \tau_{CF}I^H \\
T^{PROD} &= \sum_i \tau_{Y,i}P_iY_i \\
T^{PRODUCTION} &= \sum_i \tau_{K,i}P_iY_i \\
T^{EXPORT} &= \tau_{EXPORT}C_l
\end{aligned}$$

A receita total é:

$$Y_G = T^{SI} + T^{LAB} + T^{VAT} + T^{CAP} + T^{FIRM} + T^{CF} + T^{PROD} + T^{PRODUCTION} + T^{EXPORT}$$

Definição das variáveis

Símbolo	Código	Significado
Y_G	<code>model.gov.Y_G</code>	Receita total realizada do governo
W^{emp}	<code>total_employed_wages</code>	Soma dos salários de trabalhadores com 0_h != 0
$\bar{P}_{HH}$	<code>model.agg.P_bar_HH</code>	Índice de preços usado sobre a massa salarial
C^H	<code>total_household_consumption</code>	Consumo realizado agregado dos domicílios
I^H	<code>total_household_investment</code>	Investimento residencial realizado agregado
Π_i	<code>model.firms.Pi_i</code>	Lucros realizados das empresas
Π_k	<code>model.bank.Pi_k</code>	Lucro realizado do banco
θ_{DIV}	<code>model.prop.theta_DIV</code>	Fração dos lucros positivos distribuída
τ_{SIF}	<code>model.prop.tau_SIF</code>	Contribuição social patronal
τ_{SIW}	<code>model.prop.tau_SIW</code>	Contribuição social dos trabalhadores
τ_{INC}	<code>model.prop.tau_INC</code>	Imposto sobre renda
τ_{FIRM}	<code>model.prop.tau_FIRM</code>	Imposto corporativo
τ_{VAT}	<code>model.prop.tau_VAT</code>	Imposto sobre consumo
τ_{CF}	<code>model.prop.tau_CF</code>	Imposto sobre formação de capital

Símbolo	Código	Significado
$\tau_{Y,i}$	<code>model.firms.tau_Y_i</code>	Imposto líquido sobre produtos
$\tau_{K,i}$	<code>model.firms.tau_K_i</code>	Imposto líquido sobre produção
τ_{EXPORT}	<code>model.prop.tau_EXPORT</code>	Imposto sobre exportações
C_l	<code>model.rotw.C_l</code>	Exportações realizadas

Casos nulos e sinais Trabalhadores com `0_h == 0` não entram na massa salarial empregada. Se nenhum trabalhador estiver empregado, sua soma salarial e as receitas associadas serão zero.

Lucros negativos ou NaN são zerados por `pos` antes da tributação. Por outro lado, `tau_Y_i` e `tau_K_i` são alíquotas líquidas: valores negativos podem representar subsídios e reduzir a receita total.

Não existem denominadores. A implementação também não inclui componentes separados para imposto sobre importações, juros pessoais, benefícios sociais ou volume de apostas.

Valores desejados, esperados e realizados A função não usa consumo desejado `C_d_h`, investimento desejado `I_d_h` nem lucros esperados `Pi_e_i` e `Pi_e_k`.

Ela utiliza valores **realizados**:

- emprego e salários correntes;
- consumo `C_h` e investimento `I_h` efetivamente realizados;
- lucros `Pi_i` e `Pi_k` realizados;
- produção, preços e exportações correntes.

As apostas não entram diretamente na receita, pois `Y_h` não é usado. Elas podem afetá-la indiretamente por meio dos orçamentos que determinaram consumo e investimento realizados.

Na célula de código, `expected_government_revenue` representa apenas o resultado esperado da verificação, não uma previsão do governo.

Campos atualizados e usos posteriores A função atualiza somente:

- `model.gov.Y_G`.

A próxima função, `Bit.set_gov_loans!(model)`, subtrairá essa receita das despesas e transferências para calcular o déficit e atualizar a dívida pública `model.gov.L_G`.

Não há sorteio dentro desta função, mas suas bases tributárias podem depender dos matchings aleatórios anteriores. Com os mesmos valores de entrada, reexecutar a célula apenas sobrescreve `Y_G` com o mesmo resultado.

```
[43]: # Save the field that will be updated and the debt that must remain unchanged
government_revenue_before = model.gov.Y_G
government_debt_before = model.gov.L_G

# Compute the realized tax bases
employed_mask = model.w_act.0_h .!= 0
```



```

total_employed_wages =
    sum(model.w_act.w_h[employed_mask])

total_household_consumption =
    sum(model.w_act.C_h) +
    sum(model.w_inact.C_h) +
    sum(model.firms.C_h) +
    model.bank.C_h

total_household_investment =
    sum(model.w_act.I_h) +
    sum(model.w_inact.I_h) +
    sum(model.firms.I_h) +
    model.bank.I_h

firm_profits = copy(model.firms.Pi_i)
bank_profit = model.bank.Pi_k

# Reproduce the behavior of Bit.pos
positive_firm_profits = [
    isnan(profit) || profit < zero(profit) ? zero(profit) : profit
    for profit in firm_profits
]
positive_bank_profit =
    isnan(bank_profit) || bank_profit < zero(bank_profit) ?
    zero(bank_profit) : bank_profit

positive_profit_base =
    sum(positive_firm_profits) +
    positive_bank_profit

# Reproduce every revenue component
social_security_revenue =
    (model.prop.tau_SIF + model.prop.tau_SIW) *
    total_employed_wages *
    model.agg.P_bar_HH

labour_income_tax =
    model.prop.tau_INC *
    (1 - model.prop.tau_SIW) *
    model.agg.P_bar_HH *
    total_employed_wages

value_added_tax =
    model.prop.tau_VAT *
    total_household_consumption

```

```

capital_income_tax =
    model.prop.tau_INC *
    (1 - model.prop.tau_FIRM) *
    model.prop.theta_DIV *
    positive_profit_base

corporate_income_tax =
    model.prop.tau_FIRM *
    positive_profit_base

capital_formation_tax =
    model.prop.tau_CF *
    total_household_investment

product_tax_revenue =
    sum(model.firms.tau_Y_i .* model.firms.P_i .* model.firms.Y_i)

production_tax_revenue =
    sum(model.firms.tau_K_i .* model.firms.P_i .* model.firms.Y_i)

export_tax_revenue =
    model.prop.tau_EXPORT *
    model.rotw.C_l

expected_government_revenue =
    social_security_revenue +
    labour_income_tax +
    value_added_tax +
    capital_income_tax +
    corporate_income_tax +
    capital_formation_tax +
    product_tax_revenue +
    production_tax_revenue +
    export_tax_revenue

# Execute the model function exactly once
Bit.set_gov_revenues!(model)

@assert isapprox(
    model.gov.Y_G,
    expected_government_revenue;
    nans = true,
)

@assert isapprox(
    model.gov.L_G,
    government_debt_before;
    nans = true,

```

```

)

revenue_summary = DataFrame(
    source = [
        "Previous stored revenue",
        "Labour and social security",
        "Consumption and capital formation",
        "Capital and corporate income",
        "Product and production taxes",
        "Export tax",
        "Computed total revenue",
        "Stored total revenue",
    ],
    amount = [
        government_revenue_before,
        social_security_revenue + labour_income_tax,
        value_added_tax + capital_formation_tax,
        capital_income_tax + corporate_income_tax,
        product_tax_revenue + production_tax_revenue,
        export_tax_revenue,
        expected_government_revenue,
        model.gov.Y_G,
    ],
)

revenue_summary

```

[43]:

	source	amount
	String	Float64
1	Previous stored revenue	0.0
2	Labour and social security	15873.0
3	Consumption and capital formation	0.0
4	Capital and corporate income	4288.54
5	Product and production taxes	2636.27
6	Export tax	0.0
7	Computed total revenue	22797.8
8	Stored total revenue	22797.8

2.44 Atualização da dívida pública

Objetivo econômico `Bit.set_gov_loans!(model)` calcula o déficit ou superávit realizado do governo e o incorpora ao estoque da dívida pública.

Um resultado positivo representa déficit e aumenta `L_G`; um resultado negativo representa superávit e reduz a dívida.

Equações A soma dos salários de referência dos trabalhadores desempregados é:

$$W^{unemp} = \sum_{h:O_h=0} w_h$$

Os benefícios pagos aos inativos são:

$$B^{inact} = H_{inact} sb_{inact} \bar{P}_{HH}$$

Os benefícios de desemprego são:

$$B^{unemp} = \theta_{UB} W^{unemp} \bar{P}_{HH}$$

O benefício geral pago a todos os domicílios é:

$$B^{other} = H sb_{other} \bar{P}_{HH}$$

Assim, o total de benefícios sociais é:

$$B = B^{inact} + B^{unemp} + B^{other}$$

O resultado fiscal usado pela implementação é:

$$\Pi_G = B + C_j + r_G L_{G,old} - Y_G$$

O estoque atualizado da dívida é:

$$L_{G,new} = L_{G,old} + \Pi_G$$

Definição das variáveis

Símbolo	Código	Significado
$L_{G,old}$	<code>government_debt_before</code>	Dívida pública antes da atualização
$L_{G,new}$	<code>model.gov.L_G</code>	Dívida pública atualizada
Π_G	<code>expected_government_balance</code>	Déficit positivo ou superávit negativo
W^{unemp}	<code>total_unemployed_wages</code>	Soma dos salários de referência com <code>0_h == 0</code>
H_{inact}	<code>model.prop.H_inact</code>	Número de pessoas inativas
H	<code>model.prop.H</code>	Número total de domicílios
sb_{inact}	<code>model.gov.sb_inact</code>	Benefício específico dos inativos
sb_{other}	<code>model.gov.sb_other</code>	Benefício geral por domicílio

Símbolo	Código	Significado
θ_{UB}	<code>model.prop.theta_UB</code>	Taxa de reposição do benefício de desemprego
$\bar{P}_{HH}$	<code>model.agg.P_bar_HH</code>	Índice de preços dos domicílios
C_j	<code>model.gov.C_j</code>	Consumo público realizado
r_G	<code>model.cb.r_G</code>	Taxa de juros da dívida pública
Y_G	<code>model.gov.Y_G</code>	Receita governamental realizada

Casos nulos e sinais Se não houver trabalhadores com `0_h == 0`, então $W^{unemp} = 0$ e o benefício de desemprego será zero.

A implementação não possui denominadores nem impõe piso zero à dívida ou ao resultado fiscal. Portanto:

- $\Pi_G > 0$ aumenta a dívida;
- $\Pi_G < 0$ reduz a dívida;
- `L_G` pode tornar-se negativo se o superávit superar o estoque anterior;
- entradas negativas ou NaN são utilizadas ou propagadas sem correção.

Valores desejados, esperados e realizados O consumo público **desejado** `model.gov.C_d_j` foi definido anteriormente a partir do processo de despesas do governo. Essa etapa contém um choque aleatório e utiliza inflação esperada.

A função atual não usa `C_d_j`. Ela utiliza `model.gov.C_j`, o consumo público **realizado** no mercado. Também usa:

- a receita realizada `model.gov.Y_G`;
- o estado realizado de emprego;
- os níveis correntes dos benefícios;
- o estoque corrente da dívida e seus juros.

Não existe uma dívida desejada ou esperada calculada por esta função. Na célula de código, `expected_government_debt` é somente o resultado esperado da verificação.

Campos atualizados e usos posteriores A função atualiza somente:

- `model.gov.L_G`.

Ela não altera `model.gov.Y_G`. A nova dívida entra na identidade patrimonial do banco central e, no período seguinte, determina:

- os juros pagos pelo governo;
- a receita de juros e o patrimônio do banco central.

Não há sorteio dentro desta função. Entretanto, `C_j` e a situação de emprego podem depender dos sorteios e matchings anteriores.

Atenção: a função não é idempotente. Reexecutá-la tratará a dívida já atualizada como novo saldo inicial e aplicará novamente o resultado fiscal, inclusive recalculando os juros sobre essa dívida.

```

[44]: # Save the field that will be updated and the revenue that must remain unchanged
government_debt_before = model.gov.L_G
government_revenue = model.gov.Y_G
realized_government_consumption = model.gov.C_j

# Reproduce the social-benefit calculation
unemployed_mask = model.w_act.0_h .== 0
total_unemployed_wages =
    sum(model.w_act.w_h[unemployed_mask])

inactive_benefits =
    model.prop.H_inact *
    model.gov.sb_inact *
    model.agg.P_bar_HH

unemployment_benefits =
    model.prop.theta_UB *
    total_unemployed_wages *
    model.agg.P_bar_HH

general_benefits =
    model.prop.H *
    model.gov.sb_other *
    model.agg.P_bar_HH

total_social_benefits =
    inactive_benefits +
    unemployment_benefits +
    general_benefits

government_interest_payments =
    model.cb.r_G *
    government_debt_before

# Positive values are deficits; negative values are surpluses
expected_government_balance =
    total_social_benefits +
    realized_government_consumption +
    government_interest_payments -
    government_revenue

expected_government_debt =
    government_debt_before +
    expected_government_balance

# Execute the model function exactly once
Bit.set_gov_loans!(model)

```

```

@assert isapprox(
    model.gov.L_G,
    expected_government_debt;
    nans = true,
)
@assert isapprox(
    model.gov.Y_G,
    government_revenue;
    nans = true,
)

debt_summary = DataFrame(
    component = [
        "Opening government debt",
        "Inactive-person benefits",
        "Unemployment benefits",
        "General household benefits",
        "Realized government consumption",
        "Government interest payments",
        "Government revenue",
        "Computed deficit (+) or surplus (-)",
        "Closing government debt",
    ],
    value = [
        government_debt_before,
        inactive_benefits,
        unemployment_benefits,
        general_benefits,
        realized_government_consumption,
        government_interest_payments,
        government_revenue,
        expected_government_balance,
        model.gov.L_G,
    ],
)

debt_summary

```

[44]:

	component	value
	String	Float64
1	Opening government debt	2.32611e5
2	Inactive-person benefits	9244.87
3	Unemployment benefits	1024.5
4	General household benefits	5237.61
5	Realized government consumption	0.0
6	Government interest payments	2089.1
7	Government revenue	22797.8
8	Computed deficit (+) or surplus (-)	-5201.72
9	Closing government debt	2.27409e5

2.45 Atualização dos depósitos operacionais das empresas

Objetivo econômico `Bit.set_firms_deposits!(model)` atualiza a posição de caixa de cada empresa somando os fluxos monetários realizados: vendas, custos, impostos, dividendos, juros, investimento, crédito novo e amortização.

`model.firms.D_i` pertence à operação da empresa. Ele é diferente de `model.firms.D_h`, que representa o depósito pessoal de seu proprietário.

Equações A função auxiliar `pos` usada pela implementação é:

$$pos(x) = \begin{cases} 0, & x < 0 \text{ ou } x = \text{NaN} \\ x, & \text{caso contrário} \end{cases}$$

Os componentes do fluxo de caixa da empresa i são:

$$\begin{aligned}
F_i^{sales} &= P_i Q_i \\
F_i^{labour} &= -(1 + \tau_{SIF}) w_i N_i \bar{P}_{HH} \\
F_i^{material} &= -DM_i \bar{P}_i \\
F_i^{product} &= -\tau_{Y,i} P_i Y_i \\
F_i^{production} &= -\tau_{K,i} P_i Y_i \\
F_i^{corporate} &= -\tau_{FIRM} pos(\Pi_i) \\
F_i^{dividend} &= -\theta_{DIV} (1 - \tau_{FIRM}) pos(\Pi_i) \\
F_i^{interest} &= -r [L_i + pos(-D_{i,old})] + \bar{r} pos(D_{i,old}) \\
F_i^{investment} &= -P_{CF,i} I_i \\
F_i^{credit} &= DL_i \\
F_i^{repayment} &= -\theta L_i
\end{aligned}$$

A variação dos depósitos é:

$$\Delta D_i = F_i^{sales} + F_i^{labour} + F_i^{material} + F_i^{product} + F_i^{production} + F_i^{corporate} + F_i^{dividend} + F_i^{interest} + F_i^{investment} + F_i^{credit} + F_i^{repayment}$$

O novo saldo operacional é:

$$D_{i,new} = D_{i,old} + \Delta D_i$$

Definição das variáveis

Símbolo	Código	Significado
$D_{i,old}$	<code>firm_deposits_before</code>	Posição operacional anterior
$D_{i,new}$	<code>model.firms.D_i</code>	Nova posição operacional
D_i^h	<code>model.firms.D_h</code>	Depósito pessoal do proprietário, não alterado aqui
$P_i Q_i$	<code>sales</code>	Receita monetária das vendas realizadas
$w_i N_i$	<code>model.firms.w_i .*</code> <code>model.firms.N_i</code>	Folha salarial da empresa
DM_i	<code>model.firms.DM_i</code>	Variação realizada dos materiais
Π_i	<code>model.firms.Pi_i</code>	Lucro realizado
L_i	<code>model.firms.L_i</code>	Empréstimo anterior
I_i	<code>model.firms.I_i</code>	Investimento realizado
$P_{CF,i}$	<code>model.firms.P_CF_i</code>	Índice de preços do investimento da empresa
DL_i	<code>model.firms.DL_i</code>	Crédito novo efetivamente obtido
θ	<code>model.prop.theta</code>	Parcela amortizada do empréstimo
r	<code>model.bank.r</code>	Taxa sobre empréstimos e saldos negativos
$\bar{r}$	<code>model.cb.r_bar</code>	Taxa recebida sobre depósitos positivos
ΔD_i	<code>expected_deposit_change</code>	Fluxo de caixa líquido do período

Diferença entre lucro e fluxo de caixa O lucro Π_i foi calculado anteriormente segundo critérios contábeis. Esta função calcula caixa:

- as vendas usam somente $P_i Q_i$, sem a variação de estoques finais DS_i ;
- compras de materiais entram por $DM_i \bar{P}_i$;
- investimentos entram como saída integral de caixa;
- crédito novo entra no caixa;
- amortizações saem do caixa.

Um D_i negativo representa uma posição devedora adicional. Nesse caso, a empresa paga juros tanto sobre L_i quanto sobre $\text{pos}(-D_i)$.

Lucros negativos ou NaN não geram imposto corporativo nem dividendos porque `pos` os converte em zero. Alíquotas líquidas negativas sobre produtos ou produção podem transformar esses componentes em entradas de caixa.

Não há denominadores nessa função.

Valores desejados, esperados e realizados Os planos **desejados** da empresa incluem campos como $I_{d,i}$, $DM_{d,i}$ e $DL_{d,i}$. Os lucros esperados estão em $Pi_{e,i}$.

A função atual não usa esses valores. Ela utiliza resultados **realizados**, incluindo:

- vendas Q_i ;
- materiais DM_i ;
- investimento I_i ;
- crédito obtido DL_i ;
- lucro Pi_i ;
- produção, custos e preços correntes.

Na célula de código, `expected_firm_deposits` representa somente o resultado esperado da verificação, não depósitos esperados pelas empresas.

Campos atualizados e usos posteriores A função atualiza somente:

- `model.firms.D_i`.

Em seguida:

- `Bit.set_firms_loans!(model)` atualizará L_i , incorporando crédito novo e amortização;
- `Bit.set_firms_equity!(model)` usará os novos depósitos e empréstimos;
- `Bit.set_bank_deposits!(model)` incorporará D_i ao balanço do banco.

`model.firms.D_h` e `model.firms.L_i` permanecem inalterados nesta função.

Não há sorteio direto, mas vendas, materiais, investimentos e crédito podem depender dos matchings aleatórios anteriores.

Atenção: a função não é idempotente. Reexecutá-la repetirá os fluxos do período e calculará os juros a partir dos depósitos já atualizados.

```
[45]: # Save the updated field and related fields that must remain unchanged
firm_deposits_before = copy(model.firms.D_i)
owner_deposits_before = copy(model.firms.D_h)
firm_loans_before = copy(model.firms.L_i)

# Reproduce the positive-part helper used by the implementation
positive_profits = Bit.pos.(model.firms.Pi_i)
positive_opening_deposits = Bit.pos.(firm_deposits_before)
opening_overdrafts = Bit.pos.(-firm_deposits_before)

# Reproduce every operating cash-flow component
sales =
    model.firms.P_i .* model.firms.Q_i

labour_cost =
    -(1 + model.prop.tau_SIF) .*
    model.firms.w_i .*
```

```

model.firms.N_i .*
model.agg.P_bar_HH

material_cost =
    -model.firms.DM_i .* model.firms.P_bar_i

product_taxes =
    -model.firms.tau_Y_i .* model.firms.P_i .* model.firms.Y_i

production_taxes =
    -model.firms.tau_K_i .* model.firms.P_i .* model.firms.Y_i

corporate_tax =
    -model.prop.tau_FIRM .* positive_profits

dividend_payments =
    -model.prop.theta_DIV .*
    (1 - model.prop.tau_FIRM) .*
    positive_profits

interest_payments =
    -model.bank.r .*
    (firm_loans_before .+ opening_overdrafts)

interest_received =
    model.cb.r_bar .* positive_opening_deposits

investment_cost =
    -model.firms.P_CF_i .* model.firms.I_i

new_credit = copy(model.firms.DL_i)

debt_repayment =
    -model.prop.theta .* firm_loans_before

expected_deposit_change =
    sales +
    labour_cost +
    material_cost +
    product_taxes +
    production_taxes +
    corporate_tax +
    dividend_payments +
    interest_payments +
    interest_received +
    investment_cost +
    new_credit +

```

```

    debt_repayment

expected_firm_deposits =
    firm_deposits_before .+ expected_deposit_change

# Execute the model function exactly once
Bit.set_firms_deposits!(model)

@assert all(isapprox.(
    model.firms.D_i,
    expected_firm_deposits;
    nans = true,
))
@assert all(isapprox.(
    model.firms.D_h,
    owner_deposits_before;
    nans = true,
))
@assert all(isapprox.(
    model.firms.L_i,
    firm_loans_before;
    nans = true,
))

deposit_summary = DataFrame(
    component = [
        "Opening operating position",
        "Sales",
        "Labour and material cash flow",
        "Taxes and dividends",
        "Net interest cash flow",
        "Investment cash flow",
        "New credit",
        "Debt repayment",
        "Net deposit change",
        "Closing operating position",
    ],
    value = [
        sum(firm_deposits_before),
        sum(sales),
        sum(labour_cost + material_cost),
        sum(
            product_taxes +
            production_taxes +
            corporate_tax +
            dividend_payments
        ),
    ],

```

```

        sum(interest_payments + interest_received),
        sum(investment_cost),
        sum(new_credit),
        sum(debt_repayment),
        sum(expected_deposit_change),
        sum(model.firms.D_i),
    ],
)

deposit_summary

```

[45]:

	component	value
	String	Float64
1	Opening operating position	54049.0
2	Sales	0.0
3	Labour and material cash flow	-0.0
4	Taxes and dividends	-12284.1
5	Net interest cash flow	-6630.04
6	Investment cash flow	-0.0
7	New credit	0.0
8	Debt repayment	-11846.0
9	Net deposit change	-30760.1
10	Closing operating position	23288.9

2.46 Atualização dos empréstimos das empresas

Objetivo econômico `Bit.set_firms_loans!(model)` atualiza o estoque de empréstimos de cada empresa após a amortização programada da dívida anterior e a incorporação do novo crédito efetivamente concedido.

Equações A amortização do estoque anterior é:

$$A_i = \theta L_{i,old}$$

A parcela remanescente da dívida é:

$$L_i^{remaining} = (1 - \theta)L_{i,old}$$

O novo estoque de empréstimos é:

$$L_{i,new} = (1 - \theta)L_{i,old} + DL_i$$

Equivalentemente:

$$L_{i,new} = L_{i,old} - A_i + DL_i$$

Definição das variáveis

Símbolo	Código	Significado
$L_{i,old}$	<code>firm_loans_before</code>	Estoque de empréstimos antes da atualização
$L_{i,new}$	<code>model.firms.L_i</code>	Estoque atualizado de empréstimos
A_i	<code>scheduled_repayment</code>	Amortização programada
θ	<code>model.prop.theta</code>	Taxa de amortização da dívida
DL_i^d	<code>model.firms.DL_d_i</code>	Crédito novo desejado
DL_i	<code>model.firms.DL_i</code>	Crédito novo efetivamente concedido
L_i^e	<code>model.firms.L_e_i</code>	Estoque esperado após amortização, antes do crédito novo
D_i	<code>model.firms.D_i</code>	Depósito operacional, não alterado nesta função

Casos nulos

- Se $\theta = 0$, nenhuma parte da dívida anterior é amortizada.
- Se $\theta = 1$, toda a dívida anterior é eliminada antes da adição de DL_i .
- Se $DL_i = 0$, o novo saldo contém somente a parcela não amortizada.
- Se $L_{i,old} = 0$, o novo saldo é igual ao crédito recebido.

Não há denominadores nem aplicação de piso zero. Valores negativos ou NaN são usados ou propagados diretamente. Vetores vazios também são tratados normalmente.

Valores desejados, esperados e realizados Anteriormente, as empresas calcularam:

- L_e_i , o estoque **esperado** após a amortização;
- DL_d_i , a quantidade **desejada** de crédito novo.

O mercado de crédito transformou essa demanda em DL_i , o crédito **realizado**. A concessão considera as restrições bancárias e embaralha a ordem das empresas candidatas, de modo que empresas com demanda positiva podem receber menos que DL_d_i ou não receber crédito.

A função atual utiliza somente o estoque anterior L_i , a taxa `theta` e o crédito realizado DL_i . Ela não usa diretamente L_e_i nem DL_d_i .

Na célula de código, `expected_firm_loans` representa o resultado esperado da verificação, não a expectativa econômica armazenada em L_e_i .

Campos atualizados e usos posteriores A função atualiza somente:

- `model.firms.L_i`.

Ela não altera `model.firms.D_i`. A função anterior já registrou no caixa das empresas a entrada de DL_i e a saída de `theta * L_i`.

Em seguida:

- `Bit.set_firms_equity!(model)` usará o novo estoque de empréstimos;
- `Bit.set_bank_deposits!(model)` subtrairá esses empréstimos no balanço bancário;
- nos períodos seguintes, `L_i` influenciará juros, lucros e decisões de crédito.

Não há sorteio dentro desta função. A aleatoriedade vem indiretamente do matching de crédito que determinou `DL_i`.

Atenção: a função não é idempotente. Reexecutá-la amortizará novamente o saldo já atualizado e adicionará outra vez o mesmo `DL_i`.

[46]: *# Save the updated field and the deposits that must remain unchanged*

```
firm_loans_before = copy(model.firms.L_i)
firm_deposits_before = copy(model.firms.D_i)

desired_new_credit = copy(model.firms.DL_d_i)
realized_new_credit = copy(model.firms.DL_i)
repayment_rate = model.prop.theta

# Reproduce firms_loans
scheduled_repayment =
    repayment_rate .* firm_loans_before

remaining_old_loans =
    (1 - repayment_rate) .* firm_loans_before

expected_firm_loans =
    remaining_old_loans .+ realized_new_credit

# Execute the model function exactly once
Bit.set_firms_loans!(model)

@assert all(isapprox.(
    model.firms.L_i,
    expected_firm_loans;
    nans = true,
))

@assert all(isapprox.(
    model.firms.D_i,
    firm_deposits_before;
    nans = true,
))

loan_summary = DataFrame(
    component = [
        "Opening loan stock",
        "Scheduled repayment",
        "Remaining old loans",
        "Desired new credit",
```

```

        "Realized new credit",
        "Closing loan stock",
    ],
    value = [
        sum(firm_loans_before),
        sum(scheduled_repayment),
        sum(remaining_old_loans),
        sum(desired_new_credit),
        sum(realized_new_credit),
        sum(model.firms.L_i),
    ],
)

loan_summary

```

[46]:

	component	value
	String	Float64
1	Opening loan stock	236919.0
2	Scheduled repayment	11846.0
3	Remaining old loans	225073.0
4	Desired new credit	0.0
5	Realized new credit	0.0
6	Closing loan stock	225073.0

2.47 Atualização do patrimônio líquido das empresas

Objetivo econômico `Bit.set_firms_equity!(model)` calcula o patrimônio líquido de cada empresa avaliando seus ativos aos preços correntes e subtraindo o estoque atualizado de empréstimos.

A função calcula uma posição patrimonial, não o lucro do período.

Equações Para uma empresa cujo produto principal pertence ao setor G_i , o valor unitário de seus materiais é:

$$P_i^M = \sum_s a_{s,G_i} \bar{P}_s$$

O valor do estoque de materiais é:

$$V_i^M = M_i P_i^M$$

O valor dos estoques de produtos finais é:

$$V_i^S = P_i S_i$$

O valor do estoque de capital é:

$$V_i^K = \bar{P}_{CF} K_i$$

O patrimônio líquido é:

$$E_i = D_i + V_i^M + V_i^S + V_i^K - L_i$$

Definição das variáveis

Símbolo	Código	Significado
E_i	<code>model.firms.E_i</code>	Patrimônio líquido atualizado
D_i	<code>model.firms.D_i</code>	Depósito ou posição operacional da empresa
M_i	<code>model.firms.M_i</code>	Estoque físico de materiais intermediários
P_i^M	<code>material_unit_values</code>	Valor unitário da cesta de materiais do setor
a_{s,G_i}	<code>model.prop.a_sg[:, model.firms.G_i]</code>	Coefficientes de insumos do setor da empresa
$\bar{P}_s$	<code>model.agg.P_bar_g</code>	Índices de preços dos produtos
S_i	<code>model.firms.S_i</code>	Estoque de produtos finais
P_i	<code>model.firms.P_i</code>	Preço da empresa
K_i	<code>model.firms.K_i</code>	Estoque físico de capital
$\bar{P}_{CF}$	<code>model.agg.P_bar_CF</code>	Índice geral de preços dos bens de capital
L_i	<code>model.firms.L_i</code>	Estoque atualizado de empréstimos
D_i^h	<code>model.firms.D_h</code>	Depósito pessoal do proprietário, não incluído no patrimônio da empresa

Interpretação patrimonial Os ativos contabilizados são:

- depósitos operacionais;
- materiais intermediários;
- estoques de produtos finais;
- capital produtivo.

Os empréstimos são passivos e, por isso, são subtraídos.

Um `D_i` negativo reduz diretamente o patrimônio. A implementação não aplica `pos`, não impõe um piso zero e não possui denominadores. Valores negativos ou `NaN` nos componentes são usados ou propagados diretamente.

Valores desejados, esperados e realizados Anteriormente, as empresas calcularam capital esperado `K_e_i`, empréstimos esperados `L_e_i` e demandas desejadas de materiais, investimento e crédito.

Esses valores não são usados nesta função. O patrimônio é calculado com os estoques **realizados e atualizados**:

- D_i , após os fluxos operacionais;
- L_i , após amortização e crédito novo;
- M_i , S_i e K_i , após as transações realizadas;
- preços e índices correntes.

Não existe um patrimônio desejado ou esperado calculado aqui. Na célula de código, `expected_firm_equity` é apenas o resultado esperado da verificação.

Campos atualizados e usos posteriores A função atualiza somente:

- `model.firms.E_i`.

Ela não altera depósitos, empréstimos ou estoques. No início do próximo passo, `Bit.finance_insolvent_firms!(model)` verificará conjuntamente:

$$D_i < 0 \quad \text{e} \quad E_i < 0$$

Empresas que atendam às duas condições serão refinanciadas com recursos do banco.

Não há sorteio dentro desta função. Porém, estoques, vendas, investimentos e crédito podem depender dos matchings aleatórios anteriores.

Como E_i anterior não aparece na fórmula, reexecutar a célula com os mesmos ativos, passivos e preços apenas sobrescreve o campo com o mesmo resultado.

```
[47]: # Save the field that will be updated and its main inputs
firm_equity_before = copy(model.firms.E_i)
firm_deposits = copy(model.firms.D_i)
firm_loans = copy(model.firms.L_i)

# Value the sector-specific basket of intermediate materials
material_unit_values = vec(sum(
    model.prop.a_sg[:, model.firms.G_i] .*
    model.agg.P_bar_g;
    dims = 1,
))

material_values =
    model.firms.M_i .* material_unit_values

inventory_values =
    model.firms.P_i .* model.firms.S_i

capital_values =
    model.agg.P_bar_CF .* model.firms.K_i

expected_firm_equity =
```

```

    firm_deposits .+
    material_values .+
    inventory_values .+
    capital_values .-
    firm_loans

# Execute the model function exactly once
Bit.set_firms_equity!(model)

@assert all(isapprox.(
    model.firms.E_i,
    expected_firm_equity;
    nans = true,
))
@assert all(isapprox.(
    model.firms.D_i,
    firm_deposits;
    nans = true,
))
@assert all(isapprox.(
    model.firms.L_i,
    firm_loans;
    nans = true,
))

equity_summary = DataFrame(
    component = [
        "Previous stored equity",
        "Operating deposits",
        "Intermediate materials",
        "Finished-goods inventories",
        "Capital stock",
        "Outstanding loans",
        "Computed equity",
        "Stored equity",
    ],
    value = [
        sum(firm_equity_before),
        sum(firm_deposits),
        sum(material_values),
        sum(inventory_values),
        sum(capital_values),
        sum(firm_loans),
        sum(expected_firm_equity),
        sum(model.firms.E_i),
    ],
)

```

equity_summary

[47]:

	component	value
	String	Float64
1	Previous stored equity	0.0
2	Operating deposits	23288.9
3	Intermediate materials	80090.1
4	Finished-goods inventories	0.0
5	Capital stock	750826.0
6	Outstanding loans	225073.0
7	Computed equity	629132.0
8	Stored equity	629132.0

2.48 Atualização da posição financeira do resto do mundo

Objetivo econômico `Bit.set_rotw_deposits!(model)` acumula o saldo financeiro das transações comerciais entre a economia doméstica e o resto do mundo.

O saldo aumenta com os pagamentos domésticos pelas importações e diminui com os pagamentos dos compradores externos pelas exportações.

Equações O valor das importações realizadas é:

$$M = \sum_m P_m Q_m$$

O imposto pago pelos compradores externos é:

$$T^{export} = \tau_{EXPORT} C_l$$

O pagamento total pelas exportações é:

$$X^{gross} = (1 + \tau_{EXPORT}) C_l$$

A variação da posição externa é:

$$\Delta D_{RoW} = M - X^{gross}$$

O novo saldo é:

$$D_{RoW,new} = D_{RoW,old} + \Delta D_{RoW}$$

Definição das variáveis

Símbolo	Código	Significado
$D_{RoW,old}$	<code>external_position_before</code>	Posição financeira externa anterior
$D_{RoW,new}$	<code>model.rotw.D_RoW</code>	Posição financeira externa atualizada
ΔD_{RoW}	<code>expected_external_position_change</code>	Variação causada pelo comércio realizado
P_m	<code>model.rotw.P_m</code>	Preços dos produtos importados
Q_m	<code>model.rotw.Q_m</code>	Quantidades importadas efetivamente vendidas
M	<code>import_payments</code>	Pagamentos pelas importações
C_l	<code>model.rotw.C_l</code>	Valor realizado das exportações antes do imposto
τ_{EXPORT}	<code>model.prop.tau_EXPORT</code>	Alíquota sobre exportações
T^{export}	<code>export_tax_payment</code>	Imposto associado às exportações
X^{gross}	<code>gross_export_payments</code>	Pagamento externo total pelas exportações
D_k	<code>model.bank.D_k</code>	Posição do banco comercial, não alterada aqui

Interpretação dos sinais Segundo a convenção da implementação:

- importações aumentam D_{RoW} ;
- exportações, incluindo o imposto, reduzem D_{RoW} ;
- se importações e exportações brutas forem iguais, o saldo não muda;
- exportações brutas superiores às importações produzem uma variação negativa;
- importações superiores produzem uma variação positiva.

A função não aplica `max`, não possui denominadores e não restringe o sinal do saldo. Valores negativos ou NaN em preços, quantidades, exportações ou impostos são usados ou propagados diretamente. Um vetor vazio de importações produz soma zero.

Valores desejados, esperados e realizados Anteriormente, `Bit.set_rotw_import_export!(model)` definiu:

- Y_m , a oferta planejada de importações;
- $C_{d,l}$, a demanda desejada por exportações;
- P_m , com base nos índices de preços e na inflação esperada.

Após o matching de bens:

- Q_m contém as importações **realizadas**;
- C_l contém as exportações **realizadas**.

A função atual não usa diretamente Y_m nem $C_{d,l}$. Ela registra apenas os fluxos realizados. Na célula de código, `expected_external_position` representa o resultado esperado da verificação, não uma expectativa econômica do resto do mundo.

Campos atualizados e usos posteriores A função atualiza somente:

- `model.rotw.D_RoW`.

Ela não altera `model.bank.D_k`. Depois que `Bit.set_bank_deposits!(model)` atualizar a posição bancária, `D_RoW` participará da identidade patrimonial:

$$E_{CB} + D_{RoW} - L_G + D_k = 0$$

O saldo atualizado também será a posição inicial para a próxima variação comercial.

Não há sorteio dentro desta função. Contudo, as demandas de comércio e o matching que determinam `Q_m` e `C_l` dependem de choques e de aleatoriedade anteriores.

Atenção: a função não é idempotente. Reexecutá-la adicionará novamente o mesmo saldo comercial à posição externa.

```
[48]: # Save the updated field and the bank position that must remain unchanged
external_position_before = model.rotw.D_RoW
commercial_bank_position_before = model.bank.D_k

import_prices = copy(model.rotw.P_m)
realized_import_quantities = copy(model.rotw.Q_m)
realized_exports = model.rotw.C_l
export_tax_rate = model.prop.tau_EXPORT

# Reproduce rotw_deposits
import_payments =
    sum(import_prices .* realized_import_quantities)

export_value =
    realized_exports

export_tax_payment =
    export_tax_rate * realized_exports

gross_export_payments =
    (1 + export_tax_rate) * realized_exports

expected_external_position_change =
    import_payments - gross_export_payments

expected_external_position =
    external_position_before +
    expected_external_position_change

# Execute the model function exactly once
Bit.set_rotw_deposits!(model)
```

```

@assert isapprox(
    model.rotw.D_RoW,
    expected_external_position;
    nans = true,
)
@assert isapprox(
    model.bank.D_k,
    commercial_bank_position_before;
    nans = true,
)

external_summary = DataFrame(
    component = [
        "Opening external position",
        "Import payments",
        "Exports before tax",
        "Export tax payment",
        "Gross export payments",
        "Net position change",
        "Closing external position",
    ],
    value = [
        external_position_before,
        import_payments,
        export_value,
        export_tax_payment,
        gross_export_payments,
        expected_external_position_change,
        model.rotw.D_RoW,
    ],
)

external_summary

```

[48]:

	component	value
	String	Float64
1	Opening external position	0.0
2	Import payments	0.0
3	Exports before tax	0.0
4	Export tax payment	0.0
5	Gross export payments	0.0
6	Net position change	0.0
7	Closing external position	0.0

2.49 Atualização da posição líquida do banco comercial

Objetivo econômico `Bit.set_bank_deposits!(model)` calcula a posição residual do banco comercial a partir de seu balanço: depósitos operacionais das empresas, posições financeiras dos

domicílios, patrimônio bancário e empréstimos concedidos às empresas.

Apesar do nome da função, `model.bank.D_k` não representa apenas depósitos de clientes. Ele é o item residual de crédito ou débito do banco.

Equações A posição financeira agregada dos domicílios é:

$$D^H = \sum_h D_h^{act} + \sum_h D_h^{inact} + \sum_i D_i^{owner} + D_k^{owner}$$

A posição do banco é:

$$D_k = \sum_i D_i + D^H + E_k - \sum_i L_i$$

A mesma equação pode ser escrita como a identidade do balanço bancário:

$$\sum_i D_i + D^H + E_k - \sum_i L_i - D_k = 0$$

Definição das variáveis

Símbolo	Código	Significado
D_k	<code>model.bank.D_k</code>	Posição líquida residual do banco comercial
D_i	<code>model.firms.D_i</code>	Posições operacionais das empresas
D_h^{act}	<code>model.w_act.D_h</code>	Posições dos trabalhadores ativos
D_h^{inact}	<code>model.w_inact.D_h</code>	Posições dos trabalhadores inativos
D_i^{owner}	<code>model.firms.D_h</code>	Posições pessoais dos proprietários das empresas
D_k^{owner}	<code>model.bank.D_h</code>	Posição pessoal do proprietário do banco
D^H	<code>total_household_positions</code>	Soma das posições financeiras dos domicílios
E_k	<code>model.bank.E_k</code>	Patrimônio líquido do banco
L_i	<code>model.firms.L_i</code>	Empréstimos concedidos às empresas
$D_{k,old}$	<code>bank_position_before</code>	Posição bancária armazenada antes da atualização

Tratamento dos sinais Todos os depósitos entram com seus sinais originais:

- posições positivas aumentam o lado dos depósitos;
- posições negativas representam dívidas ou descobertos e reduzem a soma;

- os empréstimos das empresas são subtraídos;
- D_k pode ser positivo ou negativo.

A função não aplica `max`, não possui denominadores e não impõe limites. Vetores vazios produzem soma zero, enquanto valores NaN são propagados.

Valores desejados, esperados e realizados A função usa os estoques **realizados e atualizados** pelas etapas anteriores:

- depósitos pessoais dos domicílios;
- depósitos operacionais das empresas;
- empréstimos após amortização e crédito realizado;
- patrimônio corrente do banco.

Ela não usa consumo desejado, lucros esperados, empréstimos esperados ou demanda desejada de crédito.

A posição anterior D_k também não entra na fórmula: ela é totalmente sobrescrita pelo novo valor contábil. Na célula de código, `expected_bank_position` é apenas o resultado esperado da verificação.

Campos atualizados e usos posteriores A função atualiza somente:

- `model.bank.D_k`.

Ela não altera o depósito pessoal `model.bank.D_h`.

A nova posição completa a identidade do balanço do banco e também participa da identidade patrimonial do banco central:

$$E_{CB} + D_{RoW} - L_G + D_k = 0$$

Nos períodos seguintes, D_k será usado no cálculo dos lucros do banco comercial e do patrimônio do banco central. Como o patrimônio do banco central deste período já foi calculado antes desta etapa, a nova posição bancária terá efeito nesse cálculo somente no próximo passo.

Não há sorteio dentro desta função. Os saldos utilizados podem depender dos mercados aleatórios anteriores.

Como D_k anterior não participa da fórmula, reexecutar somente esta célula com os mesmos componentes produz o mesmo resultado.

[49]: *# Save the updated field and every balance-sheet component*

```
bank_position_before = model.bank.D_k

firm_operating_positions = copy(model.firms.D_i)
active_worker_positions = copy(model.w_act.D_h)
inactive_worker_positions = copy(model.w_inact.D_h)
firm_owner_positions = copy(model.firms.D_h)
bank_owner_position = model.bank.D_h
```

```

bank_equity = model.bank.E_k
firm_loans = copy(model.firms.L_i)

# Reproduce bank_deposits
total_household_positions =
    sum(active_worker_positions) +
    sum(inactive_worker_positions) +
    sum(firm_owner_positions) +
    bank_owner_position

expected_bank_position =
    sum(firm_operating_positions) +
    total_household_positions +
    bank_equity -
    sum(firm_loans)

# Execute the model function exactly once
Bit.set_bank_deposits!(model)

@assert isapprox(
    model.bank.D_k,
    expected_bank_position;
    nans = true,
)
@assert isapprox(
    model.bank.D_h,
    bank_owner_position;
    nans = true,
)

bank_summary = DataFrame(
    component = [
        "Previous bank position",
        "Firm operating positions",
        "Active-worker positions",
        "Inactive-worker positions",
        "Firm-owner positions",
        "Bank-owner position",
        "Bank equity",
        "Outstanding firm loans",
        "Computed bank position",
        "Stored bank position",
    ],
    value = [
        bank_position_before,
        sum(firm_operating_positions),
        sum(active_worker_positions),

```

```

        sum(inactive_worker_positions),
        sum(firm_owner_positions),
        bank_owner_position,
        bank_equity,
        sum(firm_loans),
        expected_bank_position,
        model.bank.D_k,
    ],
)

bank_summary

```

[49]:

	component	value
	String	Float64
1	Previous bank position	126431.0
2	Firm operating positions	23288.9
3	Active-worker positions	1.30355e5
4	Inactive-worker positions	69446.8
5	Firm-owner positions	42970.9
6	Bank-owner position	21966.7
7	Bank equity	89460.0
8	Outstanding firm loans	225073.0
9	Computed bank position	1.52415e5
10	Stored bank position	1.52415e5

2.50 Registro da produção agregada no histórico do modelo

Objetivo econômico Apesar do nome `Bit.set_gross_domestic_product!(model)`, a implementação não calcula o PIB nominal ou real pelas identidades das contas nacionais.

Ela soma a produção física corrente de todas as empresas e registra o resultado no histórico `model.agg.Y`, utilizado para formar expectativas futuras de produto e crescimento.

Equações A produção agregada corrente é:

$$Y_t = \sum_i Y_{i,t}$$

A posição do histórico correspondente ao período atual é:

$$j = T' + t$$

A função primeiro acrescenta um zero ao final do vetor:

$$Y^{history} \leftarrow [Y^{history}; 0]$$

Depois, grava a produção corrente no índice calculado:

$$Y_{T'+t}^{history} = Y_t$$

Em uma execução sequencial normal:

$$\text{length}(Y_{before}^{history}) = T' + t - 1$$

$$\text{length}(Y_{after}^{history}) = T' + t$$

Definição das variáveis

Símbolo	Código	Significado
$Y_{i,t}$	<code>model.firms.Y_i</code>	Produção corrente de cada empresa
Y_t	<code>expected_aggregate_output</code>	Soma da produção das empresas
$Y^{history}$	<code>model.agg.Y</code>	Histórico usado nas expectativas de produto
T'	<code>model.prop.T_prime</code>	Número de observações históricas iniciais
t	<code>model.agg.t</code>	Contador atual da simulação
j	<code>history_index</code>	Índice em que a nova produção é gravada
Y_t^e	<code>model.agg.Y_e</code>	Produto esperado, calculado anteriormente
γ_t^e	<code>model.agg.gamma_e</code>	Crescimento esperado

Diferença em relação aos dados de PIB `model.agg.Y` armazena somente:

$$\sum_i Y_i$$

Já `model.data.nominal_gdp` e `model.data.real_gdp` são calculados separadamente por `Bit.collect_data!(model)`, usando preços, impostos, consumo intermediário e componentes de despesa.

Portanto, esta função não atualiza diretamente os campos de PIB nominal ou real em `model.data`.

Valores desejados, esperados e realizados As empresas definiram anteriormente vendas desejadas em `Q_s_i`, mas esse campo não é usado aqui. A função soma a produção corrente **realizada** `Y_i`.

O valor `model.agg.Y_e` é o produto **esperado** calculado no início do passo a partir do histórico disponível naquele momento. A nova observação ainda não participou dessa expectativa.

No próximo período, a nova entrada de `agg.Y` será incluída no cálculo de `Y_e` e `gamma_e`. Não há um PIB desejado calculado nesta função.

Na célula de código, `expected_aggregate_output` representa o resultado esperado da verificação, não a expectativa econômica armazenada em `model.agg.Y_e`.

Campos atualizados e usos posteriores A função altera somente:

- o conteúdo de `model.agg.Y`;
- o comprimento de `model.agg.Y`, acrescentando uma posição.

Ela não altera `model.agg.t`. A próxima função, `Bit.set_time!(model)`, incrementará o contador. No início do próximo passo, `Bit.set_growth_inflation_expectations!(model)` utilizará o histórico ampliado.

Não há sorteio dentro desta função, mas `Y_i` pode depender do matching aleatório de trabalho e das decisões anteriores.

A função não aplica limites e não possui denominadores. Produção agregada zero, negativa ou NaN é armazenada diretamente. Valores não positivos podem causar problemas posteriormente porque as expectativas aplicam `log` ao histórico.

Atenção: a função não é idempotente estruturalmente. Reexecutá-la acrescentará outro zero ao vetor, embora volte a escrever no mesmo índice `T_prime + t`, deixando uma posição excedente no histórico.

```
[50]: # Save the history and the simulation time before the update
output_history_before = copy(model.agg.Y)
history_length_before = length(output_history_before)
simulation_time_before = model.agg.t

firm_production = copy(model.firms.Y_i)

# Reproduce gross_domestic_product and its history update
expected_aggregate_output =
    sum(firm_production)

history_index =
    model.prop.T_prime + simulation_time_before

expected_output_history =
    copy(output_history_before)

push!(expected_output_history, 0.0)
expected_output_history[history_index] =
    expected_aggregate_output

# Execute the model function exactly once
Bit.set_gross_domestic_product!(model)
```

```

@assert length(model.agg.Y) ==
    length(expected_output_history)

@assert all(isapprox.(
    model.agg.Y,
    expected_output_history;
    nans = true,
))

@assert model.agg.t == simulation_time_before

output_summary = DataFrame(
    metric = [
        "History length before",
        "Simulation time",
        "History index written",
        "Number of firms",
        "Aggregate firm production",
        "Stored history value",
        "History length after",
    ],
    value = [
        history_length_before,
        simulation_time_before,
        history_index,
        length(firm_production),
        expected_aggregate_output,
        model.agg.Y[history_index],
        length(model.agg.Y),
    ],
)

output_summary

```

[50]:

	metric	value
	String	Float64
1	History length before	54.0
2	Simulation time	1.0
3	History index written	55.0
4	Number of firms	624.0
5	Aggregate firm production	1.34636e5
6	Stored history value	1.34636e5
7	History length after	55.0

2.51 Avanço do contador da simulação

Objetivo econômico `Bit.set_time!(model)` encerra o passo atual incrementando o contador temporal da simulação. Esta é a última função executada em `src/one_step.jl`.

A função não calcula nenhuma variável econômica; ela apenas prepara o modelo para o próximo período.

Equações A implementação completa é:

$$t_{new} = t_{old} + 1$$

Depois do incremento, o último valor de produção gravado na etapa anterior ocupa a posição:

$$T' + t_{old} = T' + t_{new} - 1$$

Essa é exatamente a última posição utilizada no próximo cálculo de expectativas.

Definição das variáveis

Símbolo	Código	Significado
t_{old}	<code>simulation_time_before</code>	Contador antes do incremento
t_{new}	<code>model.agg.t</code>	Contador depois do incremento
T'	<code>model.prop.T_prime</code>	Número de períodos históricos iniciais
Y	<code>model.agg.Y</code>	Histórico de produção, não alterado aqui
Y^e	<code>model.agg.Y_e</code>	Produto esperado do período concluído
γ^e	<code>model.agg.gamma_e</code>	Crescimento esperado do período concluído
π^e	<code>model.agg.pi_e</code>	Inflação esperada do período concluído

Valores desejados, esperados e realizados A função não possui valores desejados, esperados ou realizados próprios.

As expectativas `Y_e`, `gamma_e` e `pi_e` armazenadas atualmente pertencem ao passo que acabou de ser concluído. A produção realizada já foi acrescentada a `model.agg.Y` por `Bit.set_gross_domestic_product!(model)`.

No próximo passo, `Bit.set_growth_inflation_expectations!(model)` usará o contador incrementado para incluir essa nova observação no histórico disponível.

Na célula de código, `expected_simulation_time` representa apenas o resultado esperado da verificação.

Campos atualizados e usos posteriores A função atualiza somente:

- `model.agg.t`.

Ela não altera `model.agg.Y` nem qualquer outro campo econômico.

O novo contador será usado:

- na formação das expectativas do próximo período;
- na posição dos históricos de produção e inflação;
- na aplicação de choques com duração temporal;
- em `Bit.collect_data!(model)`, que registra `agg.t` em `collection_time`.

Não há denominadores, valores de ponto flutuante ou sorteios nessa função.

Atenção: a função não é idempotente. Reexecutar a célula incrementará `t` novamente sem acrescentar as observações correspondentes aos históricos, podendo desalinhá-los e causar erro no próximo cálculo de expectativas.

```
[51]: # Save the updated field and the history that must remain unchanged
simulation_time_before = model.agg.t
output_history_before = copy(model.agg.Y)

expected_simulation_time =
    simulation_time_before + 1

# Execute the model function exactly once
Bit.set_time!(model)

@assert model.agg.t == expected_simulation_time
@assert length(model.agg.Y) == length(output_history_before)
@assert all(isapprox.(
    model.agg.Y,
    output_history_before;
    nans = true,
))

time_summary = DataFrame(
    metric = [
        "Simulation time before",
        "Applied increment",
        "Expected simulation time",
        "Stored simulation time",
        "Output history length",
    ],
    value = [
        simulation_time_before,
        1,
        expected_simulation_time,
        model.agg.t,
        length(model.agg.Y),
    ],
)
```



```
time_summary
```

```
[51]:
```

	metric	value
	String	Int64
1	Simulation time before	1
2	Applied increment	1
3	Expected simulation time	2
4	Stored simulation time	2
5	Output history length	55

```
[ ]:
```